VOLUMEN 1 · EDICIÓN PARA MACOS

Fundamentos desde cero

La terminal, Git y GitHub, y JavaScript, explicados pieza a pieza: qué es cada cosa, por qué existe y qué hace cada comando

Para quién: alguien que empieza y quiere entender, no copiar

Cómo está escrito: cada concepto desde su principio, cada comando desmontado, cada resultado mostrado tal y como lo verás en tu pantalla

Incluye: ejercicios con soluciones explicadas y un proyecto final que junta todo

Preparado para Álvaro · Septiembre de 2026

Índice

PARTE 0 · ANTES DE EMPEZAR 4

1 Cómo leer este manual 5

2 Lo que hay que saber de un ordenador 7

3 Preparar tu Mac 9

PARTE 1 · LA TERMINAL 14

4 Qué es la terminal y cómo se lee un comando 15

5 Moverse por las carpetas 19

6 Crear, copiar, mover y borrar 23

7 Leer y buscar dentro de los ficheros 28

8 Encadenar comandos 32

9 Procesos y puertos 38

10 Variables de entorno 41

Ejercicios · Parte 1 44

Soluciones · Parte 1 46

PARTE 2 · GIT Y GITHUB 51

11 El problema y la idea 52

12 Dónde se guarda todo 54

13 Tu primer repositorio, comando a comando 56

14 Ramas 63

15 Conflictos 67

16 Deshacer cosas 71

17 GitHub 73

18 Lo que nunca debe subirse 78

19 Git sin terminal 80

Ejercicios · Parte 2 81

Soluciones · Parte 2 83

PARTE 3 · JAVASCRIPT DESDE CERO 89

20 Qué es JavaScript y dónde se ejecuta 90

21 Valores y tipos 93

22 Variables 95

23 Trabajar con texto 99

24 Trabajar con números 102

25 Leer los errores 105

26 Tomar decisiones 107

27 Arrays: listas de valores 110

28 Objetos: fichas con campos 113
29 Valor y referencia 115
30 Funciones 118
31 Bucles 121
Ejercicios · Parte 3 124
Soluciones · Parte 3 127

PARTE 4 · PROYECTO FINAL: TODO JUNTO 132

32 La calculadora de presupuestos 133
33 Guardarlo y publicarlo 139

APÉNDICES · PARA TENER A MANO 142

A Chuleta de terminal 143
B Chuleta de Git y GitHub 145
C Chuleta de JavaScript 147
D Glosario 149
E Qué viene en el Volumen 2 151

PARTE 0

Antes de empezar

Tres capítulos cortos para que nada de lo que venga después te pille sin base: cómo leer este manual, lo mínimo que hay que saber de cómo organiza las cosas un ordenador, y cómo dejar tu Mac listo para trabajar.

- 1. Cómo leer este manual
- 2. Lo que hay que saber de un ordenador
- 3. Preparar tu Mac

1. Cómo leer este manual

Este volumen tiene una regla: nada aparece sin explicar antes qué es, para qué existe y qué hace cada una de sus piezas. Si en algún momento ves algo que no se ha explicado, es un fallo del manual, no tuyo.

1.1 Por qué este volumen es distinto

La mayoría del material para aprender a programar te enseña *qué escribir*. Aquí el objetivo es que entiendas *por qué se escribe así*. La diferencia importa mucho: quien memoriza comandos se queda bloqueado en cuanto algo sale distinto a lo previsto; quien entiende qué hace cada pieza puede averiguar el resto por su cuenta.

Por eso cada tema sigue siempre el mismo orden:

1. **Qué es**, contado con una comparación de la vida normal.
2. **Por qué existe**: qué problema había antes y cómo lo resuelve.
3. **Cómo se usa**, con cada comando desmontado pieza a pieza.
4. **Qué vas a ver** en tu pantalla, copiado de una ejecución real.
5. **Qué puede salir mal**, con los errores reales y cómo leerlos.

1.2 Las marcas visuales

Vas a encontrar siempre los mismos bloques. Conviene reconocerlos de un vistazo.

Lo que **tú escribes** aparece en un recuadro claro con esta etiqueta arriba a la derecha:

TÚ ESCRIBES

```
pwd
```

Lo que **te responde el ordenador** aparece en un recuadro oscuro:

RESULTADO

```
/Users/alvaro
```

El código **JavaScript** (a partir de la Parte 3) no se escribe en la terminal, sino dentro de un fichero con VS Code. Aparece con esta otra etiqueta:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log("Hola");
```

Cuando lo que te responde es un **error**, el recuadro lleva una marca roja:

ERROR

```
zsh: command not found: pdw
```

Cuando un comando tiene varias piezas, aparece **desmontado**, así:

DESMONTANDO EL COMANDO

`mkdir -p proyectos/web`

`mkdir`	El programa que se ejecuta. Viene de *make directory*: crear carpeta.
`-p`	Una opción que modifica cómo se comporta el programa.
`proyectos/web`	Sobre qué actúa: la carpeta que quieres crear.

Y al final de cada idea importante, un resumen de una o dos frases:

> **IDEA CLAVE**
>
> Si solo te quedas con una cosa de cada capítulo, que sean los recuadros morados.

1.3 Cómo estudiar con él

- **Con el ordenador delante.** Cada recuadro de "Tú escribes" es para que lo escribas tú, no para leerlo.
- **Escribiendo, no copiando.** Copiar y pegar un comando no te enseña nada. Teclearlo te obliga a fijarte en cada carácter, y los errores que cometas al teclear son justo los que te enseñan a leer los mensajes de error.
- **Prediciendo antes de pulsar Intro.** Antes de ejecutar algo, piensa qué va a pasar. Luego compruébalo. Si acertaste, lo has entendido. Si no, ahí hay algo que aprender.
- **Sin saltarte las soluciones.** Al final de cada parte hay ejercicios y, después, sus soluciones explicadas. Intenta primero; lee la solución después aunque te haya salido, porque explica el porqué.

2. Lo que hay que saber de un ordenador

Antes de dar órdenes a un ordenador conviene saber cómo organiza las cosas. Son cinco ideas, y todas las usarás a diario.

2.1 Un programa es una lista de instrucciones

Un programa es un conjunto de instrucciones que el ordenador sigue en orden, una detrás de otra. Safari es un programa. Spotify es un programa. La calculadora es un programa. Y lo que vas a aprender a hacer es escribir tus propias listas de instrucciones.

La comparación más útil es una receta de cocina. Una receta dice "pela la cebolla, córtala, échala en la sartén". Si cambias el orden, el resultado cambia. Si una instrucción es ambigua ("échale un poco de sal"), una persona lo interpreta; un ordenador no puede. Por eso programar exige ser exacto: el ordenador hace *literalmente* lo que pone, ni más ni menos.

> **IDEA CLAVE**
>
> El ordenador no adivina lo que querías decir. Hace exactamente lo que escribiste. Casi todos los errores de quien empieza vienen de aquí.

2.2 Ficheros y carpetas

Todo lo que guardas en el ordenador es un **fichero** (también se dice *archivo*; es lo mismo): una foto, una canción, un documento, un programa. Y los ficheros se organizan en **carpetas** (también llamadas *directorios*; otra vez, lo mismo).

Las carpetas pueden contener otras carpetas, que a su vez contienen otras. Eso forma un **árbol**:

```
/                        ← la raíz: la carpeta que lo contiene todo
├── Applications/
├── Users/
│   └── alvaro/          ← tu carpeta personal
│       ├── Desktop/     ← el Escritorio
│       ├── Documents/   ← Documentos
│       ├── Downloads/   ← Descargas
│       └── Music/
└── System/
```

Cuando en el Finder haces doble clic en una carpeta para entrar, y luego en otra, y luego en otra, estás bajando por ese árbol. En la terminal harás exactamente lo mismo, pero escribiendo.

2.3 La ruta: la dirección de un fichero

Cada fichero tiene una **ruta**: la lista de carpetas que hay que atravesar para llegar a él, separadas por barras `/`. Es como una dirección postal.

```
/Users/alvaro/Documents/apuntes.txt
```

Se lee de izquierda a derecha: "desde la raíz `/`, entra en `Users`, luego en `alvaro`, luego en `Documents`, y ahí está el fichero `apuntes.txt`".

Hay dos tipos de ruta, y la diferencia es importante:

Tipo	Cómo empieza	Comparación
Absoluta	Por `/`	Una dirección completa: "Calle Mayor 5, Santander". Funciona estés donde estés.
Relativa	Por cualquier otra cosa	Unas indicaciones: "la segunda puerta a la derecha". Solo tiene sentido desde donde estás ahora.

Si estás dentro de `/Users/alvaro`, la ruta relativa `Documents/apuntes.txt` y la absoluta `/Users/alvaro/Documents/apuntes.txt` apuntan al mismo fichero.

2.4 La extensión: el apellido del fichero

Lo que va después del último punto en el nombre de un fichero es su **extensión**. Le dice al sistema qué tipo de contenido tiene y con qué programa abrirlo:

Extensión	Qué contiene
`.txt`	Texto plano, sin formato
`.md`	Texto con formato sencillo (Markdown). Los README de GitHub
`.js`	Código JavaScript
`.html` / `.css`	Páginas web y sus estilos
`.json`	Datos estructurados (lo verás mucho)

Todo el código que vas a escribir es **texto plano**: letras y símbolos, sin negritas ni fuentes ni colores. Por eso no se escribe en Word ni en Pages, que añaden formato por debajo. Se escribe en un **editor de código**, que es un editor de texto plano con ayudas para programar. Usaremos VS Code.

2.5 Los ficheros ocultos

Los ficheros cuyo nombre empieza por un punto (`.git`, `.env`, `.zshrc`) están **ocultos**: el Finder no los enseña por defecto. No son secretos ni peligrosos; se ocultan porque son de configuración y normalmente no quieres verlos.

> **TRUCO DE MAC**
>
> En cualquier ventana del Finder, pulsa `Cmd` + `Shift` + `.` (punto) para mostrar u ocultar los ficheros ocultos. Te va a servir en cuanto lleguemos a Git.

3. Preparar tu Mac

Vas a instalar cinco herramientas. Para cada una verás qué es, por qué la necesitas, cómo se instala y cómo comprobar que ha funcionado. Hazlo en este orden: cada una depende de las anteriores.

Herramienta	Qué es, en una frase
Terminal	La ventana donde das órdenes escritas al Mac. Ya la tienes.
Homebrew	Un instalador de programas que se maneja desde la terminal.
Git	El programa que guarda el historial de versiones de tus proyectos.
Node.js	El programa que ejecuta JavaScript fuera del navegador.
VS Code	El editor donde escribirás el código.

3.1 Abrir la Terminal

La Terminal ya viene instalada en todos los Mac. Para abrirla, la forma más rápida es con Spotlight:

1. Pulsa `Cmd` + `Espacio`.
2. Escribe `terminal`.
3. Pulsa `Intro`.

Se abre una ventana con una línea parecida a esta:

RESULTADO

```
alvaro@MacBook-Air ~ %
```

Eso se llama el **prompt** (la "invitación"): la terminal te está diciendo que está lista para recibir una orden. Lo desmontamos en el capítulo 4. Por ahora, basta con saber que lo que escribas aparecerá a la derecha del `%`, y que para ejecutarlo se pulsa `Intro`.

> **DÉJALA A MANO**
>
> Con la Terminal abierta, haz clic derecho en su icono del Dock → Opciones → *Mantener en el Dock*. La vas a abrir todos los días.

3.2 Homebrew: el instalador

Qué es. Homebrew es una "tienda de aplicaciones" que se maneja escribiendo. En lugar de buscar una web, descargar un fichero, arrastrarlo a Aplicaciones y aceptar avisos, escribes `brew install nombre` y se encarga de todo.

Por qué lo necesitas. Casi todas las herramientas de programación se instalan así. Además, con Homebrew actualizar todo es un solo comando, y desinstalar no deja restos.

Cómo se instala. Entra en `brew.sh` desde el navegador y copia el comando que aparece en la portada. Es este:

TÚ ESCRIBES

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Esta es la única vez en todo el manual que te voy a pedir que copies y pegues un comando en lugar de escribirlo, porque es largo y cualquier error lo estropea. Aun así, merece la pena entender qué hace:

DESMONTANDO EL COMANDO

`/bin/bash -c "$(curl -fsSL https://...install.sh)"`

`curl -fsSL https://...`	Descarga de internet el programa de instalación de Homebrew. `curl` es un programa que baja cosas de una dirección web.
`$( ... )`	"Ejecuta lo de dentro y pon aquí su resultado". Es decir: el texto del instalador que acaba de descargar.
`/bin/bash -c "..."`	Ejecuta ese texto como una lista de instrucciones.

Traducido: *"descarga el instalador de Homebrew y ejecútalo"*.

Durante la instalación pasarán tres cosas que conviene conocer de antemano:

1. **Te pedirá tu contraseña del Mac.** Al escribirla *no verás nada en pantalla*, ni puntos ni asteriscos. Es normal: la terminal oculta las contraseñas por completo. Escríbela y pulsa `Intro`.
2. **Te pedirá confirmar** pulsando `Intro`.
3. **Puede instalar las "Command Line Tools" de Apple**, que tardan varios minutos. Déjalo terminar.

EL PASO QUE CASI TODO EL MUNDO SE SALTA

Al terminar, el instalador escribe un apartado llamado **"Next steps"** con dos o tres comandos. En los Mac con procesador Apple (M1, M2, M3, M4) son obligatorios: sin ellos, el Mac no encuentra Homebrew. Son parecidos a estos:

TÚ ESCRIBES

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Copia los que te muestre *tu* pantalla, que son los correctos para tu máquina. Qué hacen exactamente lo entenderás en el capítulo 10 (variables de entorno); por ahora, basta con saber que le dicen al Mac dónde está Homebrew.

Cómo comprobar que ha funcionado. Cierra la Terminal, ábrela de nuevo y escribe:

TÚ ESCRIBES

```
brew --version
```

RESULTADO

```
Homebrew 4.4.2
```

El número exacto da igual. Si sale un número de versión, está instalado. Si sale esto:

ERROR

```
zsh: command not found: brew
```

...es que no se hizo el paso de "Next steps". Vuelve a él.

> **EL PATRÓN DE COMPROBACIÓN**
>
> Casi cualquier programa responde a `nombre --version` diciendo qué versión tiene. Es la forma estándar de comprobar si algo está instalado. La vas a usar con todas las herramientas de este capítulo.

3.3 Git

Qué es. El programa que guarda el historial de cambios de tus proyectos. Tiene una parte entera de este manual (la Parte 2), así que aquí solo lo instalamos.

TÚ ESCRIBES

```
brew install git
```

Verás varias líneas de progreso. Al terminar, comprueba:

TÚ ESCRIBES

```
git --version
```

RESULTADO

```
git version 2.46.0
```

Ahora dile a Git quién eres. Esto se hace **una sola vez en la vida** en cada ordenador, y sirve para que cada cambio que guardes lleve tu nombre:

TÚ ESCRIBES

```
git config --global user.name "Álvaro Crespo"
git config --global user.email "tu-correo@ejemplo.com"
```

DESMONTANDO EL COMANDO

`git config --global user.name "Álvaro Crespo"`

`git`	El programa.
`config`	La acción: "configurar". Git tiene muchas acciones, y esta es la que cambia ajustes.
`--global`	Para todos tus proyectos, no solo para uno.
`user.name`	El ajuste que cambias: el nombre de usuario.
`"Álvaro Crespo"`	El valor. Va entre comillas porque tiene un espacio; sin ellas, Git pensaría que "Crespo" es otra cosa distinta.

Usa el mismo correo con el que vayas a crear tu cuenta de GitHub. Si no coinciden, GitHub no sabrá que esos cambios son tuyos.

Para comprobarlo, pide el valor sin dar uno nuevo:

TÚ ESCRIBES

```
git config --global user.name
```

RESULTADO

```
Álvaro Crespo
```

3.4 Node.js

Qué es. JavaScript nació para funcionar dentro del navegador. Node.js es un programa que permite ejecutarlo en cualquier otro sitio: en tu terminal, en un servidor. Es lo que usarás en la Parte 3 para ejecutar tus propios programas.

TÚ ESCRIBES

```
brew install node
```

Comprueba las dos cosas que instala:

TÚ ESCRIBES

```
node --version
npm --version
```

RESULTADO

```
v22.9.0
10.8.3
```

`npm` es el gestor de paquetes de Node: sirve para instalar librerías de código escritas por otras personas. Viene incluido y lo usarás más adelante.

3.5 VS Code

Qué es. El editor de código que usa la mayoría del sector. Es gratuito, de Microsoft.

TÚ ESCRIBES

```
brew install --cask visual-studio-code
```

El `--cask` se usa para programas con ventana e icono (los que normalmente arrastrarías a Aplicaciones). Sin él, Homebrew instala herramientas de terminal.

Ábrelo desde Aplicaciones o Spotlight. Luego activa una cosa muy útil: poder abrirlo desde la terminal.

1. En VS Code, pulsa `Cmd` + `Shift` + `P`. Se abre una barra de búsqueda de órdenes.
2. Escribe `shell command`.
3. Elige *Shell Command: Install 'code' command in PATH*.

Desde ese momento, en la terminal podrás escribir `code .` para abrir la carpeta en la que estés dentro de VS Code. El punto significa "esta carpeta"; lo verás en el capítulo 5.

3.6 La comprobación final

Cierra la Terminal, ábrela de nuevo, y ejecuta estas cuatro líneas:

TÚ ESCRIBES

```
brew --version
git --version
node --version
code --version
```

Si las cuatro responden con un número de versión, tu Mac está listo. Si alguna responde `command not found`, vuelve a su apartado: casi siempre es un paso de instalación que no terminó.

> **IDEA CLAVE**
>
> Homebrew instala, Git guarda versiones, Node ejecuta JavaScript y VS Code es donde escribes. Y el comando `--version` es tu forma de preguntar "¿estás instalado?".

PARTE 1

La terminal

La terminal es una forma de hablar con el ordenador escribiendo en lugar de haciendo clic. Al principio parece antigua e incómoda. Al terminar esta parte entenderás por qué la usa todo el mundo que programa.

- 4. Qué es la terminal y cómo se lee un comando
- 5. Moverse por las carpetas
- 6. Crear, copiar, mover y borrar
- 7. Leer y buscar dentro de los ficheros
- 8. Encadenar comandos
- 9. Procesos y puertos
- 10. Variables de entorno

4. Qué es la terminal y cómo se lee un comando

4.1 Qué es

Cuando usas el Finder, haces clic en una carpeta y se abre. Por debajo, el Finder le está diciendo al sistema operativo "abre esta carpeta y enséñame lo que hay". Lo único que hace el Finder es traducir tus clics a órdenes.

La terminal te deja dar esas órdenes **directamente, escribiéndolas**. Sin intermediario.

Técnicamente, la ventana se llama *Terminal*, y el programa que hay dentro, el que lee lo que escribes y lo ejecuta, se llama **shell**. En los Mac actuales ese programa se llama **zsh** (se pronuncia "zeta shell"). Lo menciono porque lo verás en algunos mensajes de error: cuando algo empieza por `zsh:`, es la shell quien te habla.

4.2 Por qué existe, si ya hay ratón

Buena pregunta, y la respuesta tiene tres partes:

1. **Hace cosas que los botones no hacen.** Un programa con ventana solo te ofrece los botones que su diseñador pensó. En la terminal tienes acceso a todo.
2. **Es repetible.** Renombrar 400 ficheros con el ratón es una tarde. En la terminal es una línea, y la puedes guardar para repetirla mañana.
3. **Es lo único que hay en un servidor.** Las máquinas donde viven las webs y las aplicaciones no tienen pantalla ni ratón. Solo terminal. Cuando publiques tu primer proyecto, lo harás desde aquí.

Y hay una cuarta razón, más práctica: todas las herramientas de programación (Git, Node, npm) se manejan principalmente escribiendo. No es una preferencia: es cómo funcionan.

4.3 El prompt

Al abrir la Terminal ves esto:

RESULTADO

```
alvaro@MacBook-Air ~ %
```

DESMONTANDO EL COMANDO

`alvaro@MacBook-Air ~ %`

`alvaro`	Tu nombre de usuario en el Mac.
`@MacBook-Air`	El nombre del ordenador. La `@` se lee "en": "alvaro en MacBook-Air".
`~`	La carpeta en la que estás ahora mismo. `~` es la abreviatura de tu carpeta personal (`/Users/alvaro`).
`%`	El final del prompt. A partir de aquí escribes tú.

Lo más importante de esa línea es la parte central: **la terminal siempre está "dentro" de una carpeta**, igual que una ventana del Finder siempre está mostrando una carpeta. Todo lo que hagas ocurrirá ahí, salvo que digas otra cosa. Esa carpeta se llama el **directorio de trabajo**.

> **NO COPIES EL PROMPT**
>
> Cuando veas comandos en internet, a veces aparecen con un `$` o un `%` delante: `$ ls`. Ese símbolo representa el prompt; **no se escribe**. Solo se escribe lo que va detrás.

4.4 La anatomía de un comando

Aquí está la idea más útil de toda la Parte 1. **Todos los comandos tienen la misma estructura**, y una vez la ves, cualquier comando se vuelve legible:

```
programa   -opciones   argumentos
```

Pieza	Qué es	Comparación
Programa	Qué herramienta usas. Siempre va primero.	El verbo: "corta".
Opciones	Cómo quieres que se comporte. Empiezan por `-` o `--`. Son opcionales.	El adverbio: "en trozos pequeños".
Argumentos	Sobre qué actúa.	El objeto: "la cebolla".

Un ejemplo real:

DESMONTANDO EL COMANDO

`ls -l Documents`

`ls`	Programa: "listar" lo que hay en una carpeta.
`-l`	Opción: en formato largo, con detalles (tamaño, fecha...).
`Documents`	Argumento: la carpeta que quieres listar.

Traducido: *"lista, con detalles, lo que hay en Documents"*.

Las piezas **se separan con espacios**. Por eso los espacios importan tanto en la terminal: `ls-l` (sin espacio) no es "ls con la opción -l", es un programa llamado `ls-l`, que no existe.

4.5 Opciones cortas y largas

Las opciones vienen en dos formatos:

- **Cortas**: un guion y una letra. `-l`, `-a`. Se pueden juntar: `-la` es lo mismo que `-l -a`.
- **Largas**: dos guiones y una palabra. `--version`, `--help`. Son más fáciles de leer y no se juntan.

Muchas opciones tienen las dos formas, y hacen lo mismo. Por ejemplo, en Git, `-m` y `--message` son idénticas.

4.6 Los trucos que te ahorran horas

Tecla	Qué hace	Por qué importa
`Tab`	Autocompleta nombres de ficheros, carpetas y comandos	Escribes `cd Doc`, pulsas Tab, y aparece `cd Documents/`. Evita erratas y es más rápido. Úsalo **siempre**.
`↑` / `↓`	Recorre los comandos que has escrito antes	Para repetir o corregir un comando sin reescribirlo entero.
`Ctrl` + `C`	Interrumpe lo que se esté ejecutando	Tu botón de emergencia. Si algo se queda colgado, esto lo para. Ojo: es **Ctrl**, no Cmd.
`Cmd` + `K`	Limpia la pantalla	Cuando tienes demasiado texto encima.
`Ctrl` + `A` / `Ctrl` + `E`	Va al principio / final de la línea	El ratón no funciona para moverte dentro de un comando.

4.7 Cómo pedir ayuda al propio ordenador

No hace falta memorizar las opciones de cada programa. Casi todos tienen su propio manual incorporado:

TÚ ESCRIBES

```
man ls
```

`man` viene de *manual*. Abre las instrucciones completas de `ls`. Te mueves con las flechas, buscas escribiendo `/palabra` y **sales pulsando** `q` (de *quit*). Esto último es importante: la primera vez que alguien abre `man`, no sabe cómo salir.

Para programas modernos como Git o Node, la ayuda rápida es `--help`:

TÚ ESCRIBES

```
git --help
```

4.8 El primer error

Escribe esto, a propósito mal:

TÚ ESCRIBES

```
pdw
```

ERROR

```
zsh: command not found: pdw
```

Léelo por partes: `zsh:` es la shell la que habla; `command not found` significa "no encuentro ningún programa con ese nombre"; `pdw` es lo que escribiste. Es el error más frecuente de todos, y casi siempre significa una de dos cosas: **una errata**, o **un programa que no está instalado**.

> **IDEA CLAVE**
>
> Todo comando es `programa -opciones argumentos`, separados por espacios. La terminal siempre está dentro de una carpeta. Y `Tab` es tu mejor amigo.

5. Moverse por las carpetas

Lo primero que hay que saber hacer en la terminal es lo mismo que haces en el Finder: saber dónde estás, ver qué hay y entrar y salir de carpetas. Son tres comandos.

5.1 pwd — ¿dónde estoy?

TÚ ESCRIBES

```
pwd
```

RESULTADO

```
/Users/alvaro
```

`pwd` significa *print working directory*: "escribe el directorio de trabajo". Te dice la ruta completa de la carpeta en la que estás. No tiene opciones ni argumentos que necesites. Úsalo cada vez que te pierdas.

5.2 ls — ¿qué hay aquí?

TÚ ESCRIBES

```
ls
```

RESULTADO

```
Applications  Desktop  Documents  Downloads  Library  Movies  Music  Pictures  Public
```

`ls` viene de *list*. Lista lo que hay en la carpeta actual. Es exactamente lo que verías al abrir tu carpeta personal en el Finder.

Tiene dos opciones que usarás constantemente:

TÚ ESCRIBES

```
ls -la
```

RESULTADO

```
total 24
drwxr-x---+  15 alvaro  staff   480 28 sep 10:12 .
drwxr-xr-x    5 root    admin   160 12 sep 09:01 ..
-rw-------    1 alvaro  staff  2413 28 sep 10:12 .zsh_history
-rw-r--r--    1 alvaro  staff    94 20 sep 18:30 .zprofile
drwx------+   4 alvaro  staff   128 25 sep 11:02 Desktop
drwx------+   6 alvaro  staff   192 27 sep 16:44 Documents
...
```

DESMONTANDO EL COMANDO

`ls -la`

`-l`	*long*: formato largo. Una fila por elemento, con permisos, dueño, tamaño y fecha.
`-a`	*all*: muestra también los ocultos (los que empiezan por punto).

No hace falta que entiendas todas las columnas ahora. Fíjate solo en dos cosas: la **primera letra** de cada fila (`d` significa *directory*, carpeta; `-` significa fichero normal) y la **última columna**, que es el nombre.

Y fíjate en las dos primeras filas, que se llaman `.` y `..`. Son especiales, y las explicamos ahora mismo.

5.3 cd — entrar y salir

`cd` viene de *change directory*: "cambiar de carpeta". Es el doble clic de la terminal.

TÚ ESCRIBES

```
cd Documents
pwd
```

RESULTADO

```
/Users/alvaro/Documents
```

Has entrado. Observa que el prompt también ha cambiado: ahora pone `Documents` donde antes ponía `~`. Para volver atrás:

TÚ ESCRIBES

```
cd ..
pwd
```

RESULTADO

```
/Users/alvaro
```

5.4 Los atajos de ruta

Estos cuatro símbolos funcionan en cualquier comando que reciba una ruta, no solo en `cd`:

Símbolo	Significa	Ejemplo
`.`	La carpeta actual	`code .` → abre en VS Code la carpeta donde estás
`..`	La carpeta de arriba (la "madre")	`cd ..` → sube un nivel
`~`	Tu carpeta personal	`cd ~` → vuelve a casa desde cualquier sitio
`-`	La carpeta en la que estabas antes	`cd -` → como el botón "atrás"

Se pueden combinar. `cd ../..` sube dos niveles. `cd ~/Documents` va a Documentos desde donde estés.

Y `cd` a secas, sin nada detrás, también te lleva a tu carpeta personal.

5.5 Rutas absolutas y relativas, en la práctica

Recuerda la diferencia del capítulo 2. Supón que estás en `/Users/alvaro`:

TÚ ESCRIBES

```
cd Documents                  # relativa: "entra en Documents, que está aquí"
cd /Users/alvaro/Documents    # absoluta: "ve a esta dirección exacta"
```

Las dos te llevan al mismo sitio. La diferencia aparece si estás en otro lugar: la relativa fallaría, porque desde otra carpeta no hay un `Documents` "aquí", mientras que la absoluta funciona siempre.

> **QUÉ ES ESO DE `#`**
>
> En la terminal, todo lo que va detrás de un `#` es un **comentario**: una nota para humanos que la terminal ignora. En este manual lo uso para explicar al lado de cada comando qué hace. No hace falta que lo escribas tú.

5.6 Errores típicos

TÚ ESCRIBES

```
cd Documentos
```

ERROR

```
cd: no such file or directory: Documentos
```

"No existe ese fichero o carpeta". En macOS, aunque el Finder te enseñe "Documentos" en español, la carpeta de verdad se llama `Documents`, en inglés. Lo mismo con `Desktop` (Escritorio), `Downloads` (Descargas) y `Music` (Música). El Finder solo traduce el nombre en pantalla.

TÚ ESCRIBES

```
cd documents
```

En macOS esto *suele* funcionar aunque el nombre real tenga mayúscula, porque el disco del Mac ignora mayúsculas por defecto. Pero no te acostumbres: en los servidores (Linux) las mayúsculas sí importan, y `documents` y `Documents` son dos carpetas distintas. Escribe siempre el nombre exacto, y deja que `Tab` lo complete por ti.

TÚ ESCRIBES

```
cd Mis Proyectos
```

Si una carpeta tiene espacios en el nombre, esto falla: la terminal entiende "entra en `Mis`" y luego no sabe qué hacer con `Proyectos`. Recuerda que el espacio separa las piezas. Hay dos soluciones:

TÚ ESCRIBES

```
cd "Mis Proyectos"        # entre comillas: todo es un solo argumento
cd Mis\ Proyectos         # la barra invertida "protege" el espacio
```

Por esta razón, en programación **nunca se usan espacios en nombres de ficheros ni carpetas**. Se usan guiones: `mis-proyectos`.

5.7 Ver en el Finder lo que haces en la terminal

Un truco que ayuda mucho al principio:

TÚ ESCRIBES

```
open .
```

Abre en el Finder la carpeta donde estás. Así puedes comprobar con tus ojos que lo que haces en la terminal ocurre de verdad. Son el mismo disco y los mismos ficheros, vistos de dos maneras.

> **IDEA CLAVE**
>
> `pwd` te dice dónde estás, `ls` qué hay, `cd` te mueve. `.` es aquí, `..` es arriba, `~` es casa. Y nada de espacios en los nombres.

6. Crear, copiar, mover y borrar

6.1 Preparar un sitio para practicar

Antes de nada, vamos a crear una carpeta para hacer pruebas, así no tocas nada importante:

TÚ ESCRIBES

```
cd ~
mkdir practica
cd practica
pwd
```

RESULTADO

```
/Users/alvaro/practica
```

A partir de aquí, todo lo del capítulo lo haremos dentro de `practica`.

6.2 mkdir — crear carpetas

`mkdir` viene de *make directory*. Crea una carpeta nueva:

TÚ ESCRIBES

```
mkdir apuntes
ls
```

RESULTADO

```
apuntes
```

Ahora intenta crear una carpeta dentro de otra que todavía no existe:

TÚ ESCRIBES

```
mkdir web/css
```

ERROR

```
mkdir: web: No such file or directory
```

Falla porque `web` no existe, y `mkdir`, por defecto, solo crea el último nivel. Para eso existe `-p`:

DESMONTANDO EL COMANDO

`mkdir -p web/css`

`mkdir`	Crear carpeta.
`-p`	*parents*: crea también las carpetas intermedias ("madres") que falten. Y si ya existen, no se queja.
`web/css`	La ruta completa a crear.

6.3 touch — crear ficheros vacíos

TÚ ESCRIBES

```
touch notas.txt
ls
```

RESULTADO

```
apuntes  notas.txt  web
```

`touch` ("tocar") existe originalmente para actualizar la fecha de un fichero, como si lo tocaras. Pero si el fichero no existe, lo crea vacío. En la práctica, se usa sobre todo para eso: crear ficheros vacíos rápido.

6.4 La expansión de llaves: crear muchos a la vez

Aquí viene una de las cosas más potentes de la terminal. Mira:

TÚ ESCRIBES

```
touch nota{1..5}.txt
ls
```

RESULTADO

```
apuntes    nota1.txt  nota2.txt  nota3.txt  nota4.txt  nota5.txt  notas.txt  web
```

Cinco ficheros con un comando. ¿Qué ha pasado? La clave es entender que **la shell reescribe tu comando antes de ejecutarlo**. Cuando ve `{1..5}`, lo sustituye por todos los números del 1 al 5. El comando que de verdad se ejecuta es este:

```
touch nota1.txt nota2.txt nota3.txt nota4.txt nota5.txt
```

`touch` nunca llega a ver las llaves. Recibe cinco argumentos y crea cinco ficheros.

Para ver qué va a generar una expansión *sin ejecutar nada*, usa `echo`, que simplemente escribe en pantalla lo que le des:

TÚ ESCRIBES

```
echo nota{1..5}.txt
```

RESULTADO

```
nota1.txt nota2.txt nota3.txt nota4.txt nota5.txt
```

Hay dos tipos de llaves:

Forma	Genera	Ejemplo
`{1..5}`	Un rango de números o letras	`{a..d}` → `a b c d`
`{x,y,z}`	Una lista, separada por comas	`foto.{jpg,png}` → `foto.jpg foto.png`

Y se combinan con lo anterior. Esto crea una estructura de proyecto entera en una línea:

TÚ ESCRIBES

```
mkdir -p proyecto/{src/componentes,tests,docs}
```

`mkdir` no dice nada si funciona. Para ver lo que ha creado, usa `find`, que lista una carpeta y todo lo que tiene dentro, a cualquier profundidad:

TÚ ESCRIBES

```
find proyecto
```

RESULTADO

```
proyecto
proyecto/docs
proyecto/src
proyecto/src/componentes
proyecto/tests
```

6.5 cp — copiar

DESMONTANDO EL COMANDO

`cp notas.txt copia-notas.txt`

`cp`	*copy*: copiar.
`notas.txt`	Primer argumento: el **origen**, lo que copias.
`copia-notas.txt`	Segundo argumento: el **destino**, cómo se llamará la copia.

El orden es siempre **origen → destino**. Si el destino es una carpeta existente, la copia se mete dentro con el mismo nombre:

TÚ ESCRIBES

```
cp notas.txt apuntes/
```

Para copiar una carpeta entera con todo lo que contiene hace falta `-r` (*recursive*: "y todo lo de dentro, y lo de dentro de eso…"):

TÚ ESCRIBES

```
cp -r web web-copia
```

6.6 mv — mover y renombrar

`mv` viene de *move*. Y aquí hay algo que sorprende: **en la terminal, renombrar y mover son la misma operación**.

TÚ ESCRIBES

```
mv notas.txt apuntes.txt        # renombrar: mismo sitio, otro nombre
mv apuntes.txt apuntes/         # mover: otro sitio, mismo nombre
```

Tiene sentido si lo piensas en términos de rutas: renombrar es cambiar la última parte de la ruta, y mover es cambiar la parte de delante. En los dos casos, el fichero pasa de una ruta a otra.

6.7 rm — borrar

`rm` viene de *remove*.

TÚ ESCRIBES

```
rm copia-notas.txt
```

No sale nada. En la terminal, **cuando un comando funciona, normalmente no dice nada**. El silencio es buena señal. Solo te habla si algo va mal.

> **RM NO TIENE PAPELERA**
>
> Lo que borras con `rm` no va a la Papelera. Desaparece. No hay "deshacer", no hay confirmación, no hay forma sencilla de recuperarlo. Tenlo muy presente.

Para borrar carpetas:

TÚ ESCRIBES

```
rmdir apuntes       # solo si está vacía
rm -r web-copia     # con todo su contenido
```

6.8 Los comodines

El asterisco `*` significa "cualquier cosa, de cualquier longitud":

TÚ ESCRIBES

```
ls *.txt
```

RESULTADO

```
nota1.txt  nota2.txt  nota3.txt  nota4.txt  nota5.txt
```

Igual que con las llaves, **la shell sustituye el `*` antes de ejecutar** por todos los nombres que encajan. `ls` recibe la lista ya hecha.

Y aquí está el peligro. Esto borra todo lo que acabe en `.txt` de esta carpeta:

TÚ ESCRIBES

```
rm *.txt
```

Si te equivocas de carpeta, o de patrón, has borrado cosas que no querías. Y no hay vuelta atrás.

LA COSTUMBRE QUE TE SALVARÁ

Antes de cualquier `rm` con comodines, ejecuta el mismo patrón con `ls` o `echo` para ver exactamente qué va a afectar:

TÚ ESCRIBES

```
ls *.txt       # primero: ¿qué va a borrar?
rm *.txt       # después, si la lista es la que esperabas
```

6.9 Resumen del capítulo

Comando	Viene de	Hace
`mkdir -p ruta`	make directory	Crea carpetas, incluidas las intermedias
`touch fichero`	touch	Crea un fichero vacío
`cp origen destino`	copy	Copia. `-r` para carpetas
`mv origen destino`	move	Mueve o renombra
`rm fichero`	remove	Borra para siempre. `-r` para carpetas
`{1..5}`, `{a,b}`	—	Genera varias palabras antes de ejecutar
`*`	—	Encaja con cualquier texto

7. Leer y buscar dentro de los ficheros

7.1 Un fichero para practicar

Necesitamos un fichero con texto dentro. Vamos a crearlo con un comando que te servirá mucho:

TÚ ESCRIBES

```
cat > musica.txt
```

La terminal se queda esperando, sin prompt. Está esperando que escribas el contenido. Escribe estas líneas, pulsando `Intro` al final de cada una:

TÚ ESCRIBES

```
techno berlinés de los noventa
house de Chicago
techno de Detroit
minimal alemán
ERROR: formato no reconocido
house progresivo
Error al leer la pista
techno industrial
```

Cuando termines, pulsa `Ctrl`+`D`. Eso significa "fin del texto" y cierra el fichero. Vuelve el prompt.

El símbolo `>` significa "manda el resultado a este fichero en vez de a la pantalla". Lo explicamos a fondo en el capítulo 8. Por ahora, quédate con que `cat > fichero` es una forma rápida de escribir un fichero de texto sin abrir ningún editor.

7.2 cat — ver el contenido entero

TÚ ESCRIBES

```
cat musica.txt
```

RESULTADO

```
techno berlinés de los noventa
house de Chicago
techno de Detroit
minimal alemán
ERROR: formato no reconocido
house progresivo
Error al leer la pista
techno industrial
```

`cat` viene de *concatenate* ("encadenar"), porque su función original era juntar varios ficheros en uno: `cat a.txt b.txt` los escribe uno detrás de otro. Con uno solo, simplemente lo muestra.

Es perfecto para ficheros cortos. Para uno de 50.000 líneas es inútil: te lo vuelca todo de golpe.

7.3 head y tail — el principio y el final

TÚ ESCRIBES

```
head -n 3 musica.txt
```

RESULTADO

```
techno berlinés de los noventa
house de Chicago
techno de Detroit
```

TÚ ESCRIBES

```
tail -n 2 musica.txt
```

RESULTADO

```
Error al leer la pista
techno industrial
```

`head` ("cabeza") muestra las primeras líneas y `tail` ("cola") las últimas. `-n 3` dice cuántas. Si no lo pones, son 10.

¿Para qué sirven? Imagina un fichero de registro (*log*) de un servidor, donde se van apuntando todos los eventos, uno por línea. Tiene medio millón de líneas. Lo que te interesa es qué ha pasado **en los últimos minutos**, que está al final. Eso es `tail`.

Y `tail -f` (de *follow*, "seguir") no termina: se queda mirando el fichero y te enseña cada línea nueva en cuanto se escribe. Es como ver el registro en directo. Se sale con `Ctrl`+`C`.

7.4 less — leer ficheros largos

TÚ ESCRIBES

```
less musica.txt
```

Abre el fichero en un visor donde puedes moverte con las flechas, avanzar páginas con `Espacio`, buscar con `/palabra` y **salir con** `q`. Es el mismo visor que usa `man`.

7.5 grep — buscar dentro de los ficheros

Este es uno de los comandos más útiles que existen.

De dónde viene el nombre. En los años 70 se usaba un editor de texto llamado `ed`. Para buscar se escribía `g/re/p`, que significaba: **g**lobal (en todo el fichero), **r**egular **e**xpression (este patrón), **p**rint (imprime las líneas que lo contengan). Se usaba tanto que se convirtió en un programa independiente con ese nombre: `grep`.

Qué hace. Lee un fichero línea por línea y, de cada una, se pregunta: "¿contiene lo que me han pedido?". Si sí, la escribe. Si no, la descarta.

DESMONTANDO EL COMANDO

`grep "techno" musica.txt`

`grep`	El programa: buscar.
`"techno"`	Qué buscar. Las comillas no son obligatorias con una sola palabra, pero sí si hay espacios. Mejor ponerlas siempre.
`musica.txt`	Dónde buscar.

RESULTADO

```
techno berlinés de los noventa
techno de Detroit
techno industrial
```

De ocho líneas, se ha quedado con las tres que contienen "techno".

Ahora busca errores:

TÚ ESCRIBES

```
grep "error" musica.txt
```

RESULTADO

```
```

¡Nada! Porque en el fichero pone `ERROR` y `Error`, pero no `error` en minúsculas. `grep` distingue mayúsculas de minúsculas. Para que no lo haga, está la opción `-i` (*ignore case*):

TÚ ESCRIBES

```
grep -i "error" musica.txt
```

RESULTADO

```
ERROR: formato no reconocido
Error al leer la pista
```

Las opciones de `grep` que más vas a usar:

Opción	Viene de	Hace
`-i`	ignore case	No distingue mayúsculas y minúsculas
`-n`	number	Dice en qué número de línea está cada resultado
`-c`	count	Solo dice cuántas líneas coinciden, no cuáles
`-v`	invert	Al revés: las líneas que **no** contienen el patrón
`-r`	recursive	Busca en todos los ficheros de una carpeta y sus subcarpetas

TÚ ESCRIBES

```
grep -n "house" musica.txt
```

RESULTADO

```
2:house de Chicago
6:house progresivo
```

El número antes de los dos puntos es la línea donde está.

7.6 Por qué grep es tan importante al programar

Cuando trabajes en un proyecto de verdad, tendrá cientos o miles de ficheros. Alguien te dirá "hay un fallo en la función `calcularTotal`", y la primera pregunta será: ¿en qué fichero está?

TÚ ESCRIBES

```
grep -rn "calcularTotal" src/
```

RESULTADO

```
src/carrito.js:14:function calcularTotal(lineas) {
src/pedidos.js:52:  const total = calcularTotal(pedido.lineas);
src/pedidos.js:88:  return calcularTotal(items) * 1.21;
```

En un segundo sabes dónde está definida (línea 14 de `carrito.js`) y desde dónde se usa. `-r` busca dentro de toda la carpeta `src`, y `-n` te da el número de línea. El formato de cada resultado es siempre `fichero:línea:contenido`.

IDEA CLAVE

`cat` muestra todo, `head` y `tail` el principio y el final, `less` te deja navegar. `grep` se queda solo con las líneas que contienen lo que buscas, y con `-rn` encuentra cualquier cosa en un proyecto entero.

8. Encadenar comandos

Este capítulo explica la idea que hace que la terminal sea tan potente: los comandos pequeños se pueden conectar entre sí como piezas de una cadena de montaje.

8.1 La idea: salida y entrada

Todo comando tiene una **entrada** (lo que recibe) y una **salida** (lo que produce). Hasta ahora, la salida siempre iba a la pantalla. Pero se puede mandar a otros sitios.

Hay dos destinos alternativos, y cada uno tiene su símbolo:

Símbolo	Nombre	Manda la salida a…
`>` y `>>`	Redirección	…un **fichero**
`|`	Tubería (*pipe*)	…**otro comando**

8.2 Redirección: guardar el resultado en un fichero

TÚ ESCRIBES

```
grep "techno" musica.txt > solo-techno.txt
```

No sale nada en pantalla. ¿Dónde ha ido el resultado? Al fichero:

TÚ ESCRIBES

```
cat solo-techno.txt
```

RESULTADO

```
techno berlinés de los noventa
techno de Detroit
techno industrial
```

Hay dos versiones, y la diferencia es muy importante:

Símbolo	Si el fichero no existe	Si el fichero ya existe
`>`	Lo crea	**Lo borra y lo sustituye**
`>>`	Lo crea	Añade al final, sin borrar nada

TÚ ESCRIBES

```
echo "house de Nueva York" >> musica.txt      # añade una línea al final
```

> **CUIDADO CON EL > SIMPLE**
>
> `echo "hola" > musica.txt` dejaría el fichero con una sola línea: "hola". Todo lo demás, perdido. Cuando quieras añadir, `>>`. Siempre.

Y la dirección contraria: `<` hace que un comando **lea** de un fichero en lugar de esperar que escribas tú.

8.3 La tubería: conectar comandos

El símbolo `|` (en el teclado español de Mac, `⌥ Opción` + `1`) conecta la salida de un comando con la entrada del siguiente.

Piensa en una **cadena de montaje**: cada trabajador hace una sola cosa y le pasa el resultado al siguiente. Ninguno sabe lo que hacen los demás. Solo reciben algo, lo transforman y lo pasan.

TÚ ESCRIBES

```
grep "techno" musica.txt | wc -l
```

RESULTADO

```
3
```

Qué ha pasado, paso a paso:

1. `grep "techno" musica.txt` produce las tres líneas con "techno". Pero no las manda a la pantalla…
2. …porque la `|` se las pasa a `wc -l`.
3. `wc` viene de *word count* ("contar palabras"), y `-l` le dice que cuente **líneas**. Recibe tres, y escribe `3`.

Resultado: "¿cuántas líneas contienen techno?" → 3.

8.4 Tres herramientas para las cadenas

Antes del ejemplo grande, tres programas que casi siempre aparecen en las cadenas:

`sort` ordena líneas alfabéticamente. Con `-n` ordena como números; con `-r`, al revés.

`uniq` elimina líneas repetidas *que estén seguidas*. Con `-c` (*count*), además pone delante cuántas veces aparecía. Lo de "seguidas" es fundamental, ahora verás por qué.

`tr` (*translate*) cambia unos caracteres por otros. Por ejemplo, todos los espacios por saltos de línea, o las mayúsculas por minúsculas.

8.5 El ejemplo completo: las palabras más repetidas

El problema: tienes un texto y quieres saber qué palabras se repiten más. El ordenador no sabe qué es "una palabra": para él, un fichero son líneas de texto. Hay que ir preparándolo paso a paso.

Vamos a construirlo **añadiendo una pieza cada vez y mirando qué sale**. Es así como se construyen estas cadenas en la vida real. Crea primero este fichero:

TÚ ESCRIBES

```
cat > texto.txt
El perro y el gato.
El gato duerme; el perro corre.
LA casa es grande, la casa es azul.
```

(y `Ctrl`+`D` para terminar).

PIEZA 1 — UNA PALABRA POR LÍNEA

TÚ ESCRIBES

```
tr -s '[:space:]' '\n' < texto.txt
```

RESULTADO

```
El
perro
y
el
gato.
El
gato
duerme;
...
```

DESMONTANDO EL COMANDO

`tr -s '[:space:]' '\n' < texto.txt`

`tr`	Cambia unos caracteres por otros.
`'[:space:]'`	Qué cambiar: todos los "espacios en blanco" (espacios, tabuladores, saltos de línea). Es un grupo ya definido; no hace falta listarlos.
`'\n'`	Por qué cambiarlos: por un salto de línea. `\n` es la forma de escribir "salto de línea".
`-s`	*squeeze* ("apretar"): si hay varios espacios seguidos, cuenta como uno. Sin esto saldrían líneas vacías.
`< texto.txt`	Lee de este fichero.

Por qué es el primer paso: todas las herramientas que vienen después trabajan **línea a línea**. Así que lo primero es convertir cada palabra en una línea. Este paso traduce tu problema al idioma que entienden las demás piezas.

PIEZA 2 — TODO A MINÚSCULAS

TÚ ESCRIBES

```
tr -s '[:space:]' '\n' < texto.txt | tr '[:upper:]' '[:lower:]'
```

Ahora `El` se ha convertido en `el` y `LA` en `la`. **Por qué:** para ti son la misma palabra, pero para el ordenador `El` y `el` son textos distintos y los contaría por separado.

PIEZA 3 — FUERA LA PUNTUACIÓN

TÚ ESCRIBES

```
... | tr -d '[:punct:]'
```

`-d` (*delete*) borra en lugar de cambiar. `[:punct:]` es el grupo de los signos de puntuación. **Por qué:** `gato.` y `gato` contarían como dos palabras distintas.

PIEZA 4 — ORDENAR

TÚ ESCRIBES

```
... | sort
```

RESULTADO

```
azul
casa
casa
corre
duerme
el
el
el
el
es
es
gato
gato
...
```

Aquí está el paso que menos se entiende. No ordenamos porque queramos el resultado ordenado. Ordenamos porque así **todas las repeticiones de una palabra quedan juntas**, pegadas unas a otras. Y eso lo necesita el siguiente paso.

PIEZA 5 — AGRUPAR Y CONTAR

TÚ ESCRIBES

```
... | uniq -c
```

RESULTADO

```
      1 azul
      2 casa
      1 corre
      1 duerme
      4 el
      2 es
      2 gato
...
```

`uniq -c` junta las líneas repetidas en una y pone delante cuántas había.

Y ahora se entiende por qué hacía falta `sort`. `uniq` solo compara cada línea con la que tiene justo encima. No tiene memoria del resto del fichero. Mira qué pasa si quitas el `sort`:

RESULTADO

```
      1 el
      1 perro
      1 y
      1 el        ← otra vez "el", contado aparte
      1 gato
      1 el        ← y otra
...
```

Como las repeticiones están desperdigadas, `uniq` no las ve. Por eso `sort | uniq -c` van siempre juntos. Verás esta pareja cientos de veces.

PIEZA 6 — ORDENAR POR NÚMERO, DE MAYOR A MENOR

TÚ ESCRIBES

```
... | sort -rn
```

RESULTADO

```
      4 el
      2 perro
      2 la
      2 gato
      2 es
      2 casa
      1 y
...
```

Otra vez `sort`, pero ahora el número está al principio de cada línea. `-n` hace que lo trate como número (sin esto, `10` iría antes que `4`, porque el carácter "1" va antes que el "4"). `-r` invierte el orden: de mayor a menor.

PIEZA 7 — QUEDARSE CON LAS PRIMERAS

TÚ ESCRIBES

```
... | head -5
```

Y ya está. El comando completo:

TÚ ESCRIBES

```
tr -s '[:space:]' '\n' < texto.txt | tr '[:upper:]' '[:lower:]' | tr -d '[:punct:]' | sort |
uniq -c | sort -rn | head -5
```

RESULTADO

```
      4 el
      2 perro
      2 la
      2 gato
      2 es
```

Leído en español: *"parte el texto en palabras, pásalas a minúsculas, quítales la puntuación, ordénalas para que las iguales queden juntas, cuéntalas, ordena por el recuento de mayor a menor, y dame las cinco primeras"*.

CÓMO SE CONSTRUYE UNA CADENA

Nadie escribe esto de una vez. Se escribe la primera pieza, se mira la salida, se añade la segunda, se mira, y así. Si algo sale raro, quitas la última pieza y ves dónde se torció. Pulsa `↑` para recuperar el comando anterior y añadirle una pieza más, sin reescribirlo.

SI TU MAC DICE "ILLEGAL BYTE SEQUENCE"

Es un problema de `tr` con los acentos cuando el idioma de la terminal no está bien configurado. Se arregla ejecutando esto antes:

TÚ ESCRIBES

```
export LC_ALL=es_ES.UTF-8
```

Qué significa `export` lo verás en el capítulo 10.

IDEA CLAVE

`>` manda la salida a un fichero (borrándolo), `>>` la añade al final, y `|` se la pasa a otro comando. Programas pequeños, cada uno con una tarea, conectados en cadena: esa es la filosofía de la terminal.

9. Procesos y puertos

9.1 Qué es un proceso

Un programa guardado en el disco es como una receta en un libro: instrucciones quietas. Cuando lo ejecutas, el sistema lo carga en memoria y empieza a seguir sus instrucciones. Ese programa *en marcha* se llama **proceso**.

Puedes tener un programa y varios procesos de él: si abres tres ventanas de Terminal, hay tres procesos del mismo programa.

Cada proceso recibe un número único mientras está vivo: el **PID** (*process identifier*, "identificador de proceso"). Es como el número de ticket de la charcutería: sirve para referirse exactamente a ese proceso y no a otro.

9.2 Ver los procesos

TÚ ESCRIBES

```
ps aux | grep node
```

`ps` (*process status*) lista los procesos. `aux` son tres opciones juntas que hacen que salgan todos, con detalles. Como la lista es enorme, se filtra con `grep`. ¿Ves? Ya estás usando tuberías para cosas reales.

La forma visual de hacer esto en Mac es el **Monitor de Actividad** (búscalo con Spotlight). Enseña exactamente la misma información: cada proceso con su PID.

9.3 Parar un proceso

Hay dos maneras:

- **Si está en primer plano** (ocupando tu terminal): `Ctrl`+`C`.
- **Si está en otra ventana o en segundo plano**: con su PID.

TÚ ESCRIBES

```
kill 4821
```

`kill` ("matar") le pide al proceso 4821 que termine. Le pide educadamente: el proceso puede cerrar sus cosas antes de irse. Si no hace caso, se le obliga:

TÚ ESCRIBES

```
kill -9 4821
```

El `-9` es la señal de terminación forzosa. El proceso muere en el acto, sin guardar nada. Úsalo solo si el normal no funciona.

9.4 Qué es un puerto

Tu ordenador tiene una sola dirección de red, pero puede tener muchos programas comunicándose por internet a la vez: el navegador, Spotify, un servidor que estés probando… ¿Cómo sabe a qué programa va cada mensaje?

Por el **puerto**. Piensa en un edificio de pisos: la dirección de la calle es la misma para todos, pero cada vecino tiene su número de puerta. La dirección es el ordenador; el puerto es la puerta.

Son números del 0 al 65535. Algunos tienen uso habitual:

Puerto	Uso habitual
80	Webs sin cifrar (http)
443	Webs cifradas (https)
3000, 5173, 8080	Servidores que montas tú mientras desarrollas
5432	Base de datos PostgreSQL

La regla que importa: **en cada puerto solo puede escuchar un proceso a la vez**. Dos vecinos no pueden tener la misma puerta.

9.5 Práctica: ocupar un puerto y encontrar quién lo ocupa

Vas a necesitar **dos ventanas de Terminal**. Abre una segunda con `Cmd` + `N`.

En la primera ventana, arranca un servidor web de prueba:

TÚ ESCRIBES

```
python3 -m http.server 8080
```

RESULTADO

```
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
```

La terminal se queda "colgada". No es un error: el servidor está en marcha, esperando visitas. Esa ventana está ocupada mientras el servidor viva.

Abre en el navegador `http://localhost:8080`. Verás la lista de ficheros de la carpeta donde lo arrancaste. Acabas de montar un servidor web: un programa que escucha en un puerto y responde a quien llama. `localhost` significa "este mismo ordenador".

En la segunda ventana, pregunta quién está usando el puerto 8080:

DESMONTANDO EL COMANDO

`lsof -i :8080`

`lsof`	*list open files*: lista lo que tienen abierto los procesos. En macOS y Linux, las conexiones de red cuentan como "ficheros abiertos".
`-i`	Solo conexiones de red (*internet*).
`:8080`	Solo las del puerto 8080.

RESULTADO

```
COMMAND    PID    USER   FD   TYPE  DEVICE  SIZE/OFF NODE NAME
Python   48213  alvaro    4u  IPv6  0x8f...      0t0  TCP *:http-alt (LISTEN)
```

Las dos columnas que importan: **COMMAND** (qué programa: Python) y **PID** (su número: 48213). `LISTEN` al final significa que está escuchando. `http-alt` es simplemente el nombre que el sistema da al puerto 8080.

Ahora, desde la segunda ventana, mátalo con *tu* PID:

TÚ ESCRIBES

```
kill 48213
```

Mira la primera ventana: el servidor se ha parado y vuelve el prompt. Comprueba que el puerto está libre:

TÚ ESCRIBES

```
lsof -i :8080
```

No sale nada. En la terminal, **cuando `lsof` no encuentra nada, no dice nada**. No es un error: significa "ahí no hay nadie".

9.6 Por qué esto te va a servir muy pronto

Cuando empieces a hacer servidores con Node, un día te encontrarás esto:

ERROR

```
Error: listen EADDRINUSE: address already in use :::3000
```

Traducción: "la dirección ya está en uso". Hay otro proceso en el puerto 3000, casi siempre uno tuyo que dejaste abierto en otra ventana. Ya sabes resolverlo:

TÚ ESCRIBES

```
lsof -i :3000        # ¿quién está ahí?
kill 48213           # fuera (con el PID que te salga)
```

IDEA CLAVE

Un proceso es un programa en marcha y tiene un número (PID). Un puerto es la puerta por la que un proceso se comunica, y solo cabe uno por puerta. `lsof -i :puerto` te dice quién la ocupa y `kill PID` lo echa.

10. Variables de entorno

10.1 Primero: qué es una variable

Una variable es **un nombre que apunta a un valor**. Nada más.

La mejor comparación son los contactos del móvil. En tu agenda tienes `Mamá → 666 123 456`. Cuando quieres llamar, no marcas nueve dígitos: dices "llamar a Mamá". El nombre apunta al número.

Eso tiene dos ventajas, que son exactamente las razones por las que existen las variables:

1. **No necesitas recordar el valor.** Usas un nombre que tiene sentido.
2. **Si el valor cambia, lo cambias en un solo sitio.** Si el número cambia, editas el contacto una vez y todas las llamadas futuras funcionan.

Este concepto es el mismo en la terminal y en JavaScript. Aquí lo vemos en la terminal; en la Parte 3, en JavaScript.

10.2 Qué son las variables de entorno

Son variables que guarda el **sistema** (no un programa concreto) y que están disponibles para cualquier programa que arranques desde la terminal. Piensa en ellas como un tablón de anuncios compartido: cualquier programa puede leerlo.

Tu Mac ya tiene muchas. Para ver el valor de una, se pone `$` delante del nombre:

TÚ ESCRIBES

```
echo $HOME
```

RESULTADO

```
/Users/alvaro
```

El `$` significa "dame el valor de". Compara:

TÚ ESCRIBES

```
echo HOME
```

RESULTADO

```
HOME
```

Sin el `$`, `echo` escribe la palabra "HOME" tal cual. Con el `$`, la shell la sustituye por su valor *antes* de ejecutar, igual que hacía con las llaves y el asterisco.

Algunas que ya existen:

Variable	Contiene
`$HOME`	La ruta de tu carpeta personal. Por eso `~` funciona: es un atajo para esto.
`$USER`	Tu nombre de usuario
`$SHELL`	Qué shell usas (`/bin/zsh`)
`$PATH`	Dónde buscar programas. La explicamos ahora

10.3 PATH: cómo encuentra el Mac los programas

Cuando escribes `git` , ¿cómo sabe la terminal dónde está ese programa? Porque mira en `$PATH` :

TÚ ESCRIBES

```
echo $PATH
```

RESULTADO

```
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

Es una lista de carpetas separadas por dos puntos. Cuando escribes un comando, la shell lo busca en la primera carpeta; si no está, en la segunda; y así. Si llega al final sin encontrarlo, responde `command not found` .

Y ahora se entiende el paso de "Next steps" de Homebrew del capítulo 3. Homebrew instala los programas en `/opt/homebrew/bin` , y ese comando añadía esa carpeta a tu `PATH` . Sin él, los programas estaban instalados, pero la terminal no miraba ahí.

10.4 Crear tus propias variables

TÚ ESCRIBES

```
export CIUDAD="Santander"
echo $CIUDAD
```

RESULTADO

```
Santander
```

DESMONTANDO EL COMANDO

`export CIUDAD="Santander"`

`export`	Crea la variable y la pone en el "tablón compartido", para que la vean los programas que arranques después.
`CIUDAD`	El nombre. Por costumbre, las variables de entorno van en mayúsculas.
`=`	Asignación: "guarda esto aquí". **Sin espacios alrededor.** `CIUDAD = "x"` falla.
`"Santander"`	El valor.

Pero duran poco. Cierra la ventana, abre otra, y `echo $CIUDAD` saldrá vacío. Las variables creadas así solo viven en esa sesión.

Para que sean permanentes, hay que escribirlas en un fichero que la shell lee cada vez que se abre: `~/.zshrc` (o `~/.zprofile`, que fue el que usó Homebrew). Son ficheros ocultos (empiezan por punto) en tu carpeta personal. Cada vez que abres una Terminal, zsh los lee y ejecuta lo que hay dentro.

10.5 Por qué esto es tan importante al programar

Aquí está la razón de fondo, y la que te preguntarán en una entrevista.

Imagina que tu aplicación necesita la contraseña de su base de datos. Podrías escribirla directamente en el código. Pero ese código acabará en GitHub, a la vista de cualquiera, con tu contraseña dentro.

La solución es que el código **no contenga la contraseña, sino que la pida al entorno**:

TÚ ESCRIBES

```
const password = process.env.DB_PASSWORD;
```

(Eso es JavaScript; `process.env` es la forma de leer variables de entorno desde Node.) El código ya no sabe cuál es la contraseña: solo sabe dónde preguntarla. En tu Mac vale una cosa, en el servidor otra, y el secreto nunca se sube a ningún sitio.

En los proyectos, estas variables se suelen guardar en un fichero llamado `.env`, que se excluye de Git expresamente. Lo verás en la Parte 2.

> **IDEA CLAVE**
>
> Una variable es un nombre que apunta a un valor. Las de entorno las guarda el sistema y las leen todos los programas. `$NOMBRE` da su valor, `export` crea una, y `PATH` es la lista de sitios donde el Mac busca programas. Los secretos van en variables de entorno, nunca en el código.

Ejercicios · Parte 1

Hazlos en orden, en tu carpeta `~/practica`. Todos se resuelven con lo visto en los capítulos 4 a 10. Intenta cada uno antes de mirar la solución, y predice qué va a salir antes de pulsar Intro.

EJERCICIO 1.1 · ORIENTARSE

Sin abrir el Finder, averigua cuántos elementos hay en tu carpeta de Descargas. Pista: hay un comando que cuenta líneas.

EJERCICIO 1.2 · UNA ESTRUCTURA DE UN SOLO GOLPE

Con **un solo comando**, crea esta estructura dentro de `practica`:

```
musica/
├── sets/
├── produccion/
│   ├── proyectos/
│   └── samples/
└── flyers/
```

Luego comprueba con otro comando que se ha creado bien.

EJERCICIO 1.3 · CREAR, MOVER Y BORRAR EN MASA

1. Crea con un solo comando diez ficheros llamados `set01.txt`, `set02.txt`… hasta `set10.txt` (fíjate: con el cero delante).
2. Muévelos todos a `musica/sets` con un solo comando.
3. Borra solo los que van del `set06` al `set10`, comprobando antes qué vas a borrar.

EJERCICIO 1.4 · RENOMBRAR

Cambia el nombre de la carpeta `musica/flyers` a `musica/carteles`.

EJERCICIO 1.5 · BUSCAR

Crea un fichero `generos.txt` con estas líneas: `techno`, `house`, `Techno`, `minimal`, `house`, `techno`. Después:

1. Cuenta cuántas líneas contienen "techno", sin importar mayúsculas.
2. Muestra las líneas que **no** contienen "house".

EJERCICIO 1.6 · LA CADENA COMPLETA

Descarga el Quijote entero con este comando:

TÚ ESCRIBES

```
curl -sL -o quijote.txt https://www.gutenberg.org/ebooks/2000.txt.utf-8
```

Después, averigua cuáles son sus diez palabras más repetidas, y guarda el resultado en un fichero llamado `top.txt`.

EJERCICIO 1.7 · AÑADIR SIN BORRAR

Añade al final de `top.txt` una línea con la fecha y hora de hoy, sin borrar lo que ya tiene. Pista: el comando `date` escribe la fecha.

EJERCICIO 1.8 · LIBERAR UN PUERTO

Arranca un servidor de prueba en el puerto 3000. Después, desde otra ventana, averigua su PID y páralo.

EJERCICIO 1.9 · VARIABLES

1. Crea una variable de entorno `PRACTICA` que contenga la ruta completa de tu carpeta de práctica.
2. Ve a tu carpeta personal, y desde ahí vuelve a la de práctica usando la variable.
3. Abre una ventana nueva de Terminal y prueba lo mismo. ¿Qué pasa, y por qué?

Soluciones · Parte 1

Cada solución explica no solo qué escribir, sino por qué se escribe así. Léelas aunque te hayan salido.

1.1 · Cuántos elementos hay en Descargas

TÚ ESCRIBES

```
ls ~/Downloads | wc -l
```

RESULTADO

```
      47
```

`ls ~/Downloads` lista lo que hay en Descargas: `~` es tu carpeta personal, así que funciona estés donde estés. Cuando la salida de `ls` va por una tubería en lugar de a la pantalla, escribe **un elemento por línea**. Y `wc -l` cuenta líneas. Por tanto: un elemento por línea, contar líneas, igual a contar elementos.

Fíjate en que no hiciste `cd`: le diste a `ls` la ruta directamente. Casi todos los comandos aceptan una ruta, así que no hace falta moverse a una carpeta para mirar dentro.

1.2 · La estructura

TÚ ESCRIBES

```
mkdir -p musica/{sets,produccion/{proyectos,samples},flyers}
find musica
```

RESULTADO

```
musica
musica/flyers
musica/produccion
musica/produccion/proyectos
musica/produccion/samples
musica/sets
```

Aquí hay llaves **dentro** de llaves. La shell las expande de dentro hacia fuera: `produccion/{proyectos,samples}` se convierte en dos rutas, y luego el conjunto exterior genera las cuatro finales:

```
musica/sets  musica/produccion/proyectos  musica/produccion/samples  musica/flyers
```

El `-p` es imprescindible: sin él, `mkdir` no podría crear `musica/sets`, porque `musica` aún no existe. Con él, crea todos los niveles que falten.

Truco: si dudas de lo que va a generar una expansión, escribe `echo` delante y lo ves sin ejecutar nada.

1.3 · Crear, mover y borrar en masa

TÚ ESCRIBES

```
touch set{01..10}.txt
mv set*.txt musica/sets/
ls musica/sets/set{06..10}.txt
rm musica/sets/set{06..10}.txt
```

Línea 1. Si escribes el rango como `{01..10}`, con el cero delante, la shell mantiene ese formato para todos: `01, 02 ... 10`. Con `{1..10}` saldrían `set1, set2 ... set10`.

Por qué importa el cero: los ordenadores ordenan texto carácter a carácter. Sin el cero, `set10` aparecería entre `set1` y `set2`, porque "1" va antes que "2". Con el cero, todos tienen dos cifras y se ordenan bien. Es una costumbre muy útil para nombrar ficheros.

Línea 2. `set*.txt` encaja con todo lo que empiece por "set" y acabe en ".txt". La shell lo sustituye por la lista de los diez nombres, y `mv` los recibe todos. Cuando el último argumento de `mv` es una carpeta, mete todo lo anterior dentro.

Líneas 3 y 4. Primero se comprueba con `ls` qué va a afectar el patrón, y solo después se borra con `rm` *usando exactamente el mismo patrón*. Esa costumbre es la que evita desastres.

1.4 · Renombrar

TÚ ESCRIBES

```
mv musica/flyers musica/carteles
```

En la terminal, renombrar es mover a una ruta con otro nombre. `mv` no distingue entre "mover" y "renombrar": en los dos casos cambia la ruta del elemento. Funciona igual con carpetas que con ficheros, y en este caso no hace falta `-r` (a diferencia de `cp`), porque mover no copia el contenido, solo cambia la dirección.

1.5 · Buscar

TÚ ESCRIBES

```
cat > generos.txt
techno
house
Techno
minimal
house
techno
```

(Y `Ctrl`+`D`.)

TÚ ESCRIBES

```
grep -ic "techno" generos.txt
```

RESULTADO

```
3
```

`-i` ignora mayúsculas (por eso cuenta también `Techno`) y `-c` cuenta en lugar de mostrar. Se pueden juntar en `-ic`. Sin `-i`, el resultado sería 2.

TÚ ESCRIBES

```
grep -v "house" generos.txt
```

RESULTADO

```
techno
Techno
minimal
techno
```

`-v` invierte la búsqueda: muestra lo que **no** coincide. Es muy útil para quitar "ruido", por ejemplo para ver un registro sin las líneas de información rutinaria.

1.6 · La cadena completa sobre el Quijote

DESMONTANDO EL COMANDO

`curl -sL -o quijote.txt https://www.gutenberg.org/ebooks/2000.txt.utf-8`

`curl`	Descarga lo que hay en una dirección web.
`-s`	*silent*: sin barra de progreso.
`-L`	Sigue las redirecciones. Esa dirección te reenvía a otra donde está el fichero real, y sin `-L`, `curl` se quedaría con el aviso de reenvío en lugar del libro.
`-o quijote.txt`	*output*: guarda lo descargado en este fichero en lugar de volcarlo en pantalla.

Comprueba que ha llegado entero:

TÚ ESCRIBES

```
wc -l quijote.txt
```

Deberían ser decenas de miles de líneas. Ahora, la cadena del capítulo 8 con dos cambios: el nombre del fichero y `head -10`. Y al final, redirigida a un fichero:

TÚ ESCRIBES

```
tr -s '[:space:]' '\n' < quijote.txt | tr '[:upper:]' '[:lower:]' | tr -d '[:punct:]' | sort |
uniq -c | sort -rn | head -10 > top.txt
cat top.txt
```

Verás que las más repetidas son palabras como *que*, *de*, *y*, *la*, *a*, *en*, *el*, *no*… Tiene sentido: en cualquier texto largo en español, las palabras más frecuentes son las que unen las frases, no las que tienen significado. En análisis de texto se llaman **palabras vacías** y se suelen filtrar. Esto se usa de verdad en buscadores y en análisis de datos.

El `> top.txt` del final demuestra que la redirección y las tuberías se combinan: toda la cadena se ejecuta y solo el resultado final va al fichero.

1.7 · Añadir sin borrar

TÚ ESCRIBES

```
date >> top.txt
cat top.txt
```

Con `>>` la fecha se añade al final. Si hubieras usado `>`, el fichero se habría quedado solo con la fecha: se habría perdido el ranking entero. Es el error clásico que conviene cometer ahora, en un fichero de práctica, y no más tarde en uno importante.

1.8 · Liberar un puerto

En la primera ventana:

TÚ ESCRIBES

```
python3 -m http.server 3000
```

En la segunda:

TÚ ESCRIBES

```
lsof -i :3000
```

RESULTADO

```
COMMAND    PID    USER   FD   TYPE  DEVICE  SIZE/OFF NODE NAME
Python   51877  alvaro    4u  IPv6  0x7c...      0t0  TCP *:hbci (LISTEN)
```

TÚ ESCRIBES

```
kill 51877
```

(Con el PID que te salga a ti, que será distinto.) Y el nombre raro del final, `hbci`, es simplemente el nombre histórico que el sistema asigna al puerto 3000. No significa nada para ti.

Versión avanzada, en una sola línea:

TÚ ESCRIBES

```
kill $(lsof -t -i :3000)
```

`-t` hace que `lsof` devuelva *solo* el número de PID, sin tabla. Y `$( )` significa "ejecuta lo de dentro y pon aquí su resultado" (lo viste en la instalación de Homebrew). Así que la shell primero ejecuta `lsof -t -i :3000`, obtiene `51877`, y luego ejecuta `kill 51877`.

Fíjate en la diferencia con la tubería: `|` pasa la salida como *entrada* del siguiente programa; `$( )` la pone como *argumento* dentro del comando. `kill` necesita el PID como argumento, por eso aquí se usa `$( )` y no `|`.

1.9 · Variables

TÚ ESCRIBES

```
export PRACTICA="$HOME/practica"
cd ~
cd "$PRACTICA"
pwd
```

RESULTADO

```
/Users/alvaro/practica
```

Dos detalles. Primero, dentro del valor usamos `$HOME` en lugar de escribir `/Users/alvaro`: así funcionaría en el Mac de cualquier persona. Segundo, al usar la variable la ponemos entre comillas, `"$PRACTICA"`: si la ruta tuviera espacios, las comillas evitan que la shell la parta en trozos. Es una buena costumbre aunque ahora no haga falta.

En una ventana nueva:

TÚ ESCRIBES

```
echo $PRACTICA
```

RESULTADO

```

```

Sale vacío. Las variables creadas con `export` solo viven en la sesión donde las creaste; cada ventana nueva empieza de cero. Para hacerla permanente, habría que añadir esa línea `export` al fichero `~/.zshrc`, que zsh lee cada vez que se abre una ventana:

TÚ ESCRIBES

```
echo 'export PRACTICA="$HOME/practica"' >> ~/.zshrc
```

Fíjate en las comillas simples por fuera: evitan que `$HOME` se sustituya en este momento, de modo que en el fichero se guarda literalmente `$HOME` y se sustituye cada vez que se lea. Y fíjate en el `>>`: con un `>` simple habrías borrado toda tu configuración.

PARTE 2

Git y GitHub

Git guarda el historial de tu trabajo; GitHub lo pone en internet. Son las dos herramientas que se dan por supuestas en cualquier empleo de programación. Esta parte empieza por el problema que resuelven, porque sin entenderlo los comandos no tienen sentido.

- 11. El problema y la idea
- 12. Dónde se guarda todo
- 13. Tu primer repositorio, comando a comando
- 14. Ramas
- 15. Conflictos
- 16. Deshacer cosas
- 17. GitHub
- 18. Lo que nunca debe subirse
- 19. Git sin terminal

11. El problema y la idea

11.1 Cómo se trabaja sin control de versiones

Imagina que estás haciendo un proyecto. Funciona. Quieres añadirle algo, pero te da miedo estropear lo que ya va bien. ¿Qué haces? Lo que ha hecho todo el mundo alguna vez:

```
proyecto/
proyecto_copia/
proyecto_final/
proyecto_final_v2/
proyecto_final_BUENO/
proyecto_final_BUENO_este_sí/
```

Esto tiene tres problemas que no se pueden arreglar por ese camino:

1. **No sabes qué cambió** entre una copia y otra, salvo comparando fichero a fichero.
2. **No sabes por qué** cambió. Dentro de tres meses no te acordarás.
3. **Si trabajan dos personas**, una acaba pisando el trabajo de la otra. Sin remedio.

Un **sistema de control de versiones** resuelve los tres. Y Git es el que usa prácticamente todo el mundo. Lo creó en 2005 Linus Torvalds, el creador de Linux, para coordinar a miles de personas que tocaban el mismo código a la vez.

11.2 La idea central: fotografías del proyecto

Aquí está el concepto del que depende todo lo demás. **Git no guarda tus ficheros uno por uno. Guarda fotografías del proyecto entero, en los momentos que tú eliges.**

Cada una de esas fotografías se llama **commit** (en español a veces se dice "confirmación", pero todo el mundo dice commit). Y cada commit guarda cinco cosas:

1. Cómo estaba **todo** el proyecto en ese instante.
2. **Quién** lo hizo (el nombre y correo que configuraste en el capítulo 3).
3. **Cuándo**.
4. Un **mensaje** escrito por ti explicando qué cambiaste y por qué.
5. Un **enlace al commit anterior**.

El último punto es el que lo hace funcionar. Como cada commit apunta al anterior, forman una **cadena**. Siguiendo la cadena hacia atrás, tienes la historia completa del proyecto:

```
[primer commit] ← [segundo commit] ← [tercer commit] ← [el más reciente]
```

Y puedes volver a cualquiera de esos puntos cuando quieras. Es como tener una máquina del tiempo para tu proyecto.

11.3 El hash: la matrícula de cada commit

Cada commit tiene un identificador único que se parece a esto:

```
8f4fd8229df578e974cc8fc316e0b71994603344
```

Se llama **hash**. Normalmente se usan solo los siete primeros caracteres (`8f4fd82`), que son suficientes para distinguirlo.

Lo interesante es que **no es un número al azar**: se calcula a partir del contenido del commit. Si cambia una sola letra de un solo fichero, el hash sale completamente distinto. Eso significa que nadie puede modificar el historial en secreto: cualquier cambio altera las matrículas y se nota.

> **IDEA CLAVE**
>
> Un commit es una fotografía del proyecto entero, con autor, fecha, mensaje y un enlace a la foto anterior. La cadena de commits es la historia del proyecto. Cada commit se identifica por su hash.

12. Dónde se guarda todo

12.1 La carpeta .git

Esta es la respuesta a una de las preguntas más habituales, "¿dónde se guarda lo que guardo?": **en una carpeta oculta llamada `.git`, dentro de tu propio proyecto.**

Así se ve un proyecto con Git, listando también los ocultos:

TÚ ESCRIBES

```
ls -a
```

RESULTADO

```
.  ..  .git  README.md  app.js
```

Tus ficheros son los de siempre. La única diferencia es esa carpeta `.git`. Dentro está toda la historia: todas las fotografías, todos los mensajes, todas las versiones de todos los ficheros.

Tres consecuencias que conviene tener claras:

- **Si borras `.git`, pierdes el historial.** Tus ficheros siguen ahí tal y como estén ahora, pero ya no hay versiones anteriores.
- **Si copias la carpeta del proyecto** a un USB, te llevas la historia completa con ella.
- **Git funciona entero sin internet y sin cuenta en ninguna web.** Todo ocurre en tu disco. GitHub es otra cosa, y la veremos en el capítulo 17.

12.2 Los tres espacios

Este es el modelo mental que explica cómo funcionan los comandos de Git. Tu trabajo pasa por tres sitios:

Espacio	Qué es	Comparación con una foto
1. Directorio de trabajo	Tus ficheros, tal y como están ahora en el disco. Lo que ves y editas.	La habitación, con todo lo que hay.
2. Área de preparación (*staging*)	Los cambios que has elegido para la próxima foto.	Lo que colocas delante de la cámara.
3. Repositorio	Las fotos ya tomadas. El historial permanente, dentro de `.git`.	El álbum.

Y dos comandos mueven las cosas de un espacio al siguiente:

```
Tus ficheros   ──git add──►   Preparación   ──git commit──►   Historial
(lo que editas)               (lo que elegiste)               (las fotos)
```

12.3 ¿Por qué dos pasos para guardar?

Es la duda de todo el mundo al empezar. ¿Por qué `add` y luego `commit`, en vez de un solo "guardar"?

Porque te permite elegir **qué entra en cada foto**. Imagina que en una tarde has hecho tres cosas: arreglar un fallo, cambiar un color y empezar algo nuevo que aún no funciona. Si Git lo guardara todo de golpe, tendrías un commit llamado "cambios varios" que no sirve de nada. Con el paso intermedio puedes hacer esto:

TÚ ESCRIBES

```
git add login.js
git commit -m "fix: corrige la validación del login"

git add estilos.css
git commit -m "style: cambia el color del botón"

# lo que está a medias no se añade, y se queda fuera de las fotos
```

Tres cambios, dos commits limpios, uno pendiente. Cada commit hace una sola cosa, y así el historial se puede leer y cada cambio se puede deshacer por separado.

IDEA CLAVE

El historial vive en la carpeta oculta `.git` de tu proyecto, en tu disco. Los cambios pasan de tus ficheros a la preparación con `git add`, y de la preparación al historial con `git commit`.

13. Tu primer repositorio, comando a comando

Vamos a crear un repositorio desde cero y a mirar qué dice Git en cada paso. Lo importante de este capítulo no son los comandos: es aprender a leer las respuestas de Git, porque te dicen en todo momento qué está pasando.

SI TU GIT TE HABLA EN ESPAÑOL

Según cómo esté configurado tu Mac, Git puede responder en inglés o en español. Los ejemplos de este manual están en inglés, que es lo que verás en casi toda la documentación. Si el tuyo responde en español, las frases equivalen así:

Inglés	Español (aproximado)
On branch main	En la rama main
No commits yet	No hay commits todavía
Untracked files	Archivos sin seguimiento
Changes to be committed	Cambios a ser confirmados
Changes not staged for commit	Cambios no rastreados para el commit
nothing to commit, working tree clean	nada para hacer commit, el árbol de trabajo está limpio

13.1 Crear la carpeta y encender Git

TÚ ESCRIBES

```
cd ~
mkdir mi-primer-repo
cd mi-primer-repo
git init -b main
```

RESULTADO

```
Initialized empty Git repository in /Users/alvaro/mi-primer-repo/.git/
```

DESMONTANDO EL COMANDO

`git init -b main`

`git`	El programa.
`init`	La acción: *initialize*, "inicializar". Convierte esta carpeta en un repositorio creando la carpeta `.git`.
`-b main`	Llama `main` a la línea principal de trabajo (*branch*, rama; lo vemos en el capítulo 14). Es el nombre estándar hoy.

La respuesta dice literalmente lo que ha hecho: "he creado un repositorio vacío en `.../mi-primer-repo/.git/`". Ahí tienes la carpeta oculta.

Una palabra que verás mucho: **repositorio** (o *repo*). Es simplemente una carpeta que tiene Git dentro. Un proyecto con historial.

13.2 git status: tu comando más importante

TÚ ESCRIBES

```
git status
```

RESULTADO

```
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

`git status` te dice en qué estado está todo. Úsalo constantemente: antes y después de cada cosa que hagas. Léelo línea a línea:

- *On branch main* → estás en la rama principal.
- *No commits yet* → todavía no has hecho ninguna foto.
- *nothing to commit* → no hay nada que guardar. Y te da una pista de qué hacer: crea ficheros y usa `git add`.

GIT TE DICE QUÉ HACER

Fíjate en que casi todos los mensajes de Git incluyen, entre paréntesis, el siguiente comando que podrías usar. Cuando no sepas qué hacer, ejecuta `git status` y lee los paréntesis.

13.3 Crear un fichero y ver qué opina Git

TÚ ESCRIBES

```
echo "# Mi primer repositorio" > README.md
git status
```

RESULTADO

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

La parte nueva es **Untracked files**: "ficheros sin seguimiento". Git ve que existe `README.md`, pero no lo está vigilando. Está en tu carpeta, pero no en el repositorio. Y entre paréntesis te dice cómo cambiar eso: `git add`.

El fichero se llama `README.md` por convención: es el fichero de presentación de cualquier proyecto. GitHub lo muestra automáticamente como portada.

13.4 git add: preparar para la foto

TÚ ESCRIBES

```
git add README.md
git status
```

RESULTADO

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Ha cambiado de sección. Ahora está en **Changes to be committed**: "cambios que entrarán en el próximo commit". El fichero está en el área de preparación, delante de la cámara. Y el paréntesis te dice cómo sacarlo de ahí si te equivocaste.

Las formas de usar `git add`:

Comando	Prepara
`git add fichero.js`	Un fichero concreto
`git add carpeta/`	Todo lo que hay en una carpeta
`git add .`	Todo lo que haya cambiado en la carpeta actual y sus subcarpetas (el punto es "aquí", como en el capítulo 5)

13.5 git commit: hacer la foto

TÚ ESCRIBES

```
git commit -m "docs: añade README"
```

RESULTADO

```
[main (root-commit) 8f4fd82] docs: añade README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

DESMONTANDO EL COMANDO

`git commit -m "docs: añade README"`

`commit`	Crea la foto con todo lo que hay en el área de preparación.
`-m`	*message*: el mensaje va a continuación.
`"docs: añade README"`	El mensaje. Entre comillas porque tiene espacios.

La respuesta, desmontada:

- `main` → la rama donde se ha guardado.
- `root-commit` → es el primero de la historia, el único sin padre.
- `8f4fd82` → su hash, su matrícula.
- `1 file changed, 1 insertion(+)` → un fichero cambiado, una línea añadida.

SI OLVIDAS EL -M, APARECE UNA PANTALLA RARA

Si escribes solo `git commit`, Git abre un editor dentro de la terminal para que escribas el mensaje. Casi siempre es un editor llamado **Vim**, que funciona de manera muy distinta a lo habitual y del que nadie sabe salir la primera vez. Si te pasa:

1. Pulsa `i` para poder escribir, y escribe el mensaje.
2. Pulsa `Esc`.
3. Escribe `:wq` y pulsa `Intro` (*write and quit*: guardar y salir).

Para que en su lugar se abra VS Code, que es mucho más cómodo, configúralo una vez:

TÚ ESCRIBES

```
git config --global core.editor "code --wait"
```

13.6 Cómo escribir buenos mensajes

El mensaje es para la persona que lea el historial dentro de seis meses, que probablemente serás tú. Tres reglas:

1. **Di qué hace el cambio**, no qué has tocado. Bien: "corrige el cálculo del IVA". Mal: "cambios en precios.js".
2. **En presente e imperativo**: "añade", "corrige", "elimina". Como si completaras la frase "este commit...".
3. **Nunca** "cambios", "fix", "ya funciona", "asdf".

Muchos equipos añaden un prefijo que indica el tipo de cambio. Se llama *Conventional Commits* y conviene que lo uses desde ya:

Prefijo	Cuándo
`feat:`	Algo nuevo (*feature*)
`fix:`	Corrige un fallo
`docs:`	Solo documentación
`style:`	Cambios visuales o de formato que no alteran el funcionamiento
`refactor:`	Reorganiza código sin cambiar lo que hace
`chore:`	Tareas de mantenimiento y configuración

13.7 Modificar un fichero y ver la diferencia

Ahora cambia algo en un fichero que Git ya vigila:

TÚ ESCRIBES

```
echo "Apuntes de mi formación como desarrollador." >> README.md
git status
```

RESULTADO

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

Tercera sección distinta: **Changes not staged for commit**, "cambios no preparados". Git sabe que el fichero ha cambiado desde la última foto (`modified`), pero ese cambio aún no está en la preparación.

Para ver **exactamente qué ha cambiado**:

TÚ ESCRIBES

```
git diff
```

RESULTADO

```
diff --git a/README.md b/README.md
index 144ad42..4577906 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
 # Mi primer repositorio
+Apuntes de mi formación como desarrollador.
```

La parte que importa es la del final. Las líneas con `+` delante se han añadido; las que tuvieran `-` , se habrían borrado; las que no tienen nada, están igual y aparecen como contexto. Lo de arriba es información técnica que puedes ignorar por ahora.

`diff` viene de *difference*. Úsalo antes de cada commit para revisar lo que vas a guardar.

13.8 Las tres secciones de git status, juntas

Ya has visto las tres. Son los tres estados en que puede estar un fichero, y conviene reconocerlos:

Sección	Significa	Qué hacer
Untracked files	Fichero nuevo que Git no vigila	`git add` para empezar a seguirlo
Changes not staged	Fichero vigilado que ha cambiado, pero el cambio no está preparado	`git add` para prepararlo
Changes to be committed	Cambios listos para la próxima foto	`git commit`

Y si no hay ninguna de las tres:

RESULTADO

```
On branch main
nothing to commit, working tree clean
```

Working tree clean: "el directorio de trabajo está limpio". Todo lo que hay en tu carpeta está ya fotografiado. Es el estado ideal antes de empezar algo nuevo.

13.9 git log: ver el historial

Guarda el cambio anterior y mira la historia:

TÚ ESCRIBES

```
git add README.md
git commit -m "docs: amplía la descripción del README"
git log
```

RESULTADO

```
commit 1cb9542ac8613c3348adf33a51e567847a184286
Author: Álvaro Crespo <tu-correo@ejemplo.com>
Date:   Mon Sep 28 11:03:14 2026 +0200

    docs: amplía la descripción del README

commit 8f4fd8229df578e974cc8fc316e0b71994603344
Author: Álvaro Crespo <tu-correo@ejemplo.com>
Date:   Mon Sep 28 11:02:40 2026 +0200

    docs: añade README
```

Cada bloque es una foto, con su hash completo, autor, fecha y mensaje. El más reciente arriba. Si la lista es larga, `git log` se abre en el visor de `less`: te mueves con las flechas y **sales con** `q`.

La versión compacta, que es la que usarás casi siempre:

TÚ ESCRIBES

```
git log --oneline
```

RESULTADO

```
1cb9542 docs: amplía la descripción del README
8f4fd82 docs: añade README
```

13.10 El ciclo, resumido

Todo tu trabajo diario con Git es repetir este ciclo:

TÚ ESCRIBES

```
git status                      # ¿qué ha cambiado?
git diff                        # ¿exactamente qué?
git add .                       # preparar
git commit -m "tipo: mensaje"   # fotografiar
git log --oneline               # comprobar
```

IDEA CLAVE

`git status` es tu brújula: úsalo antes y después de todo. Un fichero puede estar sin seguimiento, modificado o preparado. `git add` lo prepara, `git commit` lo guarda para siempre, y `git log` te enseña la historia.

14. Ramas

14.1 El problema

Tienes un proyecto que funciona. Quieres probar una idea nueva que va a llevar varios días y que, a medio hacer, va a dejar el proyecto roto. Pero mientras tanto, a lo mejor necesitas arreglar un fallo urgente en la versión que funciona.

Sin ramas, estarías atrapado: o terminas lo nuevo, o deshaces lo que llevas. Las ramas resuelven esto.

14.2 Qué es una rama, de verdad

Una rama es una **línea de trabajo paralela**. Puedes hacer commits en ella sin tocar la línea principal, y cuando lo nuevo esté listo, juntar las dos.

Técnicamente, una rama es algo mucho más sencillo de lo que parece: **una etiqueta con un nombre que apunta a un commit**. Cuando haces un commit nuevo, la etiqueta de la rama en la que estás avanza para apuntar al commit nuevo.

```
                      main
                       ↓
[A] ← [B] ← [C]       ← la rama main apunta al commit C
```

Si creas una rama llamada `nueva`, es solo otra etiqueta que al principio apunta al mismo sitio:

```
                      main
                       ↓
[A] ← [B] ← [C]
                       ↑
                     nueva
```

Y si haces commits estando en `nueva`, solo avanza esa etiqueta. `main` se queda donde estaba:

```
                      main
                       ↓
[A] ← [B] ← [C] ← [D] ← [E]
                            ↑
                          nueva
```

En Git hay además una etiqueta especial llamada **HEAD**, que significa **"estás aquí"**. Apunta a la rama en la que estás trabajando. Cuando cambias de rama, lo que cambia es adónde apunta HEAD.

14.3 Ver, crear y cambiar de rama

TÚ ESCRIBES

```
git branch
```

RESULTADO

```
* main
```

Lista las ramas. El asterisco marca en cuál estás. Ahora, crea una y entra en ella:

TÚ ESCRIBES

```
git switch -c feat/saludo
```

RESULTADO

```
Switched to a new branch 'feat/saludo'
```

DESMONTANDO EL COMANDO

`git switch -c feat/saludo`

`switch`	Cambiar de rama.
`-c`	*create*: créala primero, porque todavía no existe.
`feat/saludo`	El nombre. La barra no crea carpetas: es una costumbre para agrupar ramas por tipo (`feat/` , `fix/`), igual que los prefijos de los commits.

TÚ ESCRIBES

```
git branch
```

RESULTADO

```
* feat/saludo
  main
```

Ahora el asterisco está en la nueva. Haz un commit aquí:

TÚ ESCRIBES

```
echo "Hola desde la rama" > saludo.txt
git add .
git commit -m "feat: añade saludo"
```

Y ahora vuelve a `main` y mira qué hay:

TÚ ESCRIBES

```
git switch main
ls
```

RESULTADO

```
Switched to branch 'main'
README.md
```

`saludo.txt` ha desaparecido. Es lo que más sorprende la primera vez. No se ha borrado: existe en la rama `feat/saludo` , pero no en `main` . Al cambiar de rama, Git cambia los ficheros de tu carpeta para que coincidan

con esa rama. Vuelve a `feat/saludo` y reaparecerá.

> **UNA COMPROBACIÓN QUE AYUDA A ENTENDERLO**
>
> Abre la carpeta en el Finder con `open .` y déjala a la vista. Cambia de rama en la terminal. Verás aparecer y desaparecer el fichero en la ventana del Finder.

14.4 Juntar ramas: merge

Cuando lo que hiciste en la rama está listo, se integra en la principal. Siempre desde la rama **que recibe** los cambios:

TÚ ESCRIBES

```
git switch main
git merge feat/saludo
```

RESULTADO

```
Updating 1cb9542..99106fd
Fast-forward
 saludo.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 saludo.txt
```

`merge` significa "fusionar". Y la palabra **Fast-forward** ("avance rápido") explica lo que ha pasado: como `main` no había cambiado desde que creaste la rama, Git no tuvo que mezclar nada. Simplemente movió la etiqueta `main` hacia delante hasta donde estaba `feat/saludo`.

Una vez integrada, la rama ya no hace falta:

TÚ ESCRIBES

```
git branch -d feat/saludo
```

RESULTADO

```
Deleted branch feat/saludo (was 99106fd).
```

Borrar la rama no borra los commits: ya forman parte de `main`. Solo quita la etiqueta.

14.5 Cómo se trabaja con ramas en una empresa

Casi todos los equipos siguen este esquema:

1. `main` siempre funciona. Nadie trabaja directamente en ella.
2. Para cada tarea, se crea una rama: `feat/exportar-pdf`, `fix/login-movil`.
3. Se hacen los commits que haga falta en esa rama.
4. Cuando está lista, se pide que un compañero la revise (en GitHub, con un *pull request*, capítulo 17).
5. Si se aprueba, se integra en `main` y se borra la rama.

> **IDEA CLAVE**
>
> Una rama es una etiqueta que apunta a un commit y avanza cuando haces commits en ella. Te permite trabajar en algo sin tocar la versión buena. `git switch -c` crea y entra, `git switch` cambia, `git merge` junta.

15. Conflictos

15.1 Qué es un conflicto

En el capítulo anterior el merge fue automático porque solo una rama había cambiado. Pero ¿qué pasa si **las dos ramas han cambiado la misma línea del mismo fichero**, de formas distintas?

Git no puede saber cuál de las dos versiones es la buena. Así que para y te pregunta. Eso es un conflicto. No es un error ni una avería: es Git diciendo *"dos personas han tocado lo mismo, decide tú"*.

Si las dos ramas tocan ficheros distintos, o líneas distintas del mismo fichero, Git lo junta solo sin preguntar.

15.2 Provocar uno, paso a paso

La mejor forma de perderles el miedo es crear uno a propósito.

Paso 1. Un fichero con una línea:

TÚ ESCRIBES

```
echo "const precio = base * 1.21;" > precio.js
git add .
git commit -m "feat: añade cálculo de precio"
```

Paso 2. Una rama que cambia esa línea. Crea la rama y abre el fichero en VS Code:

TÚ ESCRIBES

```
git switch -c feat/impuestos
code precio.js
```

Cambia la línea por esta, guarda (`Cmd` + `S`) y haz commit:

```
const precio = base * (1 + IVA);
```

TÚ ESCRIBES

```
git commit -am "refactor: usa la constante IVA"
```

El `-am` es un atajo: `-a` prepara automáticamente los ficheros *ya vigilados* que hayan cambiado (se ahorra el `git add`) y `-m` es el mensaje. No sirve para ficheros nuevos.

Paso 3. Vuelve a `main` y cambia **la misma línea** de otra forma:

TÚ ESCRIBES

```
git switch main
```

```
const precio = base * 1.21 + gastosEnvio;
```

TÚ ESCRIBES

```
git commit -am "feat: suma los gastos de envío"
```

Paso 4. Intenta juntarlas:

TÚ ESCRIBES

```
git merge feat/impuestos
```

ERROR

```
Auto-merging precio.js
CONFLICT (content): Merge conflict in precio.js
Automatic merge failed; fix conflicts and then commit the result.
```

Traducción: "he intentado juntarlo solo; hay un conflicto en `precio.js`; arréglalo y haz commit del resultado".

15.3 Leer el conflicto

Abre `precio.js`. Git ha escrito las dos versiones dentro del fichero, separadas por marcas:

```
<<<<<<< HEAD
const precio = base * 1.21 + gastosEnvio;
=======
const precio = base * (1 + IVA);
>>>>>>> feat/impuestos
```

Marca	Significa
`<<<<<<< HEAD`	Empieza **tu versión**: la de la rama donde estás (HEAD = "estás aquí", o sea, `main`).
`=======`	La línea que separa las dos versiones.
`>>>>>>> feat/impuestos`	Termina la versión **de la otra rama**.

Y `git status` lo confirma:

TÚ ESCRIBES

```
git status
```

RESULTADO

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   precio.js
```

both modified: modificado por las dos ramas. Y fíjate en el segundo paréntesis: si te agobias, `git merge --abort` cancela la fusión y deja todo como estaba antes de intentarla. Siempre hay salida.

15.4 Resolverlo

Aquí no hay un comando mágico. **Tú editas el fichero y lo dejas como debe quedar.** Puedes quedarte con una versión, con la otra, o combinar las dos. En este caso tiene sentido combinar: queremos el IVA como constante y los gastos de envío.

Borra las tres líneas de marcas y deja el fichero así:

```
const precio = base * (1 + IVA) + gastosEnvio;
```

LAS TRES MARCAS TIENEN QUE DESAPARECER

Si se te queda alguna (`<<<<<<<` , `=======` o `>>>>>>>`), el código no funcionará. Antes de continuar, búscalas: `grep -n "<<<<" precio.js` no debería devolver nada.

Después, dile a Git que ya está resuelto y cierra la fusión:

TÚ ESCRIBES

```
git add precio.js
git commit -m "merge: combina el IVA con los gastos de envío"
```

El `git add` aquí significa "este fichero ya está resuelto". Y el historial queda así:

TÚ ESCRIBES

```
git log --oneline --graph
```

RESULTADO

```
*   4f51db2 merge: combina el IVA con los gastos de envío
|\
| * 38c041c refactor: usa la constante IVA
* | d4dd3eb feat: suma los gastos de envío
|/
* c67be28 feat: añade cálculo de precio
```

`--graph` dibuja las ramas con barras. Se lee de abajo arriba: la historia se abrió en dos después de `c67be28`, cada rama avanzó por su lado, y se volvieron a juntar en el commit de fusión.

VS CODE TE LO PONE FÁCIL

Si abres en VS Code un fichero con conflicto, verás las dos versiones coloreadas y encima unos botones: *Accept Current Change* (la tuya), *Accept Incoming Change* (la otra), *Accept Both Changes* (las dos). Hacen lo mismo que editar a mano, pero con un clic.

> **IDEA CLAVE**
>
> Un conflicto aparece cuando dos ramas cambian la misma línea. Git marca las dos versiones dentro del fichero; tú lo editas para dejarlo bien, borras las marcas, haces `git add` y `git commit`. Es una decisión humana, no un fallo.

16. Deshacer cosas

La mitad de la utilidad de Git es poder dar marcha atrás. Pero cada forma de deshacer tiene su momento, y una regla que no se puede romper.

16.1 La regla de oro

> **NUNCA REESCRIBAS HISTORIA QUE YA HAS COMPARTIDO**
>
> Algunas formas de deshacer **modifican commits que ya existen**. Mientras esos commits solo estén en tu ordenador, no pasa nada. Pero si ya los has subido a GitHub (capítulo 17), otras personas pueden tenerlos, y modificarlos les rompe su copia. Para lo ya publicado, se usa `git revert`, que deshace sin borrar.

16.2 Según lo que te haya pasado

"He cambiado un fichero y quiero dejarlo como estaba en el último commit."

TÚ ESCRIBES

```
git restore precio.js
```

Descarta tus cambios sin guardar en ese fichero. **Ojo: esos cambios se pierden**, porque nunca estuvieron en ninguna foto.

"He hecho git add de algo que no quería."

TÚ ESCRIBES

```
git restore --staged precio.js
```

Lo saca del área de preparación, pero tus cambios siguen en el fichero. Es lo que te sugería `git status` entre paréntesis.

"Me he equivocado en el mensaje del último commit."

TÚ ESCRIBES

```
git commit --amend -m "fix: corrige el cálculo del IVA"
```

Amend significa "enmendar". Sustituye el último commit por uno nuevo con el mensaje corregido. Fíjate en el hash antes y después:

RESULTADO

```
eeba25c fix: arregal el bg                    ← antes
61fd5a6 fix: corrige el cálculo del IVA        ← después
```

El hash ha cambiado: **es otro commit**. Por eso `--amend` reescribe la historia, y solo se usa si aún no has subido ese commit.

"Se me olvidó añadir un fichero al último commit."

TÚ ESCRIBES

```
git add fichero-olvidado.js
git commit --amend --no-edit
```

`--no-edit` mantiene el mensaje que ya tenía. Misma advertencia: solo si no lo has subido.

"Quiero deshacer el último commit, pero conservar los cambios en mis ficheros."

TÚ ESCRIBES

```
git reset --soft HEAD~1
```

`HEAD~1` significa "un commit antes de donde estoy". El commit desaparece, pero sus cambios vuelven a la preparación, listos para rehacerlo mejor. Reescribe historia.

"Un commit que ya subí estaba mal y quiero deshacerlo."

TÚ ESCRIBES

```
git revert 38c041c
```

`revert` no borra nada: crea un commit *nuevo* que hace exactamente lo contrario del indicado. La historia queda intacta, con un commit más que dice "deshago aquello". Es la forma segura para lo que ya es público.

16.3 La red de seguridad

TÚ ESCRIBES

```
git reflog
```

Git apunta todo lo que haces, incluso lo que "borras". `reflog` te enseña esa lista, con los hashes. Si alguna vez crees que has perdido trabajo, casi seguro está ahí. En Git es muy difícil perder algo que llegó a estar en un commit.

Situación	Comando	¿Reescribe historia?
Descartar cambios de un fichero	`git restore fichero`	No (pero pierdes esos cambios)
Sacar de la preparación	`git restore --staged fichero`	No
Corregir el último commit	`git commit --amend`	Sí
Deshacer el último commit, conservando cambios	`git reset --soft HEAD~1`	Sí
Deshacer un commit ya publicado	`git revert hash`	No

17. GitHub

17.1 Git y GitHub no son lo mismo

Es la confusión más común, así que vamos a dejarla clara:

	Git	GitHub
Qué es	Un programa en tu ordenador	Una página web
Qué hace	Guarda el historial de versiones	Guarda una copia de tus repositorios en internet
¿Necesita internet?	No	Sí
¿Necesita cuenta?	No	Sí

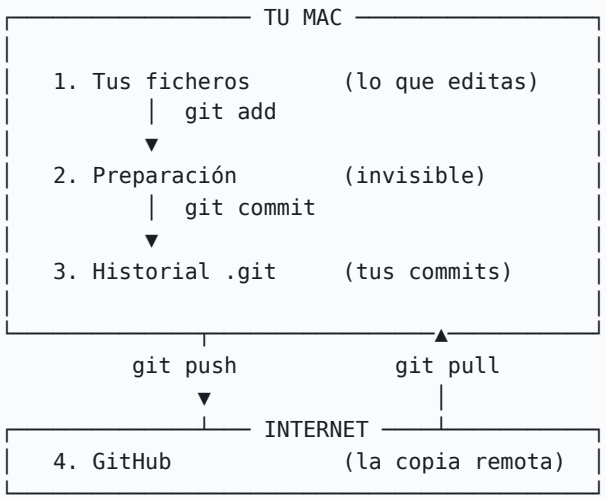
La comparación: **Git es como Word; GitHub es como Google Drive**. Puedes escribir documentos toda la vida sin Drive. Drive sirve para tenerlos en la nube, compartirlos y trabajar con otros.

Todo lo que has hecho en los capítulos 13 a 16 ha ocurrido en tu disco. GitHub no se ha enterado de nada.

17.2 Qué añade GitHub

1. **Una copia de seguridad.** Si tu Mac se estropea, `.git` se va con él. En GitHub hay otra copia.
2. **Trabajar con otros.** Tú subes, otra persona baja, y Git junta los cambios.
3. **Herramientas de equipo** que no son de Git, sino de GitHub:
   - *Pull requests*: "he terminado esta rama, ¿la revisáis antes de integrarla en main?". Es donde se hace la revisión de código en las empresas.
   - *Issues*: la lista de fallos y tareas.
   - *Actions*: ejecuta cosas automáticamente cada vez que subes código, como las pruebas.
4. **Tu currículum técnico.** Quien decida si te entrevista va a mirar tu GitHub antes que tu CV. Ese historial de commits, con fechas, es la prueba de que llevas tiempo practicando. No se puede falsificar.

17.3 Los cuatro lugares de tu código



Los tres primeros están en tu disco. Solo el cuarto está en internet, y **solo llega ahí cuando haces `git push`**. Puedes hacer commits durante semanas sin que nadie los vea.

17.4 Crear la cuenta

Entra en `github.com` y regístrate. Dos consejos:

- **Elige bien el nombre de usuario.** Va a aparecer en tu CV y en todas las direcciones de tus proyectos. Algo profesional: tu nombre y apellido, o una variante. Evita apodos y números al azar.
- **Usa el mismo correo** que pusiste en `git config` en el capítulo 3. Así GitHub reconoce tus commits como tuyos.

17.5 Conectar tu Mac con tu cuenta

Aquí está el escalón donde se atasca casi todo el mundo, así que lo aviso antes:

> **TU CONTRASEÑA DE GITHUB NO FUNCIONA EN LA TERMINAL**
>
> Si Git te pide usuario y contraseña al subir algo, y escribes tu contraseña normal, **fallará**. GitHub dejó de aceptarlas desde la terminal por seguridad. Hay que conectar tu Mac de otra forma.

La forma más sencilla es instalar `gh`, la herramienta oficial de GitHub, y dejar que ella lo configure:

TÚ ESCRIBES

```
brew install gh
gh auth login
```

Te hará unas preguntas. Se contestan con las flechas y `Intro`:

1. *Where do you use GitHub?* → **GitHub.com**
2. *What is your preferred protocol?* → **HTTPS**
3. *Authenticate Git with your GitHub credentials?* → **Yes**

4. *How would you like to authenticate?* → **Login with a web browser**

Te enseñará un código de ocho caracteres y abrirá el navegador. Pega el código, autoriza, y listo. **Esto se hace una sola vez por ordenador.**

Para comprobarlo:

TÚ ESCRIBES

```
gh auth status
```

RESULTADO

```
github.com
  ✓ Logged in to github.com account alvarocrespo (keyring)
```

17.6 Subir un repositorio por primera vez

Hay dos formas. La más rápida, con `gh`, desde dentro de la carpeta de tu repositorio:

TÚ ESCRIBES

```
gh repo create mi-primer-repo --public --source=. --push
```

DESMONTANDO EL COMANDO

`gh repo create mi-primer-repo --public --source=. --push`

`gh repo create`	Crea un repositorio nuevo en tu cuenta de GitHub.
`mi-primer-repo`	Cómo se llamará allí.
`--public`	Visible para cualquiera. Con `--private`, solo para ti.
`--source=.`	Usa como origen el repositorio de esta carpeta (el punto, "aquí").
`--push`	Y sube los commits que ya tengas.

La otra forma, más manual, es útil para entender qué pasa por debajo. Primero se crea el repositorio vacío en la web (botón **New**, le pones nombre, **no marcas ninguna casilla**, **Create repository**) y luego, en la terminal:

TÚ ESCRIBES

```
git remote add origin https://github.com/alvarocrespo/mi-primer-repo.git
git push -u origin main
```

DESMONTANDO EL COMANDO

`git remote add origin https://github.com/...`

`remote`	Gestiona los "remotos": las copias de este repositorio que están en otros sitios.
`add`	Añade uno.
`origin`	El apodo que le das. `origin` es el nombre habitual del remoto principal.
`https://...`	La dirección real.

Es como guardar un contacto: a partir de ahora, `origin` significa "esa dirección de GitHub". No sube nada; solo apunta la dirección.

DESMONTANDO EL COMANDO

`git push -u origin main`

`push`	"Empujar": sube tus commits al remoto.
`origin`	A cuál (el que acabas de apuntar).
`main`	Qué rama subes.
`-u`	*upstream*: recuerda esta conexión. Desde ahora, en esta rama bastará con escribir `git push` a secas.

Recarga la página del repositorio en GitHub y verás tus ficheros, con el README como portada.

17.7 El día a día: push y pull

Una vez conectado, el ciclo diario gana dos pasos, uno al principio y otro al final:

TÚ ESCRIBES

```
git pull                          # 1. trae lo nuevo que haya en GitHub
# ... trabajas ...
git add .                         # 2. preparas
git commit -m "feat: ..."         # 3. guardas en tu Mac
git push                          # 4. subes a GitHub
```

`pull` ("tirar") es lo contrario de `push`: baja los commits del remoto que tú no tengas y los junta con los tuyos. Si trabajas solo y desde un único ordenador, casi nunca habrá nada que bajar; pero es buena costumbre, y en un equipo es imprescindible.

17.8 Copiar un repositorio ajeno: clone

TÚ ESCRIBES

```
git clone https://github.com/sindresorhus/slugify.git
```

`clone` descarga un repositorio completo de GitHub: todos los ficheros **y toda su historia**, con su carpeta `.git`. Crea una carpeta con el nombre del proyecto y deja el remoto `origin` ya configurado. Es lo primero que harás el primer día en cualquier empresa.

> **IDEA CLAVE**
>
> Git vive en tu Mac; GitHub es una copia en internet más herramientas de equipo. Te conectas una vez con `gh auth login`. `git push` sube tus commits, `git pull` baja los de otros y `git clone` descarga un proyecto entero con su historia.

18. Lo que nunca debe subirse

18.1 El fichero .gitignore

Hay ficheros que no deben entrar nunca en el repositorio. Para eso se crea, en la raíz del proyecto, un fichero llamado exactamente `.gitignore` con una lista de nombres o patrones. Git hará como si esos ficheros no existieran: no aparecerán en `git status` y `git add .` no los cogerá.

Un `.gitignore` razonable para empezar en Mac:

```
.DS_Store
node_modules/
.env
*.log
```

Línea	Por qué se ignora
`.DS_Store`	Fichero oculto que macOS crea en cada carpeta para recordar cómo la ves en el Finder. No tiene nada que ver con tu proyecto. Es la marca más típica de alguien que empieza en Mac.
`node_modules/`	Las librerías que descarga npm. Pesan muchísimo y se pueden volver a descargar en cualquier momento.
`.env`	Contraseñas y claves secretas (capítulo 10).
`*.log`	Ficheros de registro. El `*` significa "cualquier nombre", como en la terminal.

18.2 Si ya lo subiste por error

Pasa muchísimo. Has hecho commit de un `.env` antes de tener el `.gitignore`. Añadirlo al `.gitignore` ahora no basta, porque Git ya lo está vigilando. Hay que decirle que deje de hacerlo:

TÚ ESCRIBES

```
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "chore: deja de seguir .env y lo ignora"
```

DESMONTANDO EL COMANDO

`git rm --cached .env`

`git rm`	Quita un fichero del repositorio.
`--cached`	**Pero lo deja en tu disco.** Sin esta opción, también lo borraría de tu carpeta, y tu programa dejaría de funcionar.

ESTO NO LO BORRA DEL PASADO

Lo anterior hace que `.env` deje de guardarse *a partir de ahora*, pero sigue estando en el commit donde lo subiste. Cualquiera que mire el historial puede verlo. Si alguna vez subes una contraseña o clave real a un repositorio público, el orden es: **primero, cambia esa contraseña** y dala por comprometida; después, limpia el repositorio. Nunca al revés.

IDEA CLAVE

Crea el `.gitignore` antes del primer commit. Nunca subas secretos. Si subiste algo por error, `git rm --cached` lo deja de seguir, pero lo que ya está en el historial ya es público.

19. Git sin terminal

19.1 Se puede

Todo lo de esta parte se puede hacer con el ratón:

Herramienta	Qué es
VS Code	Trae Git integrado. En la barra lateral izquierda, el icono de las ramitas (*Source Control*) te enseña los cambios en verde y rojo; escribes el mensaje, pulsas *Commit* y luego *Sync* para subir.
GitHub Desktop	Programa gratuito de GitHub (`brew install --cask github`). Todo con botones.
La web de GitHub	Permite crear y editar ficheros y hacer commits desde el navegador. Útil para cambios pequeños.

Y hacen **exactamente lo mismo** por debajo. Cuando pulsas "Commit" en VS Code, VS Code ejecuta `git commit` por ti. No hay dos Gits: hay uno, con varios mandos a distancia.

19.2 Entonces, ¿por qué aprender los comandos?

1. **Los botones cubren lo fácil.** Cuando algo se complique (un conflicto raro, algo que deshacer), todas las soluciones que encuentres estarán escritas como comandos.
2. **En las entrevistas se da por hecho.**
3. **En un servidor no hay botones.**
4. **Entender los comandos te hace entender los botones.** Sabiendo qué es el área de preparación, el panel de VS Code tiene todo el sentido; sin saberlo, es un misterio.

> **LO QUE HACE LA MAYORÍA DE PROFESIONALES**
>
> Usar el panel de Git de VS Code para el día a día (ver cambios, hacer commits) y la terminal para todo lo demás. No es trampa: es lo sensato.

Ejercicios · Parte 2

Estos ejercicios van encadenados: construyen un único repositorio que al final estará en tu GitHub. Es el primer ladrillo de tu portfolio, así que merece la pena hacerlo con cuidado.

EJERCICIO 2.1 · EL REPOSITORIO

1. Crea en tu carpeta personal una carpeta `aprendiendo-programacion` y conviértela en repositorio con la rama `main`.
2. Antes de nada, crea un `.gitignore` que ignore `.DS_Store`, `node_modules/` y `.env`.
3. Crea un `README.md` con un título y una frase que explique qué es el repositorio.
4. Haz **dos commits separados**: uno para el `.gitignore` y otro para el README. Usa prefijos de Conventional Commits.
5. Comprueba el historial en formato compacto.

EJERCICIO 2.2 · UNA RAMA DE IDA Y VUELTA

1. Crea una rama `feat/apuntes-terminal` y entra en ella.
2. Crea una carpeta `01-terminal` con un fichero `comandos.md` donde apuntes cinco comandos que hayas aprendido y qué hacen.
3. Haz commit.
4. Vuelve a `main` y comprueba con `ls` si la carpeta está. Explica lo que ves.
5. Integra la rama en `main` y bórrala.

EJERCICIO 2.3 · UN CONFLICTO A PROPÓSITO

Provoca un conflicto sobre la primera línea del README (el título) desde dos ramas distintas, y resuélvelo dejando un título que combine las dos versiones.

EJERCICIO 2.4 · ARREGLAR EL ÚLTIMO COMMIT

1. Crea un fichero cualquiera y haz commit con un mensaje con erratas.
2. Corrige el mensaje.
3. Explica por qué el hash ha cambiado y por qué esto solo es seguro antes de subirlo.

EJERCICIO 2.5 · EL SECRETO QUE SE COLÓ

1. Borra temporalmente la línea `.env` de tu `.gitignore`.
2. Crea un `.env` con `CLAVE=123` y haz commit de todo con `git add .`
3. Arréglalo: que `.env` deje de estar en el repositorio pero siga en tu disco, y que vuelva a estar ignorado.
4. Comprueba las dos cosas.

EJERCICIO 2.6 · PUBLICARLO

1. Conecta tu Mac con tu cuenta de GitHub, si no lo has hecho.
2. Publica el repositorio como público.
3. Añade al README una línea nueva, haz commit y súbela.
4. Comprueba en la web que el cambio está.

EJERCICIO 2.7 · LEER UN PROYECTO AJENO

Clona `https://github.com/sindresorhus/slugify.git` y averigua, solo con la terminal: cuántos commits tiene, quiénes son las tres personas que más commits han hecho, y cuándo fue el último cambio.

Soluciones · Parte 2

2.1 · El repositorio

TÚ ESCRIBES

```
cd ~
mkdir aprendiendo-programacion
cd aprendiendo-programacion
git init -b main
```

Ahora el `.gitignore`. Se puede escribir con `cat >` o abriéndolo en VS Code con `code .gitignore`:

TÚ ESCRIBES

```
cat > .gitignore
.DS_Store
node_modules/
.env
```

(`Ctrl`+`D` para cerrar.) **Por qué primero el `.gitignore`:** si haces un `git add .` antes de tenerlo, el `.DS_Store` que macOS crea sin avisar se colará en el primer commit.

TÚ ESCRIBES

```
echo "# Aprendiendo programación" > README.md
echo "Apuntes y ejercicios de mi formación como desarrollador." >> README.md
```

Fíjate en `>` para la primera línea (crea el fichero) y `>>` para la segunda (añade sin borrar).

TÚ ESCRIBES

```
git add .gitignore
git commit -m "chore: añade gitignore"
git add README.md
git commit -m "docs: añade README"
git log --oneline
```

RESULTADO

```
3e1a0b2 docs: añade README
9c4d7f1 chore: añade gitignore
```

Dos commits porque son dos cambios de naturaleza distinta: uno de configuración (`chore`) y otro de documentación (`docs`). Si mañana quisieras deshacer uno, el otro no se vería afectado.

2.2 · Una rama de ida y vuelta

TÚ ESCRIBES

```
git switch -c feat/apuntes-terminal
mkdir 01-terminal
code 01-terminal/comandos.md
```

Escribe en VS Code lo que quieras, guarda con `Cmd` + `S`, y vuelve a la terminal:

TÚ ESCRIBES

```
git add .
git commit -m "docs: añade apuntes de comandos de terminal"
git switch main
ls
```

RESULTADO

```
README.md
```

La carpeta no está. Ese commit solo existe en la rama `feat/apuntes-terminal`; la etiqueta `main` sigue apuntando al commit anterior, donde la carpeta no existía. Al cambiar de rama, Git pone en tu carpeta los ficheros tal y como están en esa rama.

TÚ ESCRIBES

```
git merge feat/apuntes-terminal
ls
git branch -d feat/apuntes-terminal
```

RESULTADO

```
Updating 3e1a0b2..b82f5c0
Fast-forward
 01-terminal/comandos.md | 12 ++++++++++++
 1 file changed, 12 insertions(+)
README.md  01-terminal
Deleted branch feat/apuntes-terminal (was b82f5c0).
```

Fue un *Fast-forward* porque `main` no había cambiado mientras trabajabas en la rama: Git solo tuvo que mover la etiqueta hacia delante. Los `+` después del nombre del fichero son las líneas añadidas, dibujadas.

2.3 · Un conflicto a propósito

La clave es que las **dos** ramas cambien la **misma** línea después de separarse.

TÚ ESCRIBES

```
git switch -c docs/titulo-largo
```

Cambia en VS Code la primera línea del README a `# Aprendiendo programación desde cero`, guarda, y:

TÚ ESCRIBES

```
git commit -am "docs: alarga el título"
git switch main
```

Ahora, en `main`, cambia esa misma primera línea a `# Mi camino en el desarrollo web`:

TÚ ESCRIBES

```
git commit -am "docs: cambia el título"
git merge docs/titulo-largo
```

ERROR

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

En el README verás:

```
<<<<<<< HEAD
# Mi camino en el desarrollo web
=======
# Aprendiendo programación desde cero
>>>>>>> docs/titulo-largo
```

Arriba la versión de `main` (donde estás, HEAD), abajo la de la otra rama. Deja una sola línea que combine las dos, sin marcas:

```
# Mi camino en el desarrollo web: aprendiendo desde cero
```

TÚ ESCRIBES

```
git add README.md
git commit -m "merge: combina los dos títulos"
git branch -d docs/titulo-largo
```

El `git add` después de resolver es como decirle a Git "ya lo he arreglado". Hasta que no lo haces, sigue considerando el fichero en conflicto.

2.4 · Arreglar el último commit

TÚ ESCRIBES

```
echo "prueba" > prueba.txt
git add prueba.txt
git commit -m "feat: añade fichreo de preuba"
git log --oneline -1
```

RESULTADO

```
a71c3e9 feat: añade fichreo de preuba
```

(`-1` significa "enséñame solo el último".)

TÚ ESCRIBES

```
git commit --amend -m "feat: añade fichero de prueba"
git log --oneline -1
```

RESULTADO

```
5d20b8f feat: añade fichero de prueba
```

Por qué cambia el hash: el hash se calcula a partir de todo lo que contiene el commit, incluido el mensaje. Distinto mensaje, distinto hash. Así que `--amend` no "edita" el commit anterior: crea uno nuevo y descarta el

viejo.

Por qué solo es seguro antes de subir: si `a71c3e9` ya estuviera en GitHub y alguien lo hubiera descargado, su historia tendría un commit que en la tuya ya no existe. Las dos versiones de la historia no encajarían. Por eso, para lo ya publicado se usa `git revert`, que añade en lugar de reescribir.

2.5 · El secreto que se coló

Tras quitar `.env` del `.gitignore` en VS Code:

TÚ ESCRIBES

```
echo "CLAVE=123" > .env
git add .
git commit -m "chore: añade configuración"
git ls-files
```

RESULTADO

```
.env
.gitignore
01-terminal/comandos.md
README.md
prueba.txt
```

`git ls-files` lista los ficheros que Git está siguiendo. Ahí está el `.env`. Ahora, el arreglo:

TÚ ESCRIBES

```
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "chore: deja de seguir .env y lo ignora"
```

Las dos comprobaciones:

TÚ ESCRIBES

```
git ls-files | grep env
ls -a | grep env
```

RESULTADO

```
.env
```

El primer comando no devuelve nada: Git ya no lo sigue. El segundo sí lo encuentra: sigue en tu disco. Exactamente lo que queríamos. Y fíjate en que has combinado Git con una tubería y `grep` de la Parte 1: las herramientas se mezclan con naturalidad.

Recuerda: el `.env` sigue existiendo en el commit "añade configuración". Aquí no importa porque `CLAVE=123` es inventada, pero con una clave real habría que cambiarla.

2.6 · Publicarlo

TÚ ESCRIBES

```
brew install gh
gh auth login
```

(Con las respuestas del apartado 17.5.) Después, desde dentro de la carpeta:

TÚ ESCRIBES

```
gh repo create aprendiendo-programacion --public --source=. --push
```

RESULTADO

```
✓ Created repository alvarocrespo/aprendiendo-programacion on GitHub
✓ Added remote https://github.com/alvarocrespo/aprendiendo-programacion.git
✓ Pushed commits to https://github.com/alvarocrespo/aprendiendo-programacion.git
```

Las tres líneas describen los tres pasos que hace por ti: crear el repositorio en GitHub, apuntar el remoto `origin` (lo que haría `git remote add`) y subir los commits (lo que haría `git push`).

Ahora un cambio y su subida:

TÚ ESCRIBES

```
echo "" >> README.md
echo "Estado: Parte 2 del Volumen 1 completada." >> README.md
git commit -am "docs: actualiza el estado del README"
git push
```

Esta vez basta con `git push`, sin nada más, porque `gh` ya configuró la conexión (el equivalente al `-u`). Para abrir el repositorio en el navegador sin buscarlo:

TÚ ESCRIBES

```
gh repo view --web
```

2.7 · Leer un proyecto ajeno

TÚ ESCRIBES

```
cd ~
git clone https://github.com/sindresorhus/slugify.git
cd slugify
```

Cuántos commits:

TÚ ESCRIBES

```
git log --oneline | wc -l
```

`git log --oneline` escribe un commit por línea; `wc -l` cuenta líneas. Es el mismo truco que en el ejercicio 1.1 con `ls`.

Quién contribuye más:

TÚ ESCRIBES

```
git log --format="%an" | sort | uniq -c | sort -rn | head -3
```

`--format="%an"` hace que `git log` escriba solo el nombre del autor (*author name*) de cada commit, uno por línea. Y lo demás es **exactamente la misma cadena** que usaste para contar palabras en el capítulo 8: ordenar para juntar los repetidos, contarlos, ordenar por número de mayor a menor, y quedarse con los primeros. Solo que ahora cuenta autores en lugar de palabras.

Esta es la gran ventaja de aprender las piezas por separado: un patrón que aprendiste para un problema te sirve para cualquier otro.

Último cambio:

TÚ ESCRIBES

```
git log -1
```

El commit más reciente, con su fecha. Leer así la historia de un proyecto antes de tocarlo es, literalmente, lo que harás tu primera semana en cualquier empleo.

PARTE 3

JavaScript desde cero

Ya sabes moverte por el ordenador y guardar tu trabajo. Ahora toca escribir programas. Esta parte empieza por lo más básico, un valor, y va construyendo pieza a pieza hasta que puedas escribir programas pequeños tú solo.

- 20. Qué es JavaScript y dónde se ejecuta
- 21. Valores y tipos
- 22. Variables
- 23. Trabajar con texto
- 24. Trabajar con números
- 25. Leer los errores
- 26. Tomar decisiones
- 27. Arrays: listas de valores
- 28. Objetos: fichas con campos
- 29. Valor y referencia
- 30. Funciones
- 31. Bucles

20. Qué es JavaScript y dónde se ejecuta

20.1 Qué es un lenguaje de programación

El ordenador, por dentro, solo entiende instrucciones muy elementales escritas en unos y ceros. Escribir así sería imposible. Un **lenguaje de programación** es una forma de escribir instrucciones que una persona puede leer y que un programa traductor convierte en lo que el ordenador entiende.

Es un idioma, pero un idioma muy estricto: tiene pocas palabras, reglas fijas, y ninguna tolerancia a la ambigüedad. Una coma de menos, y el ordenador no te entiende.

20.2 Por qué JavaScript

JavaScript nació en 1995 para dar vida a las páginas web: que un botón haga algo al pulsarlo, que un formulario compruebe si el correo es válido. Es el único lenguaje que entienden todos los navegadores.

Luego, en 2009, apareció Node.js (que instalaste en el capítulo 3), que permite ejecutar JavaScript también fuera del navegador. Desde entonces, con un solo lenguaje se puede programar la parte que ve el usuario y la parte que funciona en el servidor. Por eso es el lenguaje con el que empieza este manual.

Un aviso sobre el nombre: **JavaScript no tiene nada que ver con Java**. Son lenguajes completamente distintos. El nombre fue una decisión de marketing de los noventa.

20.3 Tres sitios donde ejecutarlo

Opción 1: la consola del navegador. En Chrome, pulsa `Cmd`+`⌥ Opción`+`J`. Se abre un panel donde puedes escribir JavaScript y ver el resultado al instante. No necesita nada instalado. Tiene una pega importante que veremos en el capítulo 22.

Opción 2: Node en modo interactivo. En la terminal, escribe `node` a secas:

TÚ ESCRIBES

```
node
```

RESULTADO

```
Welcome to Node.js v22.9.0.
Type ".help" for more information.
>
```

El prompt ha cambiado a `>`: ahora lo que escribes no son comandos de terminal, sino JavaScript. Prueba `2 + 3` y pulsa Intro. Para salir, escribe `.exit` o pulsa `Ctrl`+`D`.

Opción 3: un fichero y Node. Esta es la que usaremos. Escribes el programa en un fichero con extensión `.js` y lo ejecutas desde la terminal. Así el código queda guardado, lo puedes corregir, y es como se trabaja de verdad.

20.4 Tu primer programa

Crea una carpeta para esta parte dentro del repositorio del ejercicio 2.1, y un fichero:

TÚ ESCRIBES

```
cd ~/aprendiendo-programacion
mkdir 03-javascript
cd 03-javascript
code hola.js
```

En VS Code, escribe esta línea y guarda con `Cmd` + `S`:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log("Hola, mundo");
```

Vuelve a la terminal y ejecútalo:

TÚ ESCRIBES

```
node hola.js
```

RESULTADO

```
Hola, mundo
```

Acabas de escribir y ejecutar un programa. Vamos a desmontar esa línea, porque cada pieza tiene su porqué:

DESMONTANDO EL COMANDO

`console.log("Hola, mundo");`

`console`	Un objeto que ya existe en JavaScript y que representa "la salida de texto": la terminal, o la consola del navegador.
`.`	El punto significa "de": `console.log` es "el `log` de `console`". Lo explicamos a fondo en el capítulo 23.
`log`	La acción: escribir algo. Viene de "llevar un registro" (el cuaderno de bitácora de un barco).
`( )`	Los paréntesis **ejecutan** la acción. Dentro va lo que le das.
`"Hola, mundo"`	Lo que quieres escribir. Entre comillas porque es texto.
`;`	El punto y coma marca el final de la instrucción, como el punto al final de una frase.

Traducido: *"en la salida de texto, escribe Hola, mundo"*.

`console.log` será tu herramienta principal durante mucho tiempo: es la forma de ver qué está pasando dentro de tu programa.

20.5 Los comentarios

JAVASCRIPT · EN TU FICHERO .JS

```javascript
// Esto es un comentario. JavaScript lo ignora.
console.log("Esto sí se ejecuta");   // y esto también se ignora

/*
  Esto es un comentario
  de varias líneas.
*/
```

Todo lo que va después de `//`, hasta el final de la línea, es una nota para humanos. Igual que el `#` en la terminal. En este manual los uso para explicar al lado del código qué hace cada línea.

20.6 Cómo se ejecuta un programa

Esto parece obvio pero es fundamental: **un programa se ejecuta de arriba abajo, una línea cada vez, en orden**.

JAVASCRIPT · EN TU FICHERO .JS

```javascript
console.log("uno");
console.log("dos");
console.log("tres");
```

RESULTADO

```
uno
dos
tres
```

Como una receta. Cada instrucción se hace en su momento, con lo que hay en ese momento, y el programa pasa a la siguiente. Tenlo presente, porque en el capítulo 22 esta idea resolverá una de las confusiones más habituales.

IDEA CLAVE

Escribes JavaScript en un fichero `.js` y lo ejecutas con `node fichero.js`. `console.log(...)` escribe en pantalla. El programa se ejecuta de arriba abajo, una línea tras otra.

21. Valores y tipos

21.1 Qué es un valor

Un **valor** es un dato: un número, un texto, un sí o un no. Es la materia prima con la que trabaja cualquier programa. Todo lo que hace un programa, al final, es coger valores, transformarlos y producir otros.

Y cada valor tiene un **tipo**: la clase de dato que es. El tipo importa porque decide qué se puede hacer con él. Puedes multiplicar dos números, pero no puedes multiplicar dos nombres.

21.2 Los tipos básicos

Tipo	Qué es	Cómo se escribe
string (texto)	Cualquier texto: una letra, una palabra, un párrafo. "String" significa "cadena": una cadena de caracteres.	Entre comillas: `"Hola"`, `'Hola'` o `` `Hola` ``
number (número)	Cualquier número, entero o con decimales. JavaScript no los distingue.	Sin comillas: `42`, `3.14`, `-7`. Los decimales, con **punto**, no coma.
boolean (booleano)	Solo dos valores posibles: sí o no. Se llama así por el matemático George Boole.	`true` (verdadero) o `false` (falso), sin comillas
undefined	"Sin definir": algo que todavía no tiene valor.	`undefined`
null	"Vacío, a propósito": se usa para decir expresamente que algo no tiene valor.	`null`

La diferencia entre `undefined` y `null` es sutil: `undefined` es "nadie ha puesto nada aquí"; `null` es "alguien ha decidido que aquí no haya nada". En la práctica, al principio te bastará con reconocerlos.

21.3 Preguntar el tipo

`typeof` te dice de qué tipo es un valor. Crea un fichero `tipos.js`:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(typeof "hola");
console.log(typeof 42);
console.log(typeof true);
console.log(typeof undefined);
```

RESULTADO

```
string
number
boolean
undefined
```

21.4 Por qué el tipo importa tanto

Aquí está la primera sorpresa de JavaScript. Mira estas dos líneas:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(5 + 5);
console.log("5" + 5);
```

RESULTADO

```
10
55
```

¿`55`? En la primera línea, `5` y `5` son números, y `+` los suma. En la segunda, `"5"` es un **texto** (lleva comillas), y cuando `+` tiene un texto a cualquiera de los dos lados, **no suma: pega**. Convierte el `5` en texto y junta los dos: "5" y "5" dan "55".

Las comillas lo cambian todo: `5` es un número; `"5"` es un texto que contiene el carácter 5. Para JavaScript son cosas completamente distintas, igual que para ti son distintas la cantidad cinco y la palabra "cinco" escrita en un papel.

POR QUÉ ESTO IMPORTA EN LA VIDA REAL

Todo lo que escribe un usuario en un formulario web llega como texto, aunque sea un número. Si alguien escribe "5" en una casilla de cantidad y tu programa le suma 5 sin convertirlo antes, el resultado será 55. Es uno de los fallos más comunes en programas reales.

IDEA CLAVE

Todo dato es un valor y cada valor tiene un tipo: texto (con comillas), número (sin comillas), booleano (`true` / `false`), `undefined` o `null`. El tipo decide qué puedes hacer con él. `+` suma números pero pega textos.

22. Variables

22.1 Qué es una variable

Ya lo viste en el capítulo 10 con la terminal: **una variable es un nombre que apunta a un valor**.

Recuerda la comparación de los contactos del móvil: en lugar de marcar `666 123 456` cada vez, guardas ese número bajo el nombre `Mamá` y usas el nombre. Si el número cambia, lo actualizas en un solo sitio.

En JavaScript:

JAVASCRIPT · EN TU FICHERO .JS

```
const nombre = "Álvaro";
```

DESMONTANDO EL COMANDO

`const nombre = "Álvaro";`

`const`	"Voy a crear una variable". Es la palabra que la declara (la hace nacer).
`nombre`	El nombre que le das. Lo eliges tú.
`=`	"Guarda esto aquí". **No significa "es igual a"**; lo explicamos ahora.
`"Álvaro"`	El valor que se guarda.

A partir de esa línea, cada vez que escribas `nombre`, JavaScript pondrá en su lugar `"Álvaro"`:

JAVASCRIPT · EN TU FICHERO .JS

```
const nombre = "Álvaro";
console.log(nombre);
console.log("Hola, " + nombre);
```

RESULTADO

```
Álvaro
Hola, Álvaro
```

Fíjate en la diferencia entre `nombre` (sin comillas: la variable, que se sustituye por su valor) y `"nombre"` (con comillas: el texto literal "nombre"). Es la misma diferencia que había en la terminal entre `$HOME` y `HOME`.

22.2 El = es una acción, no una igualdad

Este es uno de los conceptos que más cuesta al principio, porque en matemáticas `=` significa otra cosa.

En matemáticas, `b = a` es una afirmación que siempre es verdad: si `a` cambia, `b` cambia con ella. **En programación no.** `=` es una orden que se ejecuta una vez: *"coge el valor que hay ahora mismo a la derecha y guárdalo a la izquierda"*. Después, el programa sigue y se olvida de la operación.

Por eso conviene leerlo en voz alta como **"recibe"**, nunca como "es igual a". `const precio = 10` se lee "precio recibe 10".

Mira este programa y, **antes de seguir leyendo, predice qué escribe**:

JAVASCRIPT · EN TU FICHERO .JS

```
let a = 5;
let b = a;
a = 10;
console.log(b);
```

La intuición dice 10: "b es igual a a, y a vale 10". Pero sale **5**.

Sigue el programa línea a línea, como una receta:

Línea	Qué pasa	a	b
`let a = 5;`	a recibe 5	5	—
`let b = a;`	b recibe **el valor que tiene a en este momento**: 5	5	5
`a = 10;`	a recibe 10. b no se entera de nada	10	5

En la segunda línea, `b` no guarda "a". Guarda `5`. El nombre `a` desaparece en el momento en que se ejecuta esa línea; es como si hubieras escrito `let b = 5`. Entre `a` y `b` no queda ningún vínculo.

Si todavía no lo ves claro, esta versión lo hace evidente:

JAVASCRIPT · EN TU FICHERO .JS

```
let a = 5;
let b = a * 2;     // b recibe 10
a = 100;
console.log(b);    // 10
```

A nadie se le ocurre esperar que `b` valga 200. Es obvio que `a * 2` se calculó una vez y dio 10. Pues `let b = a` es exactamente lo mismo, con una operación tan simple que no parece un cálculo.

22.3 const y let

Hay dos palabras para crear variables, y la diferencia es una sola:

	`const`	`let`
Viene de	*constant*, constante	"let": "deja que x valga..."
¿Se le puede dar otro valor después?	**No**	Sí

JAVASCRIPT · EN TU FICHERO .JS

```
let contador = 0;
contador = 1;          // correcto: con let se puede cambiar
contador = contador + 1;   // ahora vale 2

const ciudad = "Santander";
ciudad = "Somo";       // ERROR
```

ERROR

```
TypeError: Assignment to constant variable.
```

"Asignación a una variable constante": intentaste darle otro valor a algo declarado como `const`.

Fíjate en la línea `contador = contador + 1`. En matemáticas sería absurdo (ningún número es igual a sí mismo más uno), pero ya sabes que `=` es una acción: "calcula lo de la derecha (el valor actual de contador, más 1) y guárdalo en contador". Es la forma de aumentar un valor.

La regla profesional: usa `const` siempre, y cambia a `let` solo cuando necesites cambiar el valor. ¿Por qué? Porque cuando lees `const precio`, sabes que `precio` significará lo mismo en todo el programa. Es una cosa menos de la que preocuparte.

Y verás una tercera palabra en código antiguo: `var`. Es la forma antigua, con comportamientos confusos. No la uses.

22.4 Cómo nombrar variables

Reglas obligatorias (si las incumples, error):

- Sin espacios. `mi nombre` no vale.
- No pueden empezar por número. `2precio` no vale.
- No pueden ser palabras reservadas del lenguaje, como `const` o `let`.

Convenciones (no son obligatorias, pero todo el mundo las sigue):

- **camelCase**: si el nombre tiene varias palabras, se juntan y cada una empieza en mayúscula, menos la primera. `precioTotal`, `numeroDeSesiones`. Se llama así por las "jorobas" que forman las mayúsculas.
- **Nombres que se entiendan**. `precioConIva` es infinitamente mejor que `p` o `x2`. El código se lee muchas más veces de las que se escribe.
- **Las mayúsculas cuentan.** `nombre`, `Nombre` y `NOMBRE` son tres variables distintas.

22.5 La trampa de la consola del navegador

Si pruebas código en la consola del navegador, te pasará esto. Escribes:

JAVASCRIPT · EN TU FICHERO .JS

```
const ciudad = "Santander";
```

Y un rato después, en otro ejercicio:

JAVASCRIPT · EN TU FICHERO .JS

```
let ciudad = "Somo";
```

ERROR

```
SyntaxError: Identifier 'ciudad' has already been declared
```

"El identificador `ciudad` ya ha sido declarado". La consola **recuerda todo** lo que has escrito mientras no recargues la página, y no se puede crear dos veces una variable con el mismo nombre. No tiene nada que ver con `let` o `const`: es que el nombre ya está ocupado.

La solución es recargar la página (`Cmd`+`R`), que borra la memoria de la consola. Y es otra razón para trabajar con ficheros y `node`: cada vez que ejecutas un fichero, empieza de cero.

IDEA CLAVE

Una variable es un nombre que apunta a un valor. `=` significa "recibe": se ejecuta una vez, en ese momento, y no crea ningún vínculo permanente. `const` no se puede reasignar, `let` sí. Usa `const` por defecto y nombres en camelCase que se entiendan.

23. Trabajar con texto

23.1 Tres tipos de comillas

JAVASCRIPT · EN TU FICHERO .JS

```
const a = "comillas dobles";
const b = 'comillas simples';
const c = `comillas invertidas`;
```

Las dos primeras son equivalentes; elige una y sé coherente. Las invertidas (`` ` ``) tienen un superpoder que vemos en el 23.3.

> **DÓNDE ESTÁ LA COMILLA INVERTIDA EN UN MAC**
>
> En el teclado español de Mac suele estar en la tecla que hay a la derecha de la **P**. Es una tecla "muerta": al pulsarla no aparece nada hasta que pulsas la siguiente. Púlsala y luego `Espacio`.

23.2 Unir textos con +

JAVASCRIPT · EN TU FICHERO .JS

```
const nombre = "Álvaro";
const ciudad = "Santander";
console.log("Me llamo " + nombre + " y vivo en " + ciudad);
```

RESULTADO

```
Me llamo Álvaro y vivo en Santander
```

Esto se llama **concatenar**. Funciona, pero tiene dos problemas: es fácil olvidarse de un `+`, y hay que estar muy pendiente de los espacios. JavaScript pega exactamente lo que le das, ni un espacio más: si escribes `"vivo en" + ciudad`, sale `vivo enSantander`.

23.3 Plantillas: la forma buena

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(`Me llamo ${nombre} y vivo en ${ciudad}`);
```

Con comillas invertidas, puedes meter cualquier valor en mitad del texto poniéndolo dentro de `${ }`. Se lee como la frase final, con huecos donde van los valores.

Y lo más útil: **dentro de `${ }` cabe cualquier expresión**, no solo una variable. Cualquier cosa que produzca un valor:

JAVASCRIPT · EN TU FICHERO .JS

```
const edad = 25;
console.log(`Dentro de 5 años tendré ${edad + 5}`);
console.log(`Vivo en ${ciudad.toUpperCase()}`);
```

RESULTADO

```
Dentro de 5 años tendré 30
Vivo en SANTANDER
```

A partir de ahora, usa siempre plantillas para construir textos.

23.4 Métodos: las habilidades de un valor

Ese `.toUpperCase()` del ejemplo anterior es un **método**, y merece una explicación, porque vas a usar métodos constantemente.

Un método es **una acción que un valor sabe hacer consigo mismo**. Todo texto en JavaScript "sabe" ponerse en mayúsculas, recortarse, buscar dentro de sí. No necesitas una herramienta externa: se lo pides al propio texto.

DESMONTANDO EL COMANDO

`ciudad.toUpperCase()`

`ciudad`	El valor al que le pides algo.
`.`	"De": accede a algo que pertenece al valor.
`toUpperCase`	El nombre del método: *to upper case*, "a mayúsculas".
`()`	Los paréntesis lo **ejecutan**. Sin ellos no haces nada: solo mencionas el método.

Se lee literalmente como *"ciudad, ponte en mayúsculas"*.

Compáralo con una **función**, que es una herramienta independiente a la que le pasas el valor entre paréntesis:

JAVASCRIPT · EN TU FICHERO .JS

```
// función: la herramienta va primero, el dato dentro
Number("42")

// método: el dato va primero, la acción detrás del punto
"hola".toUpperCase()
```

Mezclarlas es un error muy común. Esto no funciona:

JAVASCRIPT · EN TU FICHERO .JS

```
toUpperCase(ciudad)
```

ERROR

```
ReferenceError: toUpperCase is not defined
```

JavaScript busca una herramienta suelta llamada `toUpperCase` y no la encuentra, porque no existe como función: es una habilidad que tienen los textos.

23.5 Los métodos de texto más útiles

Método	Qué hace	Ejemplo → resultado
`.toUpperCase()`	A mayúsculas	`"hola".toUpperCase()` → `"HOLA"`
`.toLowerCase()`	A minúsculas	`"HoLa".toLowerCase()` → `"hola"`
`.trim()`	Quita espacios al principio y al final	`" hola ".trim()` → `"hola"`
`.includes(x)`	¿Contiene x?	`"techno".includes("ech")` → `true`
`.startsWith(x)`	¿Empieza por x?	`"techno".startsWith("te")` → `true`
`.replace(a, b)`	Cambia la primera a por b	`"19.99".replace(".", ",")` → `"19,99"`
`.length`	Cuántos caracteres tiene	`"hola".length` → `4`

Fíjate en que `.length` **no lleva paréntesis**. No es una acción, es un dato que el texto tiene: su longitud. Las cosas que acaban en `()` hacen algo; las que no, simplemente son algo. A los datos de un valor se les llama **propiedades**.

23.6 Las mayúsculas en los nombres de métodos

Los métodos siguen camelCase, igual que las variables, y hay que escribirlos exactos. `toUppercase` (con la c minúscula) no existe; es `toUpperCase`. Si lo escribes mal:

ERROR

```
TypeError: ciudad.toUppercase is not a function
```

"No es una función": JavaScript buscó un método con ese nombre exacto dentro del texto, no lo encontró, y te dice que eso que intentas ejecutar con `()` no es algo ejecutable. VS Code te ayuda: al escribir el punto, te sugiere los métodos disponibles. Úsalo.

> **IDEA CLAVE**
>
> Usa plantillas `` `...${valor}...` `` para construir textos; dentro de `${ }` cabe cualquier expresión. Un método es una acción que un valor sabe hacer: `valor.metodo()`. Los paréntesis la ejecutan. Las propiedades como `.length` no llevan paréntesis.

24. Trabajar con números

24.1 Las operaciones

Símbolo	Operación	Ejemplo → resultado
`+`	Suma	`7 + 3` → `10`
`-`	Resta	`7 - 3` → `4`
`*`	Multiplicación	`7 * 3` → `21`
`/`	División	`7 / 2` → `3.5`
`%`	Resto de la división	`7 % 2` → `1`

El orden es el de las matemáticas: primero multiplicaciones y divisiones, luego sumas y restas. Con paréntesis se cambia: `(2 + 3) * 4` da 20.

El `%` parece raro, pero es muy útil. El resto de dividir entre 2 es 0 si el número es par y 1 si es impar. Así que `numero % 2 === 0` es la forma de preguntar "¿es par?".

24.2 La trampa de los decimales

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(0.1 + 0.2);
```

RESULTADO

```
0.30000000000000004
```

No es un fallo de JavaScript: pasa en casi todos los lenguajes. El ordenador guarda los números en binario (ceros y unos), y hay decimales que no se pueden escribir exactamente en binario, igual que 1/3 no se puede escribir exactamente en decimal (0,3333…). Se quedan con un error diminuto.

Un error de 0,00000000000000004 no importa para casi nada. Pero en dinero, sumado a lo largo de miles de operaciones, descuadra las cuentas. Por eso existe esta regla en todo el sector:

EL DINERO SE GUARDA EN CÉNTIMOS

Nunca guardes 19,99 € como `19.99`. Guárdalo como `1999` céntimos. Con números enteros no hay error posible: `1999 * 3` es exactamente `5997`. Solo al final, para enseñárselo al usuario, se divide entre 100.

24.3 toFixed: mostrar un número con decimales fijos

JAVASCRIPT · EN TU FICHERO .JS

```
const totalCentimos = 60000;
const totalEuros = totalCentimos / 100;
console.log(totalEuros);
```

RESULTADO

```
600
```

Sale `600` y no `600.00`. ¿Por qué? Porque **para un ordenador, 600 y 600.00 son el mismo número**. Los ceros del final no aportan información, así que no los escribe.

Pero un precio se escribe con dos decimales siempre. Eso no es el valor, es el **formato**. Y para eso está `.toFixed()`:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(totalEuros.toFixed(2));
```

RESULTADO

```
600.00
```

`toFixed(2)` significa "escríbeme este número con exactamente 2 decimales". Rellena con ceros si faltan, y redondea si sobran: `(3.14159).toFixed(2)` da `"3.14"`.

Y un detalle crucial: devuelve un texto, no un número.

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(typeof totalEuros.toFixed(2));
console.log(totalEuros.toFixed(2) + 50);
```

RESULTADO

```
string
600.0050
```

Como es texto, el `+` no suma: pega. De ahí la regla:

> **IDEA CLAVE**
>
> `toFixed` es el último paso, solo para mostrar. Haz todos los cálculos con números, y formatea únicamente en el momento de enseñar el resultado. Y el dinero, siempre en céntimos.

24.4 Convertir texto en número

Cuando tengas un número guardado como texto (lo que escribe un usuario, por ejemplo), hay que convertirlo antes de operar:

JAVASCRIPT · EN TU FICHERO .JS

```
const cantidad = "5";
console.log(cantidad + 5);           // "55": pega
console.log(Number(cantidad) + 5);   // 10: suma
```

`Number()` es una función (no un método: la herramienta va delante y el dato dentro) que convierte un valor en número. Si no puede, devuelve `NaN`, que significa *Not a Number*, "no es un número": `Number("hola")` da `NaN`.

25. Leer los errores

Vas a ver errores constantemente. Todos los programadores los ven, todos los días. La diferencia entre quien empieza y quien tiene experiencia no es que uno se equivoque menos, sino que sabe leer lo que el error le está diciendo.

25.1 Anatomía de un error

ERROR

```
/Users/alvaro/aprendiendo-programacion/03-javascript/prueba.js:3
console.log("Hola, " + nombre)
                       ^

ReferenceError: nombre is not defined
    at Object.<anonymous> (/Users/alvaro/.../prueba.js:3:24)
    ...
```

Parte	Qué te dice
`prueba.js:3`	**Dónde**: el fichero y el número de línea.
La línea y el `^`	La línea exacta, y la flecha señala el punto donde se ha perdido.
`ReferenceError`	**El tipo** de error. Te dice qué clase de problema es.
`nombre is not defined`	**El mensaje**: qué ha pasado, en concreto.
Las líneas con `at`	El recorrido que hizo el programa hasta llegar ahí. Al principio puedes ignorarlo.

25.2 Los tres errores que más verás

1. SyntaxError — "está mal escrito". Ni siquiera ha empezado a ejecutarse: hay algo que JavaScript no entiende como frase. Como una falta de ortografía que impide leer la oración.

JAVASCRIPT · EN TU FICHERO .JS

```
console.log("Hola, " + nombre "!");
```

ERROR

```
SyntaxError: missing ) after argument list
```

Aquí falta un `+` entre `nombre` y `"!"`. Pero fíjate: el mensaje dice que falta un paréntesis, que no es lo que pasa. **Los errores de sintaxis a menudo describen mal el problema**, porque JavaScript solo sabe que "a partir de aquí ya no entiendo nada". Fíate más de dónde señala la flecha que de lo que dice el texto, y mira justo antes de ese punto.

Causas habituales: falta un `+`, una coma, un paréntesis o una comilla, o sobra alguno.

2. ReferenceError — "no conozco ese nombre". Usas una variable o función que no existe.

```
ReferenceError: nombre is not defined
```
ERROR

Causas habituales: una errata en el nombre (`nombe`), mayúsculas distintas (`Nombre`), o usar la variable antes de crearla. Recuerda que el programa se ejecuta de arriba abajo: si la usas en la línea 3 y la creas en la línea 5, en la línea 3 aún no existe.

3. TypeError — "eso no se puede hacer con ese valor". La frase está bien escrita y los nombres existen, pero la operación es imposible.

```
TypeError: Assignment to constant variable.
```
ERROR

```
TypeError: ciudad.toUppercase is not a function
```
ERROR

```
TypeError: Cannot read properties of null (reading 'length')
```
ERROR

Reasignar un `const` , llamar a un método que no existe, o pedirle algo a un valor vacío (`null` no tiene longitud, ni nada).

25.3 El método para cuando algo falla

1. **Lee el error entero.** Parece obvio, pero es lo que más se salta. Muchas veces el mensaje dice literalmente cuál es el problema.
2. **Ve a la línea que indica** y mira dónde señala la flecha.
3. **Identifica el tipo**: ¿mal escrito, nombre desconocido u operación imposible?
4. **Cambia una sola cosa** y vuelve a ejecutar. Nunca cambies varias a la vez, o no sabrás cuál lo arregló.
5. **Si no lo ves, usa** `console.log` justo antes de la línea que falla para ver qué valor tienen las variables en ese momento.

> **IDEA CLAVE**
>
> Un error te dice dónde (fichero y línea), qué tipo (SyntaxError: mal escrito; ReferenceError: nombre desconocido; TypeError: operación imposible) y qué ha pasado. Léelo entero, ve a la línea, y cambia una sola cosa cada vez.

26. Tomar decisiones

26.1 Por qué hace falta

Hasta ahora, tus programas hacen siempre lo mismo, línea tras línea. Pero casi todo programa útil necesita **hacer cosas distintas según la situación**: si la contraseña es correcta, entra; si no, avisa. Si hay stock, vende; si no, muestra "agotado".

26.2 Comparaciones: preguntas que responden sí o no

Para decidir, primero hay que preguntar. Las comparaciones son preguntas cuya respuesta es un booleano: `true` o `false`.

Símbolo	Pregunta	Ejemplo → resultado
`===`	¿Son iguales?	`5 === 5` → `true`
`!==`	¿Son distintos?	`5 !== 3` → `true`
`>` / `<`	¿Mayor? / ¿Menor?	`7 > 3` → `true`
`>=` / `<=`	¿Mayor o igual? / ¿Menor o igual?	`18 >= 18` → `true`

Fíjate en que para comparar se usan **tres** signos igual. Ya tienes tres usos distintos del `=`, y no hay que confundirlos:

Escritura	Significa
`=`	"Recibe": guarda un valor en una variable
`===`	"¿Son iguales?": compara valor y tipo
`==`	Comparación antigua que convierte tipos por su cuenta. **No la uses.**

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(5 === "5");    // false: número y texto no son lo mismo
console.log(5 == "5");     // true: convierte el texto a número sin avisar
```

Que `==` diga que un número y un texto son iguales es justo el tipo de sorpresa que genera fallos. Usa siempre `===`.

26.3 if y else

JAVASCRIPT · EN TU FICHERO .JS

```
const edad = 20;

if (edad >= 18) {
  console.log("Puedes entrar");
} else {
  console.log("Lo siento, eres menor");
}
```

RESULTADO

```
Puedes entrar
```

DESMONTANDO EL COMANDO

`if (condición) { ... } else { ... }`

`if`	"Si".
`(edad >= 18)`	La condición: una pregunta que da `true` o `false`. Siempre entre paréntesis.
`{ ... }`	Las llaves agrupan lo que se ejecuta si la condición es `true`. Se llama un **bloque**.
`else { ... }`	"Si no": lo que se ejecuta si es `false`. Es opcional.

Solo se ejecuta **uno** de los dos bloques, nunca los dos. El programa llega al `if`, hace la pregunta, entra por un camino y se salta el otro.

Las líneas dentro de las llaves van **sangradas** (con espacios delante). A JavaScript le da igual, pero a las personas no: la sangría muestra de un vistazo qué está dentro de qué. VS Code la pone solo.

Para más de dos caminos, se encadenan con `else if`:

JAVASCRIPT · EN TU FICHERO .JS

```
const hora = 15;

if (hora < 12) {
  console.log("Buenos días");
} else if (hora < 20) {
  console.log("Buenas tardes");
} else {
  console.log("Buenas noches");
}
```

RESULTADO

```
Buenas tardes
```

Se comprueban en orden, de arriba abajo, y se ejecuta el **primero** que se cumpla. En cuanto uno se cumple, los demás se ignoran.

26.4 Combinar condiciones

Símbolo	Se lee	Es true cuando…
`&&`	"y"	…las **dos** condiciones son verdad
`||`	"o"	…**al menos una** es verdad
`!`	"no"	…invierte: `!true` es `false`

JAVASCRIPT · EN TU FICHERO .JS

```
const edad = 20;
const tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
  console.log("Adelante");
}
```

Solo entra si es mayor de edad *y* tiene entrada. Fíjate en que `tieneEntrada` ya es un booleano, así que no hace falta escribir `tieneEntrada === true`: la variable misma es la respuesta.

IDEA CLAVE

Las comparaciones (`===`, `>`, `<`…) son preguntas que dan `true` o `false`. `if` ejecuta un bloque solo si la condición es `true`, y `else` el otro. Usa siempre `===`, nunca `==`.

27. Arrays: listas de valores

27.1 El problema

Quieres guardar tus géneros musicales favoritos. Con lo que sabes:

JAVASCRIPT · EN TU FICHERO .JS

```
const genero1 = "techno";
const genero2 = "house";
const genero3 = "minimal";
```

Funciona con tres. ¿Y con cincuenta? ¿Cómo sabes cuántos hay? ¿Cómo los recorres todos? Hace falta una forma de guardar **muchos valores bajo un solo nombre**.

27.2 Qué es un array

Un **array** (en español, a veces "arreglo" o "lista") es una lista ordenada de valores bajo un solo nombre. Si una variable es una caja con una cosa dentro, un array es una **estantería con casillas numeradas**.

JAVASCRIPT · EN TU FICHERO .JS

```
const generos = ["techno", "house", "minimal"];
```

Corchetes `[ ]` para abrir y cerrar la lista, y comas entre los elementos.

27.3 La numeración empieza en cero

Cada casilla tiene un número, llamado **índice**. Y aquí está lo que despista a todo el mundo al principio: **el primero es el 0**, no el 1.

```
const generos = [ "techno", "house", "minimal" ];
//                  0         1         2
```

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(generos[0]);       // el primero
console.log(generos[2]);       // el tercero
console.log(generos[3]);       // no existe
console.log(generos.length);   // cuántos hay
```

RESULTADO

```
techno
minimal
undefined
3
```

Para sacar un elemento, se escribe el nombre del array y el índice entre corchetes. Si pides una casilla que no existe, no da error: da `undefined`, "no hay nada aquí".

Una consecuencia de empezar en 0: **el último elemento siempre está en la posición `length - 1`**. Con 3 elementos, el último es el `[2]`. Confundirse en uno es tan frecuente que tiene nombre propio en inglés: *off-by-one error*.

27.4 Qué puede contener

Cualquier tipo de valor, incluso mezclados, incluso otros arrays:

JAVASCRIPT · EN TU FICHERO .JS

```
const numeros = [10, 20, 30];
const mezcla = ["texto", 42, true];
const vacio = [];
```

27.5 Modificar un array

JAVASCRIPT · EN TU FICHERO .JS

```
const generos = ["techno", "house"];

generos.push("minimal");     // añade al final
console.log(generos);

generos.pop();               // quita el último
console.log(generos);

generos[0] = "acid";         // cambia el de la posición 0
console.log(generos);
```

RESULTADO

```
[ 'techno', 'house', 'minimal' ]
[ 'techno', 'house' ]
[ 'acid', 'house' ]
```

Pero un momento: `generos` es `const`, y lo hemos cambiado tres veces sin ningún error. ¿No decía el capítulo 22 que `const` no se puede cambiar? La respuesta es importante y tiene su propio capítulo, el 29.

27.6 Métodos útiles de los arrays

Método	Qué hace
`.push(x)`	Añade x al final
`.pop()`	Quita el último (y te lo devuelve)
`.includes(x)`	¿Está x en la lista? → `true` / `false`
`.indexOf(x)`	¿En qué posición está x? (`-1` si no está)
`.join(", ")`	Une todos los elementos en un solo texto, con lo que le digas en medio
`.length`	Cuántos elementos tiene (propiedad: sin paréntesis)

JAVASCRIPT · EN TU FICHERO .JS

```
const generos = ["techno", "house", "minimal"];
console.log(generos.includes("house"));
console.log(generos.join(" · "));
```

RESULTADO

```
true
techno · house · minimal
```

IDEA CLAVE

Un array es una lista ordenada bajo un solo nombre: `[a, b, c]`. Las posiciones empiezan en 0, y el último está en `length - 1`. `push` añade, `pop` quita, `includes` pregunta.

28. Objetos: fichas con campos

28.1 El problema

Quieres guardar los datos de una sesión: su nombre, cuánto dura, dónde es, si está confirmada. Podrías usar cuatro variables sueltas, pero esos datos van juntos: son *la misma cosa*. Hace falta una forma de agruparlos.

28.2 Qué es un objeto

Un **objeto** es un conjunto de datos con nombre, agrupados. Piensa en una **ficha** de papel con campos: "Nombre: ___", "Duración: ___", "Lugar: ___".

JAVASCRIPT · EN TU FICHERO .JS

```
const sesion = {
  nombre: "Sesión Somo",
  duracion: 90,
  lugar: "Somo",
  confirmada: true,
};
```

Llaves `{ }` para abrir y cerrar. Dentro, parejas de **clave: valor**, separadas por comas. La clave es el nombre del campo; el valor, su contenido, que puede ser de cualquier tipo.

Compáralo con un array: en un array, cada elemento se identifica por su **posición** (0, 1, 2); en un objeto, por su **nombre**. Usa un array para una lista de cosas del mismo tipo, y un objeto para describir las características de una sola cosa.

28.3 Leer y cambiar campos

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(sesion.nombre);
console.log(sesion.duracion);

sesion.duracion = 120;          // cambiar un campo
sesion.precio = 15000;          // añadir un campo nuevo

console.log(sesion);
```

RESULTADO

```
Sesión Somo
90
{
  nombre: 'Sesión Somo',
  duracion: 120,
  lugar: 'Somo',
  confirmada: true,
  precio: 15000
}
```

El punto significa "de": `sesion.nombre` es "el nombre de la sesión". Es exactamente el mismo punto que en `console.log` o `ciudad.toUpperCase()`. `console` es un objeto, `log` es uno de sus campos, y resulta que ese campo contiene una acción. Todo encaja.

28.4 Arrays de objetos

Esta combinación es, con diferencia, la forma más habitual de los datos en programas reales: **una lista de fichas**. Una lista de usuarios, de productos, de pedidos, de sesiones.

JAVASCRIPT · EN TU FICHERO .JS

```
const sesiones = [
  { nombre: "Somo", duracion: 90, precio: 15000 },
  { nombre: "Santander", duracion: 120, precio: 20000 },
  { nombre: "Laredo", duracion: 60, precio: 10000 },
];

console.log(sesiones[1].nombre);
console.log(sesiones.length);
```

RESULTADO

```
Santander
3
```

`sesiones[1].nombre` se lee de izquierda a derecha: "de la lista sesiones, la ficha en la posición 1, su campo nombre".

IDEA CLAVE

Un objeto agrupa datos con nombre: `{ clave: valor }`. Se leen y cambian con el punto: `objeto.campo`. Un array de objetos es una lista de fichas, y es la forma más común de los datos reales.

29. Valor y referencia

Este capítulo responde a la pregunta que quedó abierta en el 27: ¿por qué se puede cambiar un array declarado con const? La respuesta explica también uno de los comportamientos más sorprendentes de JavaScript.

29.1 Lo que pasa con los números y los textos

Recuerda el experimento del capítulo 22:

JAVASCRIPT · EN TU FICHERO .JS

```
let a = 5;
let b = a;
a = 10;
console.log(b);    // 5
```

`b` recibió una **copia del valor**. Desde ese momento, `a` y `b` son independientes. Como una fotocopia: si tachas el original, la fotocopia no se tacha.

29.2 Lo que pasa con los arrays y los objetos

Ahora el mismo experimento con un array:

JAVASCRIPT · EN TU FICHERO .JS

```
const lista = [1, 2, 3];
const copia = lista;
copia.push(4);
console.log(lista);
```

RESULTADO

```
[ 1, 2, 3, 4 ]
```

¡`lista` también tiene el 4! Aunque se lo añadimos a `copia`.

La explicación es que con arrays y objetos, la variable **no contiene** la lista. Contiene una **flecha que apunta a** la lista, que vive en otro sitio de la memoria. Y `const copia = lista` copia la flecha, no la lista:

```
lista  ──┐
         ├──►  [ 1, 2, 3, 4 ]      ← una sola lista
copia  ──┘
```

Dos nombres, **una sola lista**. Como dos puertas que dan a la misma habitación: entres por donde entres, la habitación es la misma. A esa flecha se le llama **referencia**.

29.3 La regla

Tipo de valor	Al hacer `b = a`, se copia...	¿Quedan conectadas?
Número, texto, booleano, `null`, `undefined`	El valor	**No**
Array, objeto	La flecha (referencia)	**Sí**

29.4 Y así se entiende const

`const` congela la flecha, no lo que hay al otro lado.

JAVASCRIPT · EN TU FICHERO .JS

```javascript
const generos = ["techno", "house"];

generos.push("minimal");    // permitido: cambias el contenido, la flecha no se mueve
generos = ["otra lista"];   // ERROR: intentas apuntar la flecha a otra lista
```

ERROR

```
TypeError: Assignment to constant variable.
```

`push` mete algo en la estantería; la etiqueta sigue apuntando a la misma estantería. Pero `generos = [...]` intenta despegar la etiqueta y pegarla en otra estantería distinta, y eso es lo que `const` prohíbe.

Por eso se dice que **`const` no significa "inmutable"** (que no puede cambiar). Significa "este nombre siempre apuntará a lo mismo".

29.5 Dos preguntas distintas

La confusión desaparece cuando ves que hay dos preguntas independientes:

- **¿`let` o `const`?** Decide si puedes volver a asignar ese nombre a otra cosa.
- **¿Primitivo u objeto?** Decide si, al copiar, se copia el valor o se comparte.

	Número / texto	Array / objeto
`let`	Se copia el valor. Se puede reasignar.	Se comparte. Se puede reasignar.
`const`	Se copia el valor. No se puede reasignar.	Se comparte. No se puede reasignar, **pero sí modificar el contenido**.

29.6 Cómo hacer una copia de verdad

Si necesitas una lista independiente, hay que crear una nueva con el mismo contenido. La forma habitual son los tres puntos:

JAVASCRIPT · EN TU FICHERO .JS

```
const lista = [1, 2, 3];
const copiaReal = [...lista];
copiaReal.push(4);

console.log(lista);
console.log(copiaReal);
```

RESULTADO

```
[ 1, 2, 3 ]
[ 1, 2, 3, 4 ]
```

`[...lista]` significa "una lista nueva con todos los elementos de `lista` desparramados dentro". Es otra estantería, con los mismos libros. Con objetos funciona igual: `{ ...objeto }`.

POR QUÉ ESTO IMPORTARÁ MUCHO MÁS ADELANTE

Cuando llegues a React (Volumen 2), verás la regla "no modifiques el estado directamente, crea uno nuevo". Esa regla existe exactamente por esto: React detecta los cambios mirando si la flecha es nueva. Si solo cambias el contenido, la flecha es la misma y React cree que no ha cambiado nada. Lo que acabas de aprender es la base de esa regla.

IDEA CLAVE

Los números y textos se copian; los arrays y objetos se comparten, porque la variable guarda una flecha y no el contenido. `const` congela la flecha, no el contenido. Para una copia independiente: `[...lista]` o `{ ...objeto }`.

30. Funciones

30.1 El problema

Supón que tienes que mostrar precios en varios sitios del programa. Cada vez escribes:

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(`${(precio / 100).toFixed(2)} €`);
```

Lo repites diez veces. Un día decides que el precio debe llevar coma en vez de punto. Tienes que cambiarlo en diez sitios, y seguro que te dejas uno.

30.2 Qué es una función

Una **función** es un bloque de código con nombre que puedes usar tantas veces como quieras. Es una **receta con nombre**: la escribes una vez ("tortilla de patatas: pela, corta, fríe…"), y a partir de ahí basta con decir "haz una tortilla".

Resuelve el problema de antes: la lógica está en un solo sitio. Si cambia, la cambias ahí y se aplica en todas partes.

30.3 Definir y usar una función

JAVASCRIPT · EN TU FICHERO .JS

```
function formatearPrecio(centimos) {
  const euros = centimos / 100;
  return `${euros.toFixed(2)} €`;
}

console.log(formatearPrecio(1999));
console.log(formatearPrecio(60000));
```

RESULTADO

```
19.99 €
600.00 €
```

DESMONTANDO EL COMANDO

`function formatearPrecio(centimos) { ... }`

`function`	"Voy a definir una función".
`formatearPrecio`	Su nombre. Como es una acción, empieza por un verbo.
`(centimos)`	El **parámetro**: el dato que la función necesita para trabajar. Es como una variable que se rellena cada vez que se usa la función.
`{ ... }`	El cuerpo: lo que hace.
`return`	"Devuelve este resultado a quien me llamó". Y termina la función.

Hay dos momentos distintos, y conviene no mezclarlos:

1. **Definirla** (`function formatearPrecio(...) {...}`): escribes la receta. No se ejecuta nada todavía.
2. **Llamarla** (`formatearPrecio(1999)`): la usas. Ahora sí se ejecuta, con `centimos` valiendo 1999. Los paréntesis son los que la ejecutan, igual que con los métodos.

30.4 return frente a console.log

Esta es la confusión número uno con las funciones, así que vale la pena detenerse.

- `console.log` escribe algo en la pantalla, para que *tú* lo veas. El programa no puede usar ese valor para nada más.
- `return` devuelve un valor *al programa*, que puede guardarlo en una variable, usarlo en un cálculo o pasárselo a otra función.

JAVASCRIPT · EN TU FICHERO .JS

```
function dobleMal(n) {
  console.log(n * 2);      // lo enseña, pero no lo devuelve
}

function dobleBien(n) {
  return n * 2;            // lo devuelve
}

const a = dobleMal(5);
const b = dobleBien(5);
console.log("a vale:", a);
console.log("b vale:", b);
```

RESULTADO

```
10
a vale: undefined
b vale: 10
```

La primera función escribió 10 en pantalla, pero no devolvió nada, así que `a` se quedó en `undefined`. La segunda no escribió nada, pero devolvió 10, y `b` lo recibió.

La comparación: `console.log` es decir el resultado en voz alta; `return` es dártelo por escrito en un papel. Si solo lo dices en voz alta, no te lo puedes llevar.

30.5 Varios parámetros

JAVASCRIPT · EN TU FICHERO .JS

```
function calcularTotal(precioUnidad, cantidad) {
  return precioUnidad * cantidad;
}

const total = calcularTotal(15000, 4);
console.log(formatearPrecio(total));
```

RESULTADO

```
600.00 €
```

Los parámetros se separan con comas, y al llamar a la función los valores se asignan **en el mismo orden**: el primero a `precioUnidad`, el segundo a `cantidad`.

Y fíjate en la última línea: el resultado de una función (`total`) se pasa a otra (`formatearPrecio`). Así se construyen los programas: funciones pequeñas que hacen una cosa cada una, combinadas.

30.6 La forma corta: funciones flecha

Verás muchas funciones escritas así:

JAVASCRIPT · EN TU FICHERO .JS

```
const doble = (n) => n * 2;
```

Es lo mismo que `function doble(n) { return n * 2; }`, más corto. La flecha `=>` se lee "da como resultado". Cuando el cuerpo es una sola expresión, el `return` va implícito. No hace falta que las uses ahora, pero sí que las reconozcas.

> **IDEA CLAVE**
>
> Una función es una receta con nombre: se define una vez y se llama las veces que haga falta. Recibe datos por sus parámetros y devuelve un resultado con `return`. `console.log` enseña; `return` entrega.

31. Bucles

31.1 El problema

Tienes una lista de sesiones y quieres mostrar el nombre de cada una. Podrías escribir `console.log(sesiones[0].nombre)`, `console.log(sesiones[1].nombre)`... ¿Y si hay cien? ¿Y si no sabes cuántas hay?

Un **bucle** repite un bloque de código automáticamente. Es una de las cosas en las que los ordenadores son imbatibles: repetir sin cansarse ni equivocarse.

31.2 for...of: recorrer una lista

JAVASCRIPT · EN TU FICHERO .JS

```
const generos = ["techno", "house", "minimal"];

for (const genero of generos) {
  console.log(`Me gusta el ${genero}`);
}
```

RESULTADO

```
Me gusta el techno
Me gusta el house
Me gusta el minimal
```

DESMONTANDO EL COMANDO

`for (const genero of generos) { ... }`

`for`	"Para".
`const genero`	Una variable que, en cada vuelta, contendrá un elemento distinto. El nombre lo eliges tú; lo lógico es el singular de la lista.
`of generos`	"De la lista generos".
`{ ... }`	Lo que se repite, una vez por cada elemento.

Se lee: *"para cada género de la lista de géneros, haz esto"*. En la primera vuelta, `genero` vale "techno"; en la segunda, "house"; en la tercera, "minimal". Luego se acaba la lista y el bucle termina.

31.3 El patrón del acumulador

Este patrón lo usarás muchísimo: recorrer una lista para calcular un total.

JAVASCRIPT · EN TU FICHERO .JS

```
const sesiones = [
  { nombre: "Somo", duracion: 90 },
  { nombre: "Santander", duracion: 120 },
  { nombre: "Laredo", duracion: 60 },
];

let totalMinutos = 0;

for (const sesion of sesiones) {
  totalMinutos = totalMinutos + sesion.duracion;
}

console.log(`Total: ${totalMinutos} minutos`);
```

RESULTADO

```
Total: 270 minutos
```

Síguelo vuelta a vuelta, que es la mejor forma de entender un bucle:

Momento	`sesion.duracion`	`totalMinutos`
Antes del bucle	—	0
Vuelta 1 (Somo)	90	0 + 90 = 90
Vuelta 2 (Santander)	120	90 + 120 = 210
Vuelta 3 (Laredo)	60	210 + 60 = 270

Tres detalles que hacen que funcione:

- `totalMinutos` se crea **antes** del bucle, a 0. Si lo crearas dentro, se reiniciaría en cada vuelta.
- Es `let`, no `const`, porque su valor cambia.
- `totalMinutos = totalMinutos + ...` es el "recibe" del capítulo 22: coge el valor actual, súmale algo y guárdalo.

Se puede escribir más corto: `totalMinutos += sesion.duracion` significa exactamente lo mismo.

31.4 Bucles con decisiones dentro

Combinando bucles e `if` ya puedes filtrar:

JAVASCRIPT · EN TU FICHERO .JS

```
for (const sesion of sesiones) {
  if (sesion.duracion >= 90) {
    console.log(`${sesion.nombre}: sesión larga`);
  }
}
```

RESULTADO

```
Somo: sesión larga
Santander: sesión larga
```

31.5 El for clásico: repetir un número de veces

Cuando no tienes una lista, sino que quieres repetir algo un número concreto de veces:

JAVASCRIPT · EN TU FICHERO .JS

```
for (let i = 1; i <= 5; i++) {
  console.log(`Vuelta número ${i}`);
}
```

DESMONTANDO EL COMANDO

`for (let i = 1; i <= 5; i++)`

`let i = 1`	Antes de empezar: crea un contador que vale 1. Se llama `i` por costumbre (de "índice").
`i <= 5`	Antes de cada vuelta: ¿sigue siendo `i` menor o igual que 5? Si sí, otra vuelta. Si no, se acaba.
`i++`	Después de cada vuelta: suma 1 a `i`. `i++` es la forma corta de `i = i + 1`.

Tres partes separadas por punto y coma: inicio, condición para seguir, y qué hacer al final de cada vuelta.

EL BUCLE INFINITO

Si la condición nunca deja de cumplirse (por ejemplo, si olvidas el `i++`), el bucle no termina nunca y el programa se queda colgado. No pasa nada grave: pulsa `Ctrl` + `C` en la terminal para pararlo, como aprendiste en el capítulo 9.

IDEA CLAVE

`for (const x of lista)` repite algo para cada elemento de una lista. Para calcular totales, crea una variable antes del bucle y ve acumulando en ella. `for (let i = 1; i <= n; i++)` repite algo n veces. Si se cuelga, `Ctrl` + `C`.

Ejercicios · Parte 3

Crea un fichero distinto para cada ejercicio dentro de `03-javascript` (por ejemplo `ej01.js`, `ej02.js`…) y ejecútalos con `node`. Así no tendrás el problema de la consola del navegador, y al final podrás subirlos a GitHub.

EJERCICIO 3.1 · PREDECIR TIPOS

Sin ejecutarlo, escribe en un papel qué crees que sale en cada línea. Luego ejecútalo y compara.

JAVASCRIPT · EN TU FICHERO .JS

```
console.log(typeof "42");
console.log(typeof 42);
console.log("3" + 3);
console.log(3 + 3);
console.log("3" * 3);
```

EJERCICIO 3.2 · PLANTILLAS

Con tres constantes `nombre`, `edad` y `ciudad`, y usando **una sola plantilla**, escribe:

```
Me llamo Álvaro, tengo 25 años y en 2030 tendré 29. Vivo en SANTANDER.
```

La edad de 2030 y la ciudad en mayúsculas tienen que calcularse dentro de la plantilla, no escribirse a mano.

EJERCICIO 3.3 · PRECIOS EN CÉNTIMOS

Una sesión cuesta 15000 céntimos y se contratan 4. Calcula el total, guárdalo en una variable, y muéstralo exactamente así: `Total: 600.00 €`.

EJERCICIO 3.4 · VALOR Y REFERENCIA

Sin ejecutar, ¿qué escribe cada bloque? Explica por qué son distintos.

JAVASCRIPT · EN TU FICHERO .JS

```javascript
// Bloque A
let x = 10;
let y = x;
x = 20;
console.log(y);

// Bloque B
const lista = ["a", "b"];
const otra = lista;
otra.push("c");
console.log(lista.length);
```

EJERCICIO 3.5 · UNA FUNCIÓN DE PRECIOS

Escribe una función `formatearPrecio` que reciba céntimos y **devuelva** el precio en formato español, con coma decimal. Por ejemplo, `formatearPrecio(1999)` debe devolver `"19,99 €"`. Pruébala con 1999, 60000 y 5.

EJERCICIO 3.6 · DECISIONES

Escribe una función `clasificarEdad` que reciba una edad y devuelva `"menor"` si es menor de 18, `"jubilado"` si tiene 67 o más, y `"adulto"` en cualquier otro caso. Pruébala con 12, 18, 25, 67 y 70. Presta atención a los casos 18 y 67.

EJERCICIO 3.7 · BUCLES

Muestra en pantalla todos los números pares del 1 al 20. Pista: el operador `%` del capítulo 24.

EJERCICIO 3.8 · UNA LISTA DE FICHAS

Con esta lista:

JAVASCRIPT · EN TU FICHERO .JS

```javascript
const sesiones = [
  { nombre: "Somo", duracion: 90, precio: 15000 },
  { nombre: "Santander", duracion: 120, precio: 20000 },
  { nombre: "Laredo", duracion: 60, precio: 10000 },
];
```

1. Calcula el precio total de todas las sesiones y muéstralo con la función del ejercicio 3.5.
2. Averigua cuál es la sesión más larga y muestra su nombre y duración.

EJERCICIO 3.9 · DETECTIVE DE ERRORES

Cada uno de estos fragmentos tiene un error. Para cada uno, di qué tipo de error dará, por qué, y cómo se arregla. Luego compruébalo ejecutándolos.

JAVASCRIPT · EN TU FICHERO .JS

```javascript
// A
const lista = ["a", "b" "c"];

// B
const total = 5;
console.log("Total: " + totl);

// C
const nombre = "ana";
console.log(nombre.toUppercase());

// D
const precio = 100;
precio = 200;
```

Soluciones · Parte 3

3.1 · Predecir tipos

RESULTADO

```
string
number
33
6
9
```

Las dos primeras: `"42"` lleva comillas, así que es texto aunque contenga cifras; `42` sin comillas es un número.

`"3" + 3` da `33`: cuando hay un texto a cualquier lado del `+`, pega en lugar de sumar. `3 + 3` da `6` porque los dos son números.

La sorpresa es la última: `"3" * 3` da `9`, no un error. Como `*` **solo** tiene sentido con números (no se pueden "multiplicar" textos), JavaScript convierte el texto en número por su cuenta. Solo el `+` tiene ese doble significado de sumar o pegar. Esta inconsistencia es famosa, y es una razón más para convertir siempre los textos con `Number()` antes de operar, en lugar de confiar en que JavaScript adivine.

3.2 · Plantillas

JAVASCRIPT · EN TU FICHERO .JS

```javascript
const nombre = "Álvaro";
const edad = 25;
const ciudad = "Santander";

console.log(`Me llamo ${nombre}, tengo ${edad} años y en 2030 tendré ${edad + 4}. Vivo en
${ciudad.toUpperCase()}.`);
```

Cuatro huecos `${ }`, y en cada uno algo distinto: una variable (`nombre`), otra variable (`edad`), una **operación** (`edad + 4`, porque de 2026 a 2030 van cuatro años) y un **método** (`ciudad.toUpperCase()`).

Los dos fallos más comunes en este ejercicio: usar comillas normales en lugar de invertidas (entonces aparece literalmente `${nombre}` en pantalla), y escribir `toUpperCase(ciudad)` en lugar de `ciudad.toUpperCase()`. Lo segundo da `ReferenceError: toUpperCase is not defined`, porque no existe como función suelta: es un método, una habilidad del propio texto.

3.3 · Precios en céntimos

JAVASCRIPT · EN TU FICHERO .JS

```javascript
const precioSesion = 15000;
const numSesiones = 4;

const totalCentimos = precioSesion * numSesiones;
const totalEuros = totalCentimos / 100;

console.log(`Total: ${totalEuros.toFixed(2)} €`);
```

Tres pasos: multiplicar en céntimos (`60000`, entero, exacto), pasar a euros (`600`), y formatear con dos decimales solo al mostrar.

El `.toFixed(2)` va dentro del `${ }` y no en la variable, a propósito: así `totalEuros` sigue siendo un número con el que se puede seguir calculando. Si hubieras guardado `totalEuros.toFixed(2)` en la variable, sería el texto `"600.00"`, y sumarle algo después lo pegaría en lugar de sumarlo.

3.4 · Valor y referencia

RESULTADO

```
10
3
```

Bloque A: `y` recibe una copia del valor de `x` en ese momento, 10. Después `x` cambia a 20, pero `y` no se entera: no hay ningún vínculo entre ellas. Los números se copian.

Bloque B: `otra` no recibe una copia de la lista, sino una copia de la flecha que apunta a ella. Las dos variables apuntan a la **misma** lista, así que el `"c"` añadido a través de `otra` aparece también en `lista`. Los arrays y objetos se comparten.

Y fíjate: los dos del bloque B son `const` y no hay error. `push` modifica el contenido sin mover la flecha, y `const` solo congela la flecha.

3.5 · Una función de precios

JAVASCRIPT · EN TU FICHERO .JS

```javascript
function formatearPrecio(centimos) {
  const euros = (centimos / 100).toFixed(2);
  return `${euros.replace(".", ",")} €`;
}

console.log(formatearPrecio(1999));
console.log(formatearPrecio(60000));
console.log(formatearPrecio(5));
```

RESULTADO

```
19,99 €
600,00 €
0,05 €
```

Paso a paso, con 1999:

1. `centimos / 100` → `19.99` (número).
2. `.toFixed(2)` → `"19.99"` (ya es texto). Los paréntesis alrededor de `centimos / 100` son necesarios: sin ellos, `.toFixed` se aplicaría solo al `100`.
3. `.replace(".", ",")` → `"19,99"`. Esto funciona precisamente *porque* `toFixed` devolvió un texto: `replace` es un método de los textos.
4. La plantilla le añade el espacio y el símbolo del euro.

El caso de 5 céntimos es el que comprueba que la función es robusta: `0.05` se convierte correctamente en `0,05 €`. Probar con casos extremos (muy pequeños, cero, muy grandes) es una costumbre que conviene coger desde ya.

Y fíjate en que la función **devuelve** con `return`, no escribe con `console.log`. Así se puede reutilizar: en el ejercicio 3.8 la usarás dentro de otra frase.

3.6 · Decisiones

JAVASCRIPT · EN TU FICHERO .JS

```
function clasificarEdad(edad) {
  if (edad < 18) {
    return "menor";
  } else if (edad < 67) {
    return "adulto";
  } else {
    return "jubilado";
  }
}

console.log(clasificarEdad(12));
console.log(clasificarEdad(18));
console.log(clasificarEdad(25));
console.log(clasificarEdad(67));
console.log(clasificarEdad(70));
```

RESULTADO

```
menor
adulto
adulto
jubilado
jubilado
```

La clave está en el orden y en los límites. Las condiciones se comprueban de arriba abajo y se ejecuta la primera que se cumpla. Por eso la segunda solo necesita preguntar `edad < 67`: si has llegado hasta ahí, ya sabes que *no* es menor de 18, porque en ese caso habría entrado en la primera.

Los casos límite: con 18, `18 < 18` es `false`, así que no es "menor", y pasa a la siguiente: adulto. Correcto. Con 67, `67 < 67` es `false`, así que cae en el `else`: jubilado. Correcto. Si hubieras escrito `<=` en lugar de `<`, los dos límites saldrían mal. Los fallos de programación se esconden justo en los límites; por eso siempre hay que probarlos.

3.7 · Bucles

JAVASCRIPT · EN TU FICHERO .JS

```
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) {
    console.log(i);
  }
}
```

RESULTADO

```
2
4
6
...
20
```

El bucle recorre del 1 al 20. En cada vuelta, `i % 2` calcula el resto de dividir entre 2: da 0 para los pares y 1 para los impares. Solo si es 0 se muestra.

Hay otra solución, más eficiente: empezar en 2 y avanzar de dos en dos, cambiando `i++` por `i += 2`. Así no hace falta el `if`. Las dos son correctas; la primera muestra cómo combinar bucle y decisión, y la segunda que casi siempre hay más de una forma de resolver algo.

3.8 · Una lista de fichas

JAVASCRIPT · EN TU FICHERO .JS

```
// (con la función formatearPrecio del ejercicio 3.5 arriba en el mismo fichero)

let totalCentimos = 0;
for (const sesion of sesiones) {
  totalCentimos += sesion.precio;
}
console.log(`Total: ${formatearPrecio(totalCentimos)}`);

let masLarga = sesiones[0];
for (const sesion of sesiones) {
  if (sesion.duracion > masLarga.duracion) {
    masLarga = sesion;
  }
}
console.log(`La más larga: ${masLarga.nombre} (${masLarga.duracion} min)`);
```

RESULTADO

```
Total: 450,00 €
La más larga: Santander (120 min)
```

El total es el patrón del acumulador del capítulo 31: una variable a 0 antes del bucle, y en cada vuelta se le suma el precio de una sesión. `+=` es la forma corta de `totalCentimos = totalCentimos + sesion.precio`.

La más larga usa otra variante del mismo patrón, "el mejor hasta ahora". Empiezas suponiendo que la primera es la más larga. Luego recorres todas, y cada vez que encuentras una más larga que la que tienes guardada, la

sustituyes. Al terminar, la que queda es la más larga de todas. Es la misma lógica que usarías tú a mano si fueras mirando fichas una a una.

Fíjate en que aquí `masLarga` guarda una flecha a una de las fichas de la lista, no una copia. Por eso puedes leer `masLarga.nombre` y `masLarga.duracion` directamente.

3.9 · Detective de errores

A:

```
ERROR
SyntaxError: Unexpected string
```

Falta una coma entre `"b"` y `"c"`. Es un error de sintaxis: la frase está mal construida. El mensaje ("texto inesperado") dice que JavaScript encontró un texto donde esperaba una coma o el cierre del corchete. Se arregla: `["a", "b", "c"]`.

B:

```
ERROR
ReferenceError: totl is not defined
```

Errata en el nombre: la variable es `total`, no `totl`. Es un error de referencia: usas un nombre que no existe. Se arregla escribiéndolo bien. El editor te ayuda: si al escribir `tot` aceptas la sugerencia de VS Code, nunca tendrás esta errata.

C:

```
ERROR
TypeError: nombre.toUppercase is not a function
```

La C de `Case` va en mayúscula: `toUpperCase`. Es un error de tipo: JavaScript buscó un método llamado exactamente `toUppercase` en el texto, no lo encontró, y al intentar ejecutarlo con `()` te dice que eso no es algo ejecutable.

D:

```
ERROR
TypeError: Assignment to constant variable.
```

Se intenta dar otro valor a una variable declarada con `const`. Si el precio tiene que cambiar, se declara con `let`. Y si no debería cambiar, el error te ha salvado de un fallo: para eso existe `const`.

El resumen de los cuatro: A es un `SyntaxError` (mal escrito), B un `ReferenceError` (nombre desconocido), y C y D son `TypeError` (operación imposible). Con reconocer esos tres tipos ya sabes por dónde empezar a buscar ante casi cualquier error.

PARTE 4

Proyecto final: todo junto

Un programa completo que usa todo lo aprendido: la terminal para crearlo y ejecutarlo, JavaScript para escribirlo, y Git y GitHub para guardarlo y publicarlo. Lo construiremos paso a paso, ejecutando después de cada paso.

- 32. La calculadora de presupuestos
- 33. Guardarlo y publicarlo

32. La calculadora de presupuestos

32.1 Qué vamos a construir

Un programa que, a partir de una lista de servicios con su precio y cantidad, calcule y muestre un presupuesto completo: cada línea, el subtotal, un descuento si el importe supera cierta cantidad, el IVA y el total. Así será el resultado final:

RESULTADO

```
PRESUPUESTO
------------------------------------
Sesión DJ (horas)            320,00 €
Equipo de sonido             250,00 €
Iluminación                  120,00 €
Desplazamiento (km)           18,00 €
------------------------------------
Subtotal                     708,00 €
Descuento 10 %               -70,80 €
IVA 21 %                     133,81 €
====================================
TOTAL                        771,01 €
```

Parece mucho, pero todo lo que hace ya lo sabes hacer: un array de objetos, funciones, un bucle con acumulador, una decisión, precios en céntimos y plantillas. Lo único nuevo son tres métodos de texto para alinear las columnas, que se explican cuando aparecen.

32.2 Antes de escribir: pensar

Un hábito que distingue a quien programa bien: **antes de tocar el teclado, escribe en palabras normales qué tiene que hacer el programa**. Así:

1. Tener una lista de servicios, cada uno con concepto, precio por unidad y cantidad.
2. Para cada servicio, calcular su importe (precio × cantidad) y mostrarlo.
3. Ir sumando los importes para tener el subtotal.
4. Si el subtotal es de 500 € o más, aplicar un 10 % de descuento.
5. Calcular el IVA sobre lo que queda.
6. Mostrar subtotal, descuento, IVA y total.

Cada punto de esa lista se convertirá en unas pocas líneas de código. Esto tiene nombre: se llama descomponer un problema, y es la habilidad más importante del oficio.

32.3 Paso 1 — Crear el fichero

TÚ ESCRIBES

```
cd ~/aprendiendo-programacion
mkdir 04-proyecto
cd 04-proyecto
code presupuesto.js
```

Ya sabes qué hace cada comando: ir al repositorio, crear una carpeta, entrar en ella, y crear y abrir el fichero en VS Code.

32.4 Paso 2 — Los datos

Escribe esto en el fichero:

JAVASCRIPT · EN TU FICHERO .JS

```
// presupuesto.js — Calcula y muestra el presupuesto de un evento

// ---------- 1. Los datos ----------
const servicios = [
  { concepto: "Sesión DJ (horas)", precioUnidad: 8000, cantidad: 4 },
  { concepto: "Equipo de sonido", precioUnidad: 25000, cantidad: 1 },
  { concepto: "Iluminación", precioUnidad: 12000, cantidad: 1 },
  { concepto: "Desplazamiento (km)", precioUnidad: 30, cantidad: 60 },
];

const IVA = 0.21;
const UMBRAL_DESCUENTO = 50000; // a partir de 500 € hay descuento
const DESCUENTO = 0.1;          // 10 %

console.log(servicios.length);
```

Qué hay aquí:

- **Un array de objetos** (capítulo 28): una lista de fichas, cada una con tres campos.
- **Precios en céntimos** (capítulo 24): 8000 son 80 €; 30 son 30 céntimos por kilómetro.
- **Tres constantes con nombre en mayúsculas.** Es una convención para los valores de configuración que no cambian. Tienen una ventaja enorme: si el IVA cambia, se cambia en una sola línea. Y el código se lee mejor: `subtotal * IVA` se entiende; `subtotal * 0.21` obliga a adivinar qué es ese 0.21. A los números sueltos sin explicación se les llama, con razón, "números mágicos".

La última línea es provisional: sirve para comprobar que los datos están bien antes de seguir. Guarda y ejecuta:

TÚ ESCRIBES

```
node presupuesto.js
```

RESULTADO

```
4
```

Cuatro servicios. Bien. Esta forma de trabajar (escribir un poco, comprobar, seguir) es la misma que usaste para construir la cadena de comandos del capítulo 8.

32.5 Paso 3 — Las funciones

Borra el `console.log` provisional y añade debajo de los datos:

JAVASCRIPT · EN TU FICHERO .JS

```
// ---------- 2. Funciones ----------
function formatearPrecio(centimos) {
  const euros = (centimos / 100).toFixed(2);
  return `${euros.replace(".", ",")} €`;
}

function calcularLinea(servicio) {
  return servicio.precioUnidad * servicio.cantidad;
}

console.log(formatearPrecio(calcularLinea(servicios[0])));
```

`formatearPrecio` es la del ejercicio 3.5. `calcularLinea` recibe una ficha de servicio y devuelve su importe.

La línea de prueba se lee de dentro hacia fuera: coge el primer servicio (`servicios[0]`), calcula su importe, formatéalo y escríbelo.

RESULTADO

```
320,00 €
```

8000 céntimos × 4 horas = 32000 céntimos = 320,00 €. Correcto. **Probar cada función por separado** antes de usarla en el programa completo te ahorra muchísimo tiempo: si algo falla después, ya sabes que no es aquí.

32.6 Paso 4 — El bucle

Borra la línea de prueba y añade:

JAVASCRIPT · EN TU FICHERO .JS

```
// ---------- 3. Las líneas del presupuesto ----------
console.log("PRESUPUESTO");

let subtotal = 0;
for (const servicio of servicios) {
  const importe = calcularLinea(servicio);
  subtotal += importe;
  console.log(`${servicio.concepto}: ${formatearPrecio(importe)}`);
}

console.log(`Subtotal: ${formatearPrecio(subtotal)}`);
```

RESULTADO

```
PRESUPUESTO
Sesión DJ (horas): 320,00 €
Equipo de sonido: 250,00 €
Iluminación: 120,00 €
Desplazamiento (km): 18,00 €
Subtotal: 708,00 €
```

Es el **patrón del acumulador** del capítulo 31, haciendo dos cosas a la vez en cada vuelta: mostrar la línea e ir sumando. `subtotal` es `let` y está fuera del bucle, por las razones que viste entonces: cambia de valor, y no debe reiniciarse en cada vuelta.

32.7 Paso 5 — El descuento, el IVA y el total

Añade al final:

JAVASCRIPT · EN TU FICHERO .JS

```
// ---------- 4. Descuento, IVA y total ----------
let descuento = 0;
if (subtotal >= UMBRAL_DESCUENTO) {
  descuento = Math.round(subtotal * DESCUENTO);
}

const base = subtotal - descuento;
const iva = Math.round(base * IVA);
const total = base + iva;

console.log(`Descuento: ${formatearPrecio(descuento)}`);
console.log(`IVA: ${formatearPrecio(iva)}`);
console.log(`TOTAL: ${formatearPrecio(total)}`);
```

Tres cosas a explicar:

`descuento` empieza en 0 y es `let`. Si el subtotal no llega al umbral, el `if` no se ejecuta y el descuento se queda en 0. Si llega, cambia. Es una forma muy habitual de escribir un valor "por defecto" que una condición puede modificar.

`Math.round`. `Math` es un objeto que ya existe en JavaScript con herramientas matemáticas, y `round` ("redondear") redondea al entero más cercano. ¿Por qué hace falta? Porque `63720 * 0.21` da `13381.2` céntimos, y no existen las décimas de céntimo. Redondeando, volvemos a tener céntimos enteros, y todo el cálculo sigue siendo exacto.

El orden de las operaciones importa. El descuento se aplica antes del IVA, porque así se hace en una factura real: el IVA se calcula sobre lo que realmente se cobra. Ese orden es una *regla de negocio*: no es una cuestión técnica, sino de cómo funciona el mundo real. Saber preguntarlas es parte del trabajo.

RESULTADO

```
...
Subtotal: 708,00 €
Descuento: 70,80 €
IVA: 133,81 €
TOTAL: 771,01 €
```

Compruébalo a mano: 708 − 70,80 = 637,20; 637,20 × 0,21 = 133,81; 637,20 + 133,81 = 771,01. Cuadra.

32.8 Paso 6 — Que quede bonito

Funciona, pero las cifras no están alineadas. Para eso hay tres métodos de texto nuevos:

Método	Qué hace	Ejemplo → resultado
`.padEnd(n)`	Rellena con espacios **por la derecha** hasta tener n caracteres	`"IVA".padEnd(6)` → `"IVA "`
`.padStart(n)`	Rellena **por la izquierda**	`"5 €".padStart(6)` → `" 5 €"`
`.repeat(n)`	Repite el texto n veces	`"-".repeat(5)` → `"-----"`

Con `padEnd` todos los conceptos ocupan lo mismo, y con `padStart` las cifras quedan alineadas a la derecha, como en una factura. Como vamos a usarlo en muchas líneas, lo metemos en una función (añádela junto a las otras):

JAVASCRIPT · EN TU FICHERO .JS

```
function fila(texto, centimos) {
  return `${texto.padEnd(24)}${formatearPrecio(centimos).padStart(14)}`;
}
```

Y ahora sustituye los `console.log` por llamadas a `fila`. El programa completo queda así.

32.9 El programa completo

JAVASCRIPT · EN TU FICHERO .JS

```
// presupuesto.js — Calcula y muestra el presupuesto de un evento

// ---------- 1. Los datos ----------
const servicios = [
  { concepto: "Sesión DJ (horas)", precioUnidad: 8000, cantidad: 4 },
  { concepto: "Equipo de sonido", precioUnidad: 25000, cantidad: 1 },
  { concepto: "Iluminación", precioUnidad: 12000, cantidad: 1 },
  { concepto: "Desplazamiento (km)", precioUnidad: 30, cantidad: 60 },
];

const IVA = 0.21;
const UMBRAL_DESCUENTO = 50000; // a partir de 500 € hay descuento
const DESCUENTO = 0.1;          // 10 %

// ---------- 2. Funciones ----------
function formatearPrecio(centimos) {
  const euros = (centimos / 100).toFixed(2);
  return `${euros.replace(".", ",")} €`;
}

function calcularLinea(servicio) {
  return servicio.precioUnidad * servicio.cantidad;
}

function fila(texto, centimos) {
  return `${texto.padEnd(24)}${formatearPrecio(centimos).padStart(14)}`;
}

// ---------- 3. Las líneas del presupuesto ----------
console.log("PRESUPUESTO");
console.log("-".repeat(38));

let subtotal = 0;
for (const servicio of servicios) {
```

```
  const importe = calcularLinea(servicio);
  subtotal += importe;
  console.log(fila(servicio.concepto, importe));
}

console.log("-".repeat(38));

// ---------- 4. Descuento, IVA y total ----------
let descuento = 0;
if (subtotal >= UMBRAL_DESCUENTO) {
  descuento = Math.round(subtotal * DESCUENTO);
}

const base = subtotal - descuento;
const iva = Math.round(base * IVA);
const total = base + iva;

console.log(fila("Subtotal", subtotal));
if (descuento > 0) {
  console.log(fila("Descuento 10 %", -descuento));
}
console.log(fila("IVA 21 %", iva));
console.log("=".repeat(38));
console.log(fila("TOTAL", total));
```

TÚ ESCRIBES

```
node presupuesto.js
```

RESULTADO

```
PRESUPUESTO
--------------------------------------
Sesión DJ (horas)             320,00 €
Equipo de sonido              250,00 €
Iluminación                   120,00 €
Desplazamiento (km)            18,00 €
--------------------------------------
Subtotal                      708,00 €
Descuento 10 %                -70,80 €
IVA 21 %                      133,81 €
======================================
TOTAL                         771,01 €
```

Dos detalles del código final:

- La línea del descuento solo se muestra si hay descuento, con un segundo `if`. No tiene sentido enseñar "Descuento: 0,00 €".
- El descuento se pasa en negativo (`-descuento`) para que aparezca con el signo menos. `formatearPrecio` funciona igual con números negativos, sin tocarla.

32.10 Los comentarios como índice

Fíjate en los comentarios `// ---------- 1. Los datos ----------`. Dividen el programa en secciones que coinciden con la lista del paso 32.2. Quien abra el fichero ve de un vistazo cómo está organizado. Los comentarios buenos no repiten lo que dice el código; explican cómo está organizado o por qué se hace algo.

33. Guardarlo y publicarlo

33.1 Ver lo que ha cambiado

TÚ ESCRIBES

```
cd ~/aprendiendo-programacion
git status
```

RESULTADO

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        03-javascript/
        04-proyecto/

nothing added to commit but untracked files present (use "git add" to track)
```

Git ve las dos carpetas nuevas: los ejercicios de la Parte 3 y el proyecto. La línea *up to date with 'origin/main'* significa que, sin contar lo nuevo, tu copia está igual que la de GitHub.

33.2 Dos commits, porque son dos cosas

TÚ ESCRIBES

```
git add 03-javascript/
git commit -m "feat: añade ejercicios de fundamentos de JavaScript"

git add 04-proyecto/
git commit -m "feat: añade calculadora de presupuestos"

git log --oneline -3
```

RESULTADO

```
c4e91a7 feat: añade calculadora de presupuestos
7b2f0d3 feat: añade ejercicios de fundamentos de JavaScript
5d20b8f feat: añade fichero de prueba
```

Podrías hacerlo todo con un `git add .` y un solo commit, pero son dos cosas distintas y así el historial lo cuenta mejor.

33.3 Subirlo

TÚ ESCRIBES

```
git push
gh repo view --web
```

En el navegador verás el repositorio con sus carpetas: `01-terminal`, `03-javascript`, `04-proyecto`. Entra en `04-proyecto/presupuesto.js`: GitHub lo muestra con colores, igual que VS Code.

33.4 Un README que lo explique

Un proyecto sin explicación no le dice nada a quien lo visita. Crea un `README.md` dentro de `04-proyecto`:

```
# Calculadora de presupuestos

Programa en JavaScript (Node.js) que calcula el presupuesto de un evento
a partir de una lista de servicios: importe por línea, descuento por
volumen, IVA y total.

## Cómo ejecutarlo

    node presupuesto.js

## Decisiones

- Todos los importes se guardan en céntimos para evitar los errores
  de precisión de los decimales.
- El descuento se aplica antes del IVA, como en una factura real.
- El IVA, el umbral y el porcentaje de descuento son constantes con
  nombre, para poder cambiarlos en un solo sitio.
```

La sección de **Decisiones** es la más valiosa: demuestra que no solo hiciste que funcionara, sino que sabes *por qué* está hecho así. Es exactamente lo que busca alguien que revisa un portfolio.

TÚ ESCRIBES

```
git add 04-proyecto/README.md
git commit -m "docs: explica la calculadora de presupuestos"
git push
```

33.5 Uniendo terminal y JavaScript

Un último truco que junta la Parte 1 con la Parte 3. La salida de tu programa es texto que va a la pantalla, igual que la de `ls` o `grep`. Así que puedes redirigirla:

TÚ ESCRIBES

```
node presupuesto.js > presupuesto.txt
cat presupuesto.txt
```

Tu programa ha generado un fichero de texto con el presupuesto. Y también puedes pasarlo por una tubería:

TÚ ESCRIBES

```
node presupuesto.js | grep TOTAL
```

RESULTADO

```
TOTAL                        771,01 €
```

Tu programa se comporta como cualquier otro comando de la terminal, y se combina con todos ellos. Ese es el sentido de todo lo que has aprendido en este volumen: no son tres temas separados, son piezas de la misma forma de trabajar.

33.6 Para seguir practicando

Ideas para ampliar el proyecto por tu cuenta, de más fácil a más difícil. Cada una en su propia rama, con su commit:

1. Añade un servicio nuevo a la lista. ¿Tienes que tocar algo más del programa? (No debería: si el código está bien hecho, solo cambian los datos.)
2. Muestra la cantidad y el precio por unidad en cada línea: `Sesión DJ (horas) 4 × 80,00 € 320,00 €`.
3. Convierte el cálculo del descuento en una función `calcularDescuento(subtotal)` que devuelva los céntimos de descuento.
4. Haz que el descuento sea por tramos: 5 % a partir de 300 €, 10 % a partir de 500 €, 15 % a partir de 1.000 €. (Pista: `if`, `else if`, y cuidado con el orden de las condiciones.)
5. Muestra al final cuál es el servicio más caro, con el patrón "el mejor hasta ahora" del ejercicio 3.8.

APÉNDICES

Para tener a mano

Resúmenes de todo lo visto, para consultar mientras trabajas. Cada línea remite a lo explicado en el volumen; si alguna no te suena, vuelve a su capítulo.

- A. Chuleta de terminal
- B. Chuleta de Git y GitHub
- C. Chuleta de JavaScript
- D. Glosario
- E. Qué viene en el Volumen 2

A. Chuleta de terminal

Moverse

Comando	Hace	Cap.
`pwd`	Dónde estoy	5
`ls` / `ls -la`	Qué hay aquí / con detalles y ocultos	5
`cd carpeta`	Entrar	5
`cd ..` / `cd ~` / `cd -`	Subir / ir a casa / volver atrás	5
`open .`	Abrir esta carpeta en el Finder	5
`code .`	Abrir esta carpeta en VS Code	3

Crear, copiar, mover, borrar

Comando	Hace	Cap.
`mkdir -p a/b/c`	Crear carpetas, con las intermedias	6
`touch fichero`	Crear fichero vacío	6
`cp origen destino`	Copiar (`-r` para carpetas)	6
`mv origen destino`	Mover o renombrar	6
`rm fichero`	Borrar para siempre (`-r` para carpetas)	6
`{1..10}` `{a,b}`	Genera varias palabras antes de ejecutar	6
`*`	Cualquier texto (comodín)	6

Leer y buscar

Comando	Hace	Cap.
`cat fichero`	Mostrar entero	7
`cat > fichero`	Escribir un fichero (terminar con Ctrl+D)	7
`head -n 5` / `tail -n 5`	Primeras / últimas líneas	7
`less fichero`	Visor navegable (salir con q)	7
`grep "texto" fichero`	Líneas que contienen el texto	7
`grep -i -n -c -v -r`	Sin mayúsculas / nº línea / contar / al revés / en carpetas	7
`find carpeta`	Listar todo lo que hay dentro, a cualquier profundidad	6

Encadenar

Símbolo	Hace	Cap.
`> fichero`	Salida a un fichero (lo sobrescribe)	8
`>> fichero`	Salida al final de un fichero (añade)	8
`< fichero`	Leer la entrada de un fichero	8
`a | b`	La salida de a es la entrada de b	8
`$( comando )`	Ejecuta y pone aquí el resultado	Sol. 1.8
`sort | uniq -c | sort -rn`	Contar repeticiones, de más a menos	8
`wc -l`	Contar líneas	8

Procesos y variables

Comando	Hace	Cap.
`Ctrl+C`	Parar lo que se está ejecutando	4, 9
`lsof -i :3000`	Quién usa el puerto 3000	9
`kill PID` / `kill -9 PID`	Pedir que termine / obligarle	9
`echo $VARIABLE`	Ver el valor de una variable	10
`export NOMBRE="valor"`	Crear una variable de entorno (solo esta sesión)	10
`~/.zshrc`	Fichero que se lee al abrir cada terminal	10

B. Chuleta de Git y GitHub

Configuración (una vez por ordenador)

TÚ ESCRIBES

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
git config --global core.editor "code --wait"
brew install gh && gh auth login
```

El ciclo diario

Comando	Hace	Cap.
`git init -b main`	Convertir la carpeta en repositorio	13
`git status`	¿Cómo está todo? (úsalo siempre)	13
`git diff`	¿Qué ha cambiado exactamente?	13
`git add fichero` / `git add .`	Preparar para la foto	13
`git commit -m "tipo: mensaje"`	Hacer la foto	13
`git log --oneline`	Ver el historial compacto	13
`git log --oneline --graph`	Con el dibujo de las ramas	15

Ramas

Comando	Hace	Cap.
`git branch`	Ver ramas (* = en la que estás)	14
`git switch -c nombre`	Crear rama y entrar	14
`git switch nombre`	Cambiar de rama	14
`git merge nombre`	Traer esa rama a la actual	14
`git branch -d nombre`	Borrar rama ya integrada	14
`git merge --abort`	Cancelar una fusión con conflicto	15

Deshacer

Comando	Hace	¿Seguro si ya subiste?
`git restore fichero`	Descarta cambios sin guardar	Sí (pero se pierden)
`git restore --staged fichero`	Saca de la preparación	Sí
`git commit --amend`	Corrige el último commit	**No**
`git reset --soft HEAD~1`	Deshace el último commit, conserva cambios	**No**
`git revert hash`	Crea un commit que deshace otro	Sí
`git reflog`	Todo lo que has hecho: la red de seguridad	—

GitHub

Comando	Hace	Cap.
`gh repo create nombre --public --source=. --push`	Publicar el repositorio actual	17
`git push`	Subir tus commits	17
`git pull`	Bajar los commits de otros	17
`git clone url`	Descargar un repositorio entero	17
`gh repo view --web`	Abrir el repositorio en el navegador	Sol. 2.6
`git rm --cached fichero`	Dejar de seguirlo sin borrarlo del disco	18

Prefijos de mensajes

`feat:` algo nuevo · `fix:` corrección · `docs:` documentación · `style:` formato · `refactor:` reorganizar sin cambiar funcionamiento · `chore:` mantenimiento.

C. Chuleta de JavaScript

JAVASCRIPT · EN TU FICHERO .JS

```javascript
// ---------- Ejecutar ----------
// node fichero.js

// ---------- Mostrar ----------
console.log("texto", variable);

// ---------- Variables (cap. 22) ----------
const fijo = 10;            // no se puede reasignar (usa esta por defecto)
let cambiante = 0;          // sí se puede
cambiante = cambiante + 1;  // "recibe": calcula a la derecha, guarda a la izquierda
cambiante += 1;             // lo mismo, corto
cambiante++;                // sumar 1

// ---------- Tipos (cap. 21) ----------
"texto"   42   3.14   true   false   undefined   null
typeof valor                // qué tipo es
Number("42")                // texto a número

// ---------- Texto (cap. 23) ----------
`Hola, ${nombre}. Tienes ${edad + 1}`   // plantilla
texto.toUpperCase()  texto.toLowerCase()  texto.trim()
texto.includes("x")  texto.replace("a", "b")  texto.length
texto.padEnd(10)  texto.padStart(10)  "-".repeat(20)

// ---------- Números (cap. 24) ----------
+  -  *  /  %                // % = resto (par: n % 2 === 0)
numero.toFixed(2)           // texto con 2 decimales (solo para mostrar)
Math.round(numero)          // redondear
// El dinero, siempre en céntimos

// ---------- Decisiones (cap. 26) ----------
===  !==  >  <  >=  <=      // nunca ==
&&  ||  !                   // y, o, no
if (condicion) {
  // ...
} else if (otra) {
  // ...
} else {
  // ...
}

// ---------- Arrays (cap. 27) ----------
const lista = ["a", "b", "c"];
lista[0]   lista.length   lista[lista.length - 1]
lista.push(x)   lista.pop()   lista.includes(x)   lista.join(", ")
const copia = [...lista];   // copia independiente

// ---------- Objetos (cap. 28) ----------
const ficha = { nombre: "Ana", edad: 30 };
ficha.nombre                // leer
ficha.edad = 31;            // cambiar
const copiaFicha = { ...ficha };

// ---------- Funciones (cap. 30) ----------
```

```
function sumar(a, b) {
  return a + b;             // return entrega; console.log solo enseña
}
const doble = (n) => n * 2; // forma corta

// ---------- Bucles (cap. 31) ----------
for (const elemento of lista) { /* una vez por elemento */ }
for (let i = 1; i <= 10; i++) { /* 10 veces */ }

let total = 0;              // acumulador: fuera del bucle
for (const x of numeros) {
  total += x;
}
```

Los tres errores

Error	Significa	Causas típicas
`SyntaxError`	Está mal escrito	Falta o sobra un `+`, coma, paréntesis, comilla o llave
`ReferenceError`	No conozco ese nombre	Errata, mayúsculas distintas, usarlo antes de crearlo
`TypeError`	Eso no se puede hacer con ese valor	Reasignar un `const`, método mal escrito, algo que es `null`

D. Glosario

Término	Significado
Argumento	En la terminal, aquello sobre lo que actúa un comando. En JavaScript, el valor que le pasas a una función al llamarla.
Array	Lista ordenada de valores bajo un solo nombre. Las posiciones empiezan en 0.
Bloque	Código agrupado entre llaves `{ }`.
Booleano	Valor que solo puede ser `true` o `false`.
Bucle	Código que se repite automáticamente.
camelCase	Forma de escribir nombres compuestos: `precioTotal`.
Commit	Fotografía del proyecto guardada en el historial de Git, con autor, fecha, mensaje y enlace a la anterior.
Concatenar	Unir textos.
Conflicto	Cuando dos ramas cambian la misma línea y Git necesita que decidas tú.
Directorio	Carpeta.
Función	Bloque de código con nombre que se puede reutilizar. Recibe parámetros y devuelve un resultado.
Hash	El identificador único de un commit, calculado a partir de su contenido.
HEAD	En Git, "estás aquí": la rama o commit en el que estás.
Índice	La posición de un elemento en un array.
localhost	"Este mismo ordenador".
Merge	Fusionar una rama en otra.
Método	Acción que un valor sabe hacer consigo mismo: `texto.toUpperCase()`.
Node.js	Programa que ejecuta JavaScript fuera del navegador.
Objeto	Conjunto de datos con nombre: `{ clave: valor }`.
Opción	Modificador de un comando de terminal. Empieza por `-` o `--`.
Parámetro	Variable de una función que recibe el valor que se le pasa.
PATH	Lista de carpetas donde la terminal busca los programas.
PID	Número que identifica a un proceso en marcha.
Pipe (tubería)	El símbolo `|`: conecta la salida de un comando con la entrada de otro.

Proceso	Un programa en ejecución.
Prompt	La línea de la terminal que indica que espera una orden.
Propiedad	Dato que tiene un valor u objeto: `texto.length`. Sin paréntesis.
Puerto	Número que identifica por qué "puerta" se comunica un programa en red.
Pull request	En GitHub, petición para que revisen e integren los cambios de una rama.
Rama	Línea de trabajo paralela en Git. Técnicamente, una etiqueta que apunta a un commit.
Redirección	`>`, `>>`, `<`: mandar la salida a un fichero o leer de él.
Referencia	La "flecha" que guarda una variable cuando su valor es un array u objeto.
Remoto	Una copia del repositorio en otro sitio, normalmente GitHub. El principal se llama `origin`.
Repositorio	Una carpeta con Git: un proyecto con historial.
Ruta	La dirección de un fichero o carpeta. Absoluta (desde `/`) o relativa (desde donde estás).
Shell	El programa que lee y ejecuta lo que escribes en la terminal. En Mac, zsh.
Staging	Área de preparación de Git: los cambios elegidos para el próximo commit.
String	Texto.
Variable	Nombre que apunta a un valor.
Variable de entorno	Variable guardada por el sistema, disponible para todos los programas.

E. Qué viene en el Volumen 2

Con este volumen tienes los cimientos: sabes manejar el ordenador desde la terminal, guardar y publicar tu trabajo con Git y GitHub, y escribir programas en JavaScript con variables, decisiones, listas, fichas, funciones y bucles.

El Volumen 2 construye encima, con el mismo estilo de explicación:

- **Cómo funciona la web**: qué pasa exactamente cuando escribes una dirección en el navegador.
- **HTML y CSS**: la estructura y el aspecto de las páginas.
- **JavaScript en el navegador**: hacer que una página reaccione a lo que hace el usuario.
- **Transformar listas** con `map`, `filter` y `reduce`, que hacen en una línea lo que ahora haces con bucles.
- **Hablar con otros programas**: pedir datos a una API.
- **Tu primera página publicada**, gratis, desde tu repositorio de GitHub.

> **ANTES DE PASAR AL VOLUMEN 2**
>
> Asegúrate de poder hacer esto sin mirar el manual: crear una carpeta con estructura desde la terminal, convertirla en repositorio, hacer commits con buenos mensajes en una rama, integrarla y subirla a GitHub; y escribir un programa que recorra una lista de objetos, calcule algo con una función y muestre el resultado formateado.
>
> Si puedes, estás listo. Si alguna parte se te resiste, repite sus ejercicios: no hay prisa, y los cimientos son lo único que no se puede saltar.